

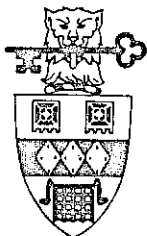
VECTOR

APL 92 Abstracts and more J:

- APL 92 News and Abstracts 49
- Hui on Verb Tables 91
- McIntyre on J with External
Data 97

... plus

- Educational Supplement 17
- Statistics Library News 33
- Graph Plotting with
I-APL/Mac 113
- Operators in Dyalog APL 119



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

Vol.8 No.4 April 1992

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible. Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette (any format). APL code can be accepted as camera-ready copy, or in workspaces from I-APL, APL*PLUS/PC, APL*PLUS II, IBM APL2/PC or Dyalog APL/386.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the editor of your intention to re-use material from VECTOR.

Membership Rates 1991-92

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£20	1	1
(Supplement for Airmail)	£8		
UK Corporate Membership	£100	10	5
Overseas Corporate	£155	10	
Sustaining	£430	50	5
Non-voting Student	£6	1	1

The membership year runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa or Mastercard at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready A5 portrait format with a 20mm blank border. Illustrations should be black-and-white photographs or line drawings. Rates are £250 per full page, £125 for half-page or less (there is a £75 surcharge per advertisement if spot colour is required).

Deadlines for bookings and copy are given under the Quick-reference Diary. Advertisements should be booked with, and sent to: Gill Smith, Brook House, Gilling East, YORK YO6 4JJ. Tel: 04393-385

CONTENTS

Guest Editorial: Now is the Time for APL	Peter Donnelly	Page 3
APL NEWS		
Quick Reference Diary		5
APL Course Diary		6
General Correspondence		7
British APL Association News		
Minutes of AGM	Anthony Camacho	9
Data Protection Act		
News from Sustaining Members	Gill Smith	11
The Education Vector	Alan Mayer	17
The Random Vector	David Eastwood	33
REVIEWS SECTION		
APL92		
Provisional Programme and Abstracts		49
Notes on Visit to St Petersburg	Anthony Camacho	57
APL Product Guide	compiled by Gill Smith	61
IBM Systems Journal (APL Edition)	reviewed by Adrian Smith	72
Byte 'Classic Languages' Article on APL	reviewed by Adrian Smith	76
APL Goes Fuzzy	previewed by Oleg Pöhl	77
FEATURING J		
Typesetting J: some Comments	R.H. Oates	82
Iverson's <i>Arithmetic</i> reviewed	K.W. Smillie	87
Verb Tables	Roger K.W. Hui	91
Using J with External Data	Donald McIntyre	97
TECHNICAL SECTION		
Hackers Corner: Reshaping your Mouse	Adrian Smith	112
Graph-plotting in I-APL/Mac 1.2	Ian A. Clark	113
Writing Operators for Dyalog APL	Phil Last	119
Author Index Vols 1-8		136
APL92 Registration Form		142
British APL Association Membership Renewal Form		144



Cocking & Drury Ltd

**Your sole UK supplier
of the entire APL*PLUS software range**

including:

APL*PLUS PC

APL*PLUS II for the 386

APL*PLUS II for UNIX

APL*PLUS VAX/VMS

APL*PLUS Mainframe

**as well as Run-Time and Run-Time Developer Kits
and all delivered ex-stock!**

**The only dealer authorised to provide maintenance and support for APL*PLUS
All software prices include 12 months free maintenance, with free upgrades (except
APL*PLUS PC).**

**We also offer a wide range of courses in APL and the most experienced APL
consultancy service in the country.**

Whatever your APL needs - call the experts first.

Cocking & Drury Ltd 180 Tottenham Court Road, LONDON W1P 9LE

Phone 071-436 9481 Fax 071-436 0524

Guest Editorial: Now is the Time for APL

by Peter Donnelly

Is APL dying? On the contrary, APL is alive and kicking and there is every reason to believe that its golden age has yet to come. "Rubbish", I hear you say, "you vendors are paid to sound optimistic; you wouldn't believe the opposition I get when I try to promote APL in my organisation. What possible justification can you have for such a claim?" Well, here is why I think I am right.

In many ways, the position of APL in recent years has been rather like that of Unix. Its adherents know it to be an extremely powerful, productive, and rewarding environment for developing applications. If only they could persuade others to take it seriously! Its critics, on the other hand, see it as arcane, obscure, unfriendly, unnecessarily complex, heavy on machine resources, and, in extreme cases, downright subversive.

Ever since I have been involved in Unix, pundits have been forecasting that NEXT year its growth would suddenly start to increase exponentially. Then, when it has only achieved a modest increase, they point to the year after, and then the year after that. This has often been called Unix's "moving hockey-stick" syndrome. Yet in the last two years the forecasts have come true, and even the most conservative of organisations, the banks, building societies and insurance companies, are moving enthusiastically into Unix. Why has Unix now succeeded after years in the doldrums? The answer I think is twofold. Firstly, thanks to the Reduced Instruction Set Computer (RISC), machines have become VERY much more powerful and dramatically less expensive. Secondly, users are demanding more and more sophisticated man-machine interfaces, which are easy to use, yet difficult to program using conventional software technology.

Let's consider the first of these reasons. Back in the mid '80s when my company first launched Dyalog APL, Unix systems were only marginally faster than personal computers. Indeed I can remember trying to explain to one sceptical customer why APL on a £200,000 1-MIP Vax 780 was no faster than on his £5,000 1-MIP PC. It was hardly surprising therefore, that Unix failed to supplant mainframe computing and was an ineffective competitor for MS-DOS, which is simpler and less power-hungry. Moreover, for many years the growth in performance of the 32-bit Motorola processors, upon which most Unix systems were built, has been matched, and occasionally bettered, by faster and faster Intel chips. The situation has recently changed dramatically, and RISC-based workstations from Sun Microsystems, Digital, IBM and Hewlett Packard have now outstripped PC technology in the race for ever-increasing performance. The differential in terms of floating-point operations, upon which most APL applications depend, is especially wide.

The impact of this technology in the financial trading arena is an interesting pointer. Once upon a time, dealing systems were entirely mainframe-based. Such applications needed the speed and communications capabilities that only mainframes could provide.

Then PCs took over as people realised that a PC on each of 100 desks gave the same performance, and a great deal more convenience, than a single mainframe computer serving 100 terminals. In latter years, Unix workstations have begun to creep in to supplement the PCs. More recently, as markets have become more and more sophisticated, traders have become increasingly reliant on advanced software to guide them; power-hungry software that gobbles memory and cpu like never before. Today in leading institutions, especially on Wall Street, the Unix workstation has ousted both the mainframe and the PC as the computing tool for day-to-day applications.

So is there a parallel for APL? I think there is. Now that machines are becoming so fast, the argument that APL is interpreted (and therefore slow) is increasingly irrelevant. Indeed, APL is one of only a handful of development tools that are capable of using today's power effectively. Does it matter that a compiled C program, that took months to write, executes in 0.1 seconds versus 1 second for the APL equivalent developed in a few days? I think not. Two years ago when we were talking about 1 second versus 10 seconds, it patently did.

The second reason for the recent success of Unix is the modern requirement for a Graphical User Interface (GUI). The GUI, pioneered by Xerox and enthusiastically championed by Apple Computer, is now an accepted part of computing technology. Although a GUI is (or should be) easy to use it is NOT easy to program. Microsoft Windows is taken for granted as a handy, friendly tool for end-user computing, yet it is notorious as one of the most difficult computer programming environments yet conceived by man. Unix GUIs are based upon the X Window System. With thousands of man-years of development behind them, they are substantially better engineered, offer genuine multi-tasking and are much easier on the applications developer. But their concepts have been developed by deep-thinking, earnest computer scientists, and are not easy to grasp. It is not surprising therefore that all the major GUIs have spawned a host of application development aids which make it relatively easy for the programmer to build an attractive user-interface. However, most of these tools fall down when you come to define the logic for your application — they just don't have the linguistic flexibility for the job. This is where APL will score. APL is, or should be, as efficient at driving a GUI as any other programming language. The fact that it is interpreted is an advantage, not a disadvantage, in this field. APL vendors are all busily working on developing means of driving GUIs, and the healthy competition amongst them will ensure a good result.

I first took to APL because it enabled me to excel at my job, yet still play golf three times a week. I was able to produce cost-effective, successful, interactive applications at a time that my non-golfing colleagues were still punching card-decks. When computing moved away from mainframes onto PCs APL still held its own, thanks to the foresight shown by STSC in realising the potential of these small machines. But APL in both environments has been held back by cost/performance considerations. Only now that MIPS and MFLOPS have become both plentiful and cheap can APL really thrive. The competition from other software products is daunting, but if the APL vendors meet the challenge, the prospects for a golden age of APL have never been better.

Quick Reference Diary 1992

Date	Venue	Event
1 May	IEE London	Utilities Day: Adrian Smith: Re-usable Code Maurice Jordan: Writing Readable Code
12 June	IEE London	AGM & Outstanding Achievement Award SigAPL Idiom Library Video of APL91 Panel "25 years of APL"
6 - 8 July	St Petersburg	APL92
25 September	IEE London	Vendor Forum
23 October	London	Field Trip to Morgan Stanley, at their new Canary Wharf offices
27 November	IEE London	Details to be Announced
August 1993	Toronto, Canada	APL93

All British APL Association meetings will be held in the IEE, Savoy Place. Nearest tube outlets: Temple or Embankment. Meetings are normally held on the 3rd Friday of the month throughout the spring and autumn.

Dates for Future Issues of VECTOR

N.B. Dates for Vector 9.1 have been advanced by 2 weeks

	Vol.9 No.1	Vol.9 No.2	Vol.9 No.3
Copy date	15th May 92	1st Sept 92	1st Dec 92
Ad booking	22nd May 92	12th Sept 92	8th Dec 92
Ad Copy	29th May 92	21st Sept 92	21st Dec 92
Distribution	July 92	October 92	January 93

APL Training Courses for 1992

compiled by Gill Smith

For confirmation of dates and/or further details please contact the vendor directly.

Level	Company	Days	Dates
Beginners	Cocking & Drury	3	Mar 17, Apr 28, June 16, Sept 8, Oct 13, Nov 17
Beginners	MicroAPL	1	Apr 9, May 28, July 23, Sept 17, Nov 12
Intermediate	MicroAPL	1	Apr 16, Jun 11, Aug 6, Sept 24, Nov 19
Advanced	MicroAPL	1	Apr 23, Jun 18, Aug 13 Oct 8, Nov 26
APL*PLUS/PC	MicroAPL	1	Mar 12, May 7, June 25, Aug 20, Oct 15, Dec 10
APL*PLUS II/PC Conversion	MicroAPL	1	Mar 19, May 14, July 9, Aug 27, Oct 22, Dec 17
Intro to APL2	MicroAPL	1	Mar 26, May 21, July 16, Sept 10, Oct 29
Statgraphics	Cocking & Drury	2	Mar 3, June 3, Sept 22, Nov 25

Vendors who wish to be included in the Training Diary, please contact Gill Smith at:

Brook House,
Gilling East,
YORK YO6 4JJ
Tel: 04393-385

General Correspondence

Comments on "ASWI Programming Standards"

From: Ray Cannon

1 February 1992

I would like to raise some objections to some of the standards as put forward in the paper "ASWI Programming Standards" by FHD v.Batenburg et al in Vector 8.2.

The main reasons for standards are human readability and maintainability; as the authors state "Good APL functions are readable". They may, as in this case, also need to define a common subset of APL that all the target machines can work with.

First, the naming of "sub-functions". Adding a System ID (SID) as a prefix to all sub-function, not only makes it far harder to maintain the workspace but, in extreme cases, may cause difficulties with the symbol table. Trying to find the name of a function from the results of `)FNS` or `⌈NL 3` is much harder if most of the functions start with the same letters. If the SID has to be included in the name, at least put it last, not first.

Next, "imported sub-functions". This naming convention for these utility functions seems silly to me. What SID is applicable for the utility function *IOTA*? Is it *SIM_IOTA* because it came from the SIMulation of whaling population growth first? Sorry, I've missed a point here, it would not be called *SIM_IOTA* but something like *SIM_CNSPIOTA* because it occurred as a sub-function to "splitting" which is a sub-function to "continuous". Surely, a true utility's name should be independent of its origin. If the origin of a function is important, then comment it, don't clutter up the name space.

The standard specifies the use of the underbar character in sub-function names, and lower case alphanumerics in global variable names. These are invalid characters in important mainframe APLs, for example, VSAPL and APL*PLUS Mainframe. Also APL2 Version 1 does not support lower-case letters. The standard should have given alternative characters to be used in these APLs.

Local variable names are often short, but it seems unnecessary to force local variable names to 4 characters or less as this could make the code less readable. Often a variable may have a "natural name". Shortening such a natural name to 4 characters may make it so cryptic that the code becomes unreadable.

In the section defining "Good APL functions:"

Point 5:

"work for missing left arguments" implies ambivalent functions. Again this is an extension to the ISO standard and may not work on all implementations.

and Point 6:

"receive data in the right argument and control in the left" is a gross over simplification. Are scalar dyadic function not "Good APL functions" because they receive data in both arguments?

I am sorry to be so negative about this, but standards like these cannot be imposed. If standards are to be implemented by programmers, they have to be useful to the author, reasonable and work. For shareware, they must work on any APL ISO conforming implementation, and should work on most of the others.

Following good standards will aid readability. Try rewriting some of the functions from Education Vector following these standards and then asking it's audience if it helps or hinders.

Ray Cannon

Minutes of the AGM of the British APL Association at the IEE on 7 June 1991

The retiring Chairman, Peter Donnelly, opened the meeting at 2pm by thanking all the members of the Committee who had worked with him during his three years as Chairman. In particular:

the Secretaries, Graham Parkhouse and Anthony Camacho;

the Treasurer, Nicholas Small;

the Education Officer, Alan Sykes, who not only edited Education Vector but also took a leading role on the ASL project and in promoting APL in education;

the Editor, Jonathan Barman, in whose first year Vector had not only maintained its standard but also come out on time. Peter has no hesitation in saying Vector is now the best APL magazine in the world;

the Activities Officer, Peter Branson, who has improved the quality of our meetings and now, with the move to the IEE, made better presentation affordable, as the Barco large screen display is always available.

Peter also called attention to the great efforts put into the APL Statistics Library. He believes the Library will fill a great need and the Association should support it and wish it great success.

Peter then summarised the state of APL in Britain:

It is under pressure in many large companies.

Software is now often bought as a fashion item.

British APL developments are as good and numerous as those in the rest of the world (he mentioned APL2/PC, Dyalog and MicroAPL II among others).

Insurance companies and Actuaries are large users but few attend our meetings.

Many excellent packages have been written in APL and this is one of its best uses (he mentioned Metapraxis, Saladin, HMW and Devon Systems) Open systems such as ADI seem old-fashioned.

Finally, Peter commended the new committee (the list was circulated with the notice of the meeting) to the Association, and in particular, the new Chairman, David Eastwood.

The Treasurer, Nicholas Small, circulated accounts for the year and the two previous years for comparison. The accounts had been done on a cash basis. He pointed out that running expenditure for 1990-1991 exceeded regular income (subscriptions and Vector advertisements), which was the reason for the subscription increase. Next year running expenditure will approximately balance the income. Project and special or extraordinary expenditure will be paid for from reserves and interest on reserves.

In answer to a questioner who asked about the variation in Vector advertising income, Nicholas explained that there was a reduction in income from 1989-90 to 1990-91 as fewer advertisements had been placed, but that the main reason for variation was late invoicing and late payment for 1988-89 advertisements which had reduced that year's income and inflated the 1989-90 receipts.

There were no other questions and the AGM was closed at 2.15pm.

Note to Members on the Data Protection Act

The British APL Association keeps its membership records on a computer and relies on conditional exemption from the Act:

"(ii) data held by unincorporated members' clubs and relating only to club members."

[Guideline to the Data Protection Act 1984 page 24]

In order to qualify for such exemption the British APL Association is required to have asked each "Data Subject" whether they object to our use of the data we hold about them and they must not have objected.

The British APL Association is also required to monitor the situation to ensure that the conditions of the exemption continue to be met.

If you object to our holding your membership details on our computer, please write to us to let us know. We shall then remove your details from our computer. The consequence of this would be that you would be omitted from all future mailings.

News from Sustaining Members

Compiled by Gill Smith

Cocking & Drury Ltd and Cocking & Drury (Software) Ltd

By the time you receive Vector we shall have shipped Version 4 of the APL*PLUS II System for the 386/486. This is the only APL product which allows you to build both Windows and DOS applications from one package.

The programmer interface to the Microsoft Windows libraries enables the building of true GUI interfaces for APL applications that run in Windows. You can:

- use standard Windows components to build interfaces
- set up APL applications so users access them through icons
- develop applications that create multiple Windows
- use the Dynamic Data Exchange (DDE) facility to share data between APL and other applications
- call third-party software through the use of a Dynamic Link Library (DLL)

In addition to this powerful new feature the new release of APL*PLUS II also includes the first evolutionary step towards compatibility with APL2; an interface to the Borland Paradox Database Manager, a Screen Interface Toolkit to aid the building of text-based application interfaces, enhancements to *WNA* which mean other programs can now call APL functions, effectively as sub-routines, and many new utilities and tools.

The price of APL*PLUS II remains unchanged. All customers on maintenance will have received their free upgrades — others can upgrade for £310 (from Version 3) or £395 (from Version 1 or 2). We are also now shipping STATGRAPHICS Plus from STSC — this uses APL*PLUS II as its calculating engine to give significant performance improvements over regular STATGRAPHICS and, because it uses all installed RAM, virtually eliminates restrictions on data-set sizes.

Our 8-page magazine 'UPGRADE' was sent out to 7,500 people early in February with news of developments with APL*PLUS and STATGRAPHICS — and Smalltalk/V. If you didn't get a copy give us a call — it's free.

We're going to be busy attending trade shows this year. The Windows Show which happened in February will be followed by Software Development '92; The Euro Object-Oriented Show and Business Computing (and if we have any energy left maybe Computers in the City too!). Most of the activity on our stand at these shows focuses on Smalltalk/V (monthly sales of which increased twelve-fold in 1991!) but we shall be showing the new APL*PLUS II as well.

On the consulting front 1992 began with a bang! Three significant new APL jobs began in the first five weeks and two more are in the offing. All of a sudden our resources are thinly stretched — and the imminent beginning of two Smalltalk jobs makes recruitment look very likely for the near future; though we are looking more for Smalltalk people than APL'ers.

The European PANAPL group of STSC dealers continues to be active — a three-day meeting in Paris at the end of February is set to look at further development of the APL*PLUS II system for GUIs and the development of more programmers' tools.

MicroAPL Ltd

We start again with news of IBM's APL software range. At the time of writing, APL2/PC Version 1.02 is being prepared for release in the UK on April 14th, and is already shipping in the USA. This new version of APL2/PC contains a number of important features which relate both to the language specification of the product and also to the development aids supplied with the product. TryAPL2, the free packaged version of APL2/PC, has been updated to use APL2/PC Version 1.02 and is available from MicroAPL on request. Both the 16 and 32 bit interpreters will run under OS/2 and Windows3 and 32-bit workspaces may be packaged to produce run-time modules (and hence free runtime licences).

At the language level new features include N-wise reduction; partition and partition with axis; index ($\square$) and index-with-axis; axis on scalar functions; selective specification (with minor exceptions); complex arithmetic in the 32-bit interpreter (with minor restrictions); the diamond ($\diamond$) statement separator; system commands may be entered in combinations of upper and lower case letters; the system commands)SIC is implemented

The environment has been enhanced by the addition of new features and APs. The Session Manager (AP120) has been enhanced with new features including a pop-up help panel and a disc-based session log. Editor 3 has been added offering function, operator and text matrix editing; DOS file editing; sorting of local name lists; flagging of errors in header, and mismatched quotes, brackets and

parentheses; flagging of duplicate labels; on-line help; hot key between editor and session manager. In addition Editor 3 offers a range of debug facilities such as automatic display of a failing line; break point setting; single step facility; stepping over called functions and operators.

AP124 has been enhanced to allow pop-up windows and other new features. Network support (AP172) has been extended to allow communication between PCs running APL2/PC with AP172. In addition facilities allow exchange of data between APL2/PC and other common programming languages. The new Universal Graphics AP (AP207) gives a simple but powerful interface to all standard IBM graphics adaptors (CGA, MCGA, EGA, VGA)

A range of new workspaces has been added to increase APL2/370 compatibility. In addition a workspace is added to read or write .WKS or .WK1 files. Printer support is extended to a wider range of IBM printers and the HP LaserJet. Facilities are supplied to provide an APL2/PC version of APE Version 2.

The development team here at MicroAPL have been busy too. Whilst APL68000/X has been under beta test at a number of RISC System/6000 sites, we have refined the shared variable interface to allow `SV0` to dynamically bind auxiliary processors. This is a slight variant on our previous mechanism for auxiliary processors and brings us more into line with mainframe conventions. The first auxiliary processor to benefit from this treatment is AP124. We have developed a widget-based implementation of AP124 which will automatically run AP124 code in its own window. We were pleasantly surprised at how good some of our old AP124 systems look. Despite putting most of our efforts into the windowing version of APL68000/X, we have not neglected dumb terminal users. Most of our Mirage and Unix dumb-terminal facilities have now been implemented on the RISC System/6000, including most of the major auxiliary processors we have developed over the years.

Dyadic Systems Limited

At the end of January, Dyadic distributed Test copies of Dyalog APL/W for Microsoft Windows 3. This new product is a true 32-bit Windows application, and contains many exciting and powerful features.

Foremost of these is support for the Windows GUI that is implemented at the interpreter level. It is not just a set of cover functions to drive C or Assembler subroutines. By implementing this support within the interpreter, Dyadic has managed to achieve a result that is fully integrated with the Dyalog APL language and development environment. A second key feature is that the GUI

support has been designed to support other systems including PM, Motif and Open Look. This means that Dyalog APL workspaces will be portable between Windows, OS/2 and Unix. You won't have to write different code to handle the different GUIs.

The system is based on three concepts: objects, properties, and events. Objects (often called "widgets") are user-interface components such as Forms, Buttons and Scrollbars. Each object has associated with it a set of properties. For example, the properties of a Form include its size, position, caption (title), border (yes or no) and so forth. Objects generate events. For example, Pushbuttons and Menuitems generate a SELECT event when the user clicks them or presses the appropriate key. To process a particular event in a particular object, you associate an APL function with it. This is termed a "callback" function, and will be invoked automatically by the interpreter whenever the event occurs. Callback functions can create new objects or change the properties of existing ones, so that a highly dynamic and responsive user-interface can be easily provided. Developing Dyalog APL applications is thus a matter of establishing a set of objects, assigning values to their properties, and attaching callback functions to various events. The need for complex control code to steer the user through an application is removed. Finally, a neat but important feature is the ability to TRACE callback functions like any others. When you TRACE your system, the callback functions pop up in windows on the screen as they are fired. You can then step through them line by line, open windows to watch your variables change, and so forth. This greatly simplifies and accelerates the task of testing and debugging a GUI application.

A second major feature of Dyalog APL/W is its interface to *Dynamic Data Exchange* (DDE). Again, this has been designed with other environments in mind and is not limited to Windows. The interface is based on Shared Variables which have been implemented in accordance with the APL standard. To share an APL variable *X* with a block of cells in an Excel spreadsheet, you simply type the expression: `'DDE:EXCEL|SHEET1' ⌈SVO 'X R1C1:R20C30'`. *X* now contains the Excel data, and assigning new values to *X* will poke them back into the spreadsheet. The DDE interface is a general one, and allows data to be passed freely between Dyalog APL/W and any other application that supports DDE. Shared variables will also be implemented in other versions of Dyalog APL using the communications protocols that are appropriate for those environments.

APL People Limited

At the time of writing (early February) things are still pretty quiet on the recruitment front, although I have noticed a marked increase in activity

compared to the three months before Christmas. I currently have several vacancies for people with APL2 experience to work in Paris, and am also looking for someone with sound APL coding skills for a position in London. Anyone who's interested, please call me on 0225 462602.

Fortunately, sales of our engineering software have been quite buoyant, with large orders from a company called Breknell Willis, who specialise in making switching gear for light railways, and from British Nuclear Fuels, plus a lot of interest from British Steel! Let's hope it continues!

HMW Trading Systems Limited

The most advanced foreign exchange dealing system in the world - now the most affordable. With over five years of evolutionary development to its credit, 4XTRA from HMW is now being launched as the leading edge technology capable of maximising profits and maintaining a competitive edge.

Developed as a front end deal capture and management system, 4XTRA provides the functionality required to monitor real time multi-currency positions, enter deals and calculate profit opportunities. With NPV profit and loss accounting as standard, the dealer is able to cover his positions accurately whilst providing his management with a real time picture of the overall risk.

4XTRA's comprehensive position keeping and deal entry screens enhanced to meet the demands of forward, spot and corporate trading together with facilities such as automatically generated spot cover deals, overnight tom-next switching and value added reporting give the trader the tools he needs to deal effectively.

And 4XTRA won't burn a hole in the IT budget. Cost is based on the number of users, and because the system is PC-based, you can forget about mainframe investment and maintenance costs - a simple PC network is all that's required. However, low cost doesn't mean low performance. Already installed in one of the world's busiest dealing rooms, 4XTRA easily manages over 4000 deals a day and can accommodate peak rates of over 60 deals a minute.

The next release of 4XTRA will use Dyalog APL-W, which will allow the system to run on other platforms (UNIX as well as DOS). A Windows version of Dyalog APL is being released later this year, with a full OS-2 version following on.

If you would like to hear more about how 4XTRA can help your FX business expand, please contact: Rob Thurston, Business Development Manager, HMW Trading Systems Limited, 1 Temple Avenue, London EC4Y 0HA. Tel: 071-353-4212; fax 071-353-3325.

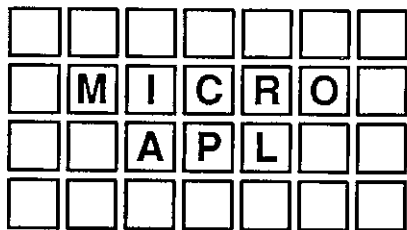
Some things are worth the wait

APL.68000 is not the only APL available for IBM's RISC System/6000. But no other APL running on the RISC System/6000 offers all these features:

- MicroAPL's acclaimed windowing interface
- Seamless integration with X-Windows
- Adherence to OSF/Motif style guidelines
- Very close compatibility with APL2/370
- Binary workspace compatibility across platforms
- High performance through MicroAPL's unique "Portable Assembler"™ technology

APL.68000 for the IBM RISC System/6000

*Fourth to announce.
First to get it right.*



MicroAPL Ltd.
South Bank Technopark
90 London Road
LONDON, SE1 6LN
Telephone: 071 922 8866
Fax: 071 928 1006

THE EDUCATION VECTOR

April 1992

Editor Alan Mayer

This Education Vector has been reprinted from VECTOR Vol.8 No.4. VECTOR is the Quarterly Journal of the British APL Association. For more information about the British APL Association, please contact: Anthony Camacho, 2 Blenheim Rd, St Albans, Herts AL1 4NR Tel: 0727-860130.

Contents

Editorial	Alan Mayer	18
APL and the Birthday Problem	Alan Sykes	20
Review of I-APL for the Macintosh	Bill McLean	23
How Much Water Under the Bridge?	Alan Mayer	25
Pass Me Another Diagonal Slice, Please!	Alan Mayer	26

Dr. Alan Mayer
European Business Management School
University College of Swansea
Singleton Park
Swansea SA2 8PP Wales, UK

Ysgol Rheolaeth Busnes Ewropeaidd
Parc Singleton,
Abertawe SA2 8PP

Tel. 0792-205678 Ext.4274 Fax: 0792 295626 JANET: MAYER@UK.AC.SWAN.MS

Editorial

It seems that another three months have passed since I wrote the last editorial for Education Vector. Looking back, I note my comment about the possible demise of the USSR - by the time anyone read it, of course, the event itself was ancient history (does anyone apart from the editor read editorials?). Perhaps I should confine my comments to things less transient than mere empires - such as the continued growth of the world-wide APL community.

On the subject of the last edition, I must apologise to Neil Sheldon for two typographical errors in our reproduction of his letter about Simpson's Rule. The quantities h_3 and h_4 should have referred to the third and fourth powers of h respectively.

I am keen to keep in close touch with developments at schools where APL is being used, and in this context I particularly welcome the contribution to this edition by Bill McLean of Dunoon Grammar School. If you are a teacher or pupil at a school using APL, or for that matter if you are considering using APL in future, please write and tell me about it. Our readers will also, I know, be pleased to see a contribution from Alan Sykes, my predecessor as editor.

Readers of Education Vector may not all be aware that it is reprinted from Vector, the official journal of the British APL Association. The sixteen-page reprint is delivered free of charge to anyone who is interested in APL. So if you are reading someone else's copy, or know of someone who would like to receive a copy, just drop me a line/phone/fax/email - I will be glad to add names to our distribution list.

The full journal, Vector, is distributed to members of the Association. If you are interested in joining, please write to Rowena Small, 8 Cardigan Road, London E3 5HU. Currently the UK private subscription is £12 per annum, for which you get four copies of Vector (144 pages per edition) and notice of meetings etc. Members and non-members are welcome to attend the meetings, most of which take place in the IEE Building (next to the Savoy Hotel) in London. Our programme for the coming months includes: (i) "A Tutorial in J" by Donald McIntyre on 27th March, (ii) a talk on the ISO APL language standards and talks by Adrian Smith and Maurice Jordan on 1st May, (iii) the Annual General Meeting, with news from SigAPL and their idiom collection (by Dick Bowman) and a video of the APL91 panel discussion "25 Years of APL" on 12th June.

Email Contact

We have received several email messages from readers in recent months, and it has been suggested that this could be a useful medium to encourage communication between APLers. We are compiling a list of email addresses, with a view to distributing news items, suggestions and APL tips to those who are interested.

If you have an email address, contact me (MAYER@UK.AC.SWAN.MS) and tell me what you think we should be doing for you.

More on *MULTIPLY*

My program *MULTIPLY* in the last edition seems to have interested a few people, but as yet I have received no contributions in that line. In response to the suggestion that I could generalise the program to test things other than multiplication, I offer the following:

```
[0]  FUN TEST VALS;RV;ANS;I;TARGET;T
[1]  RV←~1+VALS[2]+?(2×VALS[1])ρ1+-/VALS[3 2]
[2]  ANS←10
[3]  I←1
[4]  T←0 60 601~1+~4+□TS
[5]  LOOP:TARGET←(VALS[1],2)ρRV
[6]  ⍎'TARGET←TARGET[;1]',FUN,'TARGET[;2]'
[7]  'What is ',(RV[I]),FUN,(RV[I+1]),'? '
[8]  ANS←ANS,1+□
[9]  →((2×VALS[1])≥I+I+2)/LOOP
[10] 'You scored ',(RV/ANS=TARGET),' out of ',RV[VALS[1]]
[11] 'in ',(RV(0 60 601~1+~4+□TS)-T),' seconds'
```

The left argument of *TEST* is a function, such as +, -, ÷, ×, etc. and the right argument is a 3-component vector: the number of questions, followed by the minimum and maximum values to be used. For example:

```
'+' TEST 10 5 15
```

will give you 10 sums (additions) of numbers chosen from the range 5-15. This is all pretty unsophisticated stuff - if you've got something better, please share it with us!

APL and the Birthday Problem

by Alan Sykes

One of the rather beautiful probability problems that our students study is the so-called Birthday problem, which is - what is the probability that amongst N people randomly chosen, there are at least two with the same birthday?

To answer this question, as any other probability problem, assumptions have to be made. In this case, the simplest assumption to make is that for all the N people, all possible choices of their birthdays are equally likely. Let's suppose N equals 5 (although we might anticipate that the answer for N equal to 5 might be rather small).

It turns out to be easier to calculate the complementary probability that no two people have the same birthday. Hence, in line with the assumption made, the required probability will be the ratio of the total number of possible choices of N birthdays with no two being the same, to the total number of choices without restriction. The first of these is $365 \times 364 \times 363 \times 362 \times 361$ (arguing that the first person has 365 choices, the second 364 and so on). With no restriction: on whether or not the birthdays coincide, the corresponding calculation is $365 \times 365 \times 365 \times 365 \times 365$, and hence the required probability is given by:

$$365 \times 364 \times 363 \times 362 \times 361 \div 365 \times 365 \times 365 \times 365 \times 365$$

Although this APL expression is correct, it is worth pausing a moment to consider the order of execution of this statement. Working from right to left, it is clear that only one division is made, which could result in the calculation of a small number whose inaccuracy will be amplified by the subsequent multiplications. A better way to proceed is to consider the calculation as the multiplication of 5 ratios, equivalent to the APL expression:

$$(365 \div 365) \times (364 \div 365) \times (363 \div 365) \times (362 \div 365) \times (361 \div 365)$$

(The last pair of brackets is of course superfluous.)

Such an expression is somewhat tedious to type in, and (yes you have guessed it) there is a much better way to construct it which illustrates beautifully the superb array operation facilities of APL. The procedure is to construct a two-row matrix with row 1 equal to 365 364 363 362 361 and the second row 365 365 365 365 365. Then the required probability can be calculated in two stages, first by dividing row 1 by row 2, and then secondly multiplying all elements of the resulting

vector. Both operations are one of reduction - i.e. use the APL operator $/$. For the first operation the reduction has to be over the first dimension of the matrix using $\div$ hence the required expression is $\div / [1]$ or $\div /$. The second reduction is a times reduction - hence use $\times /$.

So all that is required now is the two row matrix. This is easily created using the iota function:

```
(366-15),[0.5]365
365 364 363 362 361
365 365 365 365 365
```

and so the required probability that, of 5 people, no two have the same birthday, is given by:

```
 $\times / \div / (366-15), [0.5] 365$ 
0.9728644263
```

Not surprisingly therefore there is only about a 2.7% chance of there being at least two people amongst the five with the same birthday.

Next, we ask the question - how large does N have to be before there is at least a 50% chance of at least two people having the same birthday? Or equivalently, for what value of N (instead of 5) does the above calculation give an answer less than .5?

Before answering this, note that if we require the answer for the next N , equal to 6, all we require is the product with an additional ratio - namely the ratio $360 \div 365$. Thus if we created a rather large matrix say with 50 columns, then the probabilities of no two people having the same birthday for N equal to 1,2,...50 may be calculated by replacing the times reduction $\times /$ by a times scan $\times \backslash$ (note that $\times \backslash 1 \ 2 \ 3 \ 4$ returns the vector $1 \ 2 \ 6 \ 24$). Hence the probability vector giving simultaneous answers for N equal to 1,2,...50 may be calculated using

```
PROB $\times \backslash \div / (366-150), [0.5] 365$ 
PROB[1 25]
1 0.99726 0.9918 0.98364 0.97286 0.95954 0.94376 0.92566
  0.90538 0.88305 0.85886 0.83298 0.80559 0.7769 0.7471 0.7164
  0.68499 0.65309 0.62088 0.58856 0.55631 0.5243 0.4927
  0.46166 0.4313
```

And so we see the somewhat surprising result that with N as low as 23, there is at least a 50% chance of there being at least two people with the same birthday!

An alternative way of inspecting *PROB* to find this out is to use the dyadic iota function

```
(PROB<.5)ι1
23
```

(In other words the first occurrence of 1 in the Boolean vector *PROB*<.5 occurs at the 23rd term.)

Yet another way is to count how many elements are greater than or equal to .5 and add one, i.e.

```
1++/.5≤PROB
23
```

The advantage of this approach is that it generalises to calculating several similar facts at one and the same time. Suppose that we want to know the values of *N* for which *PROB* first drops below the values .75 .5 .25. Then the expression:

```
1++/.75 .5 .25≤PROB
15 23 32
```

does what is required.

Finally, the Birthday problem gives a Mathematics teacher an opportunity to illustrate the use of logarithms and exponentials. Taking logarithms converts multiplication to addition and division to subtraction. Hence an algebraically equivalent expression may be given by:

```
PROB2+*+\\-f*(366-150),[0.5]365
```

Comparison of the two results shows that they are virtually identical to fifteen decimal places:

```
+ / |PROB-PROB2
1.00787433954252E-14
```

Review of I-APL for the Macintosh

by Bill McLean (Teacher, Dunoan Grammar School, Argyll)

John: (Our Doubting Thomas)...It's slow like the Master...

Muiry: (with the spec's).....and my P.C.

John: and uses 32k WS size. The WS's are too high-fallutin' for me and I've a better collection of idioms already!

Jason: (Our American with the economist Galbraith American/Canadian accent)
But John! If these limitations are irksome, the APL people intend you to use a "proper" APL like our APL 68000 (or one for my PC, butted in Muiry)...Besides, the notation is concise and allows lots of functions. See our Battlegame on the BBC.

Muiry: It's dearer but cheap by comparison; I like the photocopied manual. And it does support Direct Definitions!

John: Yes. I'm glad of that. I'd like to know more about the Editor window. I'm quite happy with the Del editor; I can still re-input lines there. I'm fast with that.

Teacher: Line numbers are omitted. Cut and paste are possible. Get back via "finish editing". Direct definitions are seen in the usual way...type and return.

Muiry: Do we have the equivalent of the VDU commands to build a graphics package? I saw Struan trying something with the clipboard.

Teacher: The graphics package has circles, rectangles (empty or pattern-filled), USA map or world map. With the machine code calls at the end of the book, we can use direct definitions to position rectangles, sense mouse co-ordinates. Just imagine what we can do with that! (Long pause).

John: It has a VOICE demo! How to do it looks difficult to me!

Muiry: The Session Log allows me to double-click previous items to re-input them. And we can move the cursor up and down with the arrow keys.

Teacher: Too much use of the session log seems to crash it. Better to clear rubbish now and again.

Struan: (in best bed-side manner) The Clear Window looks useful there. No bother printing on the Laser Printer. Very Macintoshy with menus, dialogues, Load, Save, Save As. No Undo as I saw somewhere.

Teacher: (QXPress?) "Save workspace?" on Quit....type a colon or something...it thinks you've changed the WS. The H.P. DeskWriter is OK so it's likely OK with an Imagewriter.

John: Except how to print a graphics window. That looks difficult to me! 73 chars per line and a warning.

Teacher: The BBC WS's are compatible (except graphics) and can be transferred to the Mac and laundered (given an icon) with the Exchange program down in the Maths dept. where they have a Mac and BBC side by side.

Muiry: I like the Nails, Rounds etc. for finding symbols.

Jason: But Muiry! Have you not learned your mnemonics yet! And you with a PC too. And did you see the customisable menu: Control. That should be a convenience for newcomers.

Struan: A real give-away! Like it better than Comal, but some will say "horses for courses", that a Mac application would be better in some other language.

Jason: Well, I'm amazed at how powerful it is. We can do things with it that would be difficult for a £300 database!

Struan: Yes, the Text window is like a wordprocessor; no word wrapping, needs CR, but both upper and lower case with Ctrl K or switch back to APL style with Ctrl K.

Muiry: I'm not worried about System 7! but will it wear?

John: Ah! But what about WINDOWS!

Jason: Nothing that the author couldn't fix! If someone paid him for about 3 man-months of work! Really, our Basic friends just don't know what they are missing. Imagine a future priest like me doing APL on the side!

How Much Water Under the Bridge?

by Alan Mayer

The *MULTIPLY* function listed in the Editorial for January contained the following expression twice:

```
0 60 60 1~1+~4+□TS
```

This was used to calculate the amount of time taken by the user to perform the multiplication test. There are several ways of calculating elapsed time in APL, some easier to use than others. In many systems, the Accounting Information (□AI) system function includes (often in the second component) an "elapsed seconds since start of session" value. Where present, this provides by far the easiest means of calculating elapsed time, but unfortunately it is not always available, and even if it is, it may measure milliseconds rather than seconds.

In Education Vector, we like to use methods that work with as many APL systems as possible. The Time Stamp (□TS) system function provides the most universal method of calculating elapsed time. This produces a seven-component vector: year, month, day, hour, minute, second, millisecond. Of course, it relies on your system clock being right!

MULTIPLY was concerned to present time in seconds, and so the millisecond component was ignored. Thus `~4+` selects the last four components of `□TS` and `~1+` drops the final one, leaving hours, minutes and seconds. The Represent function (`⊥`), with left argument `0 60 60`, converts the result to seconds. Note that the lengths of the two arguments of `⊥` must match, so the first component of the left argument can be any value, but it must be present. An alternative approach is to use a scalar left argument - since both minutes and seconds are measured in 60s, a left argument of 60 will do the trick. This is equivalent to a vector value of `60 60 60`.

Subtracting the time the test is started from the time the test is completed, using this method of calculating time, will give the time taken (provided the test does not start before midnight and end after midnight!).

If you are worried about the problem of midnight, put `86400 ⊥` in front of the final calculation (`86400` is $60 \times 60 \times 24$). Finally, of course, if you want to measure time more accurately, the m'sec component can be kept in. Just use:

```
0 60 60 10001~4+□TS.
```

Pass Me Another Diagonal Slice, Please!

by Alan Mayer

The three primitives ϕ , Θ and Φ are well-known as powerful tools, particularly in the field of matrix algebra. In its monadic form ϕ will reverse the order of the elements of a vector, or the rows of a matrix, and in its dyadic form it makes rotation of elements easy. Θ redirects the operations of ϕ so that they act down the columns of a matrix rather than along the rows (strictly speaking we should refer here to the first and last axes of a multi-dimensional matrix, but let's keep things simple for as long as we can!). Finally, the monadic form of Φ is straightforward enough - it gives us the transpose of a matrix. But what about its dyadic form?

First the rules: the left argument of Φ must be a vector whose length is equal to the rank (the number of dimensions) of the right argument. The rank of an object X can be found using $\rho\rho X$. The elements of this vector must have values taken from 1,2,3,...,r where r is the rank in question. We seem to be getting into deep water already, so let's start with a few examples...

Two Dimensional Matrices

For an ordinary matrix of rank 2, there are only a few options:

```

X←2 4ρ18
X
1 2 3 4
5 6 7 8
1 2ΦX
1 2 3 4
5 6 7 8

```

This particular form of dyadic transpose clearly has no effect whatsoever on the elements of X .

```

2 1ΦX
1 5
2 6
3 7
4 8

```

Here the effect is exactly the same as that of the monadic form ΦX :

```

1 1 @ X
1 6

```

Finally, a result which may be of some use - this can be used to extract the elements of the main diagonal (i.e. starting at the top left corner and working down through the matrix diagonally).

Perhaps it will be instructive to look at what happens to each individual element of the matrix in these transpositions.

```

1 2 @ X is equivalent to Rij = Xij      (identity)
2 1 @ X is equivalent to Rij = Xji      (transpose)
1 1 @ X is equivalent to Rij = Xii      (diagonal)

```

In other words, each element of R (the result) can be obtained by giving the relevant values to i and j . Note the correspondence between the left argument of the APL expression and the subscripts of X : $(1\ 2) \sim (ij)$ $(2\ 1) \sim (ji)$ $(1\ 1) \sim (ii)$. This is the key to understanding the use of dyadic transpose with matrices of rank 3 or more.

Three Dimensional Matrices

As an example, consider the matrix Y :

```

Y←2 3 4ρ124
Y
1 2 3 4
5 6 7 8
9 10 11 12

13 14 15 16
17 18 19 20
21 22 23 24

```

Y is a matrix of 2 levels, 3 rows and 4 columns. Although it is displayed as two separate matrices, it is best to imagine it as a three-dimensional object with the first level on top of the second level, so that, for example, 3 has 15 directly below it. Y has rank 3 (see $\rho\rho Y$), and so the dyadic transpose can be made up of the numbers 1, 2 and 3.

For example:

```

      2 1 3QY
1    2 3 4
13 14 15 16

      5. 6 7 8
17 18 19 20

      9 10 11 12
21 22 23 24

```

This can be seen as an interchange of levels and rows. The first level of the result is made up of the first rows of Y , and similarly for the second and third levels.

We can also say " $2 \ 1 \ 3QY$ is equivalent to $R_{ijk} = Y_{jik}$ ". To investigate other transpositions, try:

```

3 2 1QY (Interchange level and columns:  $R_{ijk} = Y_{kji}$ )
1 3 2QY (Interchange rows and columns:  $R_{ijk} = Y_{ikj}$ )

```

If we confine the left argument to the numbers 1 and 2 only, we get a selection of elements usually called a "diagonal slice":

```

      1 1 2QY
1    2 3 4
17 18 19 20

```

The result is a matrix of rank 2, made up of the first row of level 1 of Y and the second row of level 2 of Y . Remembering the three-dimensional nature of Y , this can be seen as the result of taking a slice out of Y by cutting through it diagonally. The transposition can also be represented by $R_{ij} = Y_{ij}$.

Other diagonal slices are:

```

      1 2 1QY
1    5 9      (1st column 1st level)
14 18 22     (2nd column 2nd level)

      1 2 2QY
1    6 11     (main diagonal, 1st level)
13 18 23     (main diagonal, 2nd level)

```

Applications with the Outer Product

Suppose we want to add a vector V to each row of a matrix M . For example:

$$\begin{array}{rcl}
 M & + & 3 \ 4 \ 0 \ 1 \ 2 \\
 M & & \\
 \begin{array}{cccc}
 1 & 2 & 3 & 4 \\
 5 & 6 & 7 & 8 \\
 9 & 10 & 11 & 12
 \end{array} & & \\
 V & + & 7 \ 9 \ 1 \ 2
 \end{array}$$

The result we are looking for is a matrix:

$$\begin{array}{cccc}
 8 & 11 & 4 & 6 \\
 12 & 15 & 8 & 10 \\
 16 & 19 & 12 & 14
 \end{array}$$

Calling the result R , where do we get the component R_{ij} ? Clearly we must start with the original value M_{ij} and add the relevant component of V . For the j th column, the relevant value is V_j . So, in subscript notation we want:

$$R_{ij} = M_{ij} + V_j$$

For example, to get the 8 in the 2nd row, 3rd column of the result, we take the 2nd row and 3rd column of M (7) and add the 3rd component of V (1). What, you may ask, has this to do with dyadic transpose?

First we consider the outer product:

$$\begin{array}{rcl}
 P & + & M \circ \cdot + V \\
 P & & \\
 \begin{array}{cccc}
 8 & 10 & 2 & 3 \\
 9 & 11 & 3 & 4 \\
 10 & 12 & 4 & 5 \\
 11 & 13 & 5 & 6 \\
 \\
 12 & 14 & 6 & 7 \\
 13 & 15 & 7 & 8 \\
 14 & 16 & 8 & 9 \\
 15 & 17 & 9 & 10 \\
 \\
 16 & 18 & 10 & 11 \\
 17 & 19 & 11 & 12 \\
 18 & 20 & 12 & 13 \\
 19 & 21 & 13 & 14
 \end{array} & &
 \end{array}$$

In subscript notation this is $P_{ijk} = M_{ij} + V_k$, the right hand side of which is not very different from the formula for R above. In fact R is obtained by setting j

equal to k . Looking at the effect this has on the left hand side of the equation, we get the result P_{ij} . This is the diagonal slice obtained using $1 \ 2 \ 2 \ \& \ P$. In other words, the whole result can be obtained using:

```

      R + 1 2 2 & M . . + V
      R
    8 11  4  6
   12 15  8 10
   16 19 12 14

```

Of course, there are lots of easier ways to add a vector to every row of a matrix, so we shall now consider a slightly more complex problem which has a similar solution. Suppose we have a matrix Q and two vectors $UPPER$ and $LOWER$, specifying upper and lower bounds for each of the rows of Q . We want to replace the values in Q , which fall outside the specified bounds, by the relevant bound value. For example:

```

      Q+4 4p1 5 12 2 4 3 8 5 7 1 6 9 3 4 1 6
      Q
    1 5 12 2
    4 3  8 5
    7 1  6 9
    3 4  1 6

      UPPER+9 7 8 5
      LOWER+3 4 6 2

```

The upper and lower bounds specified for the first row of M are 9 and 3 respectively, so entries 1 and 2 must be replaced by 3 and entry 12 by 9, and so on. The required result is:

```

    3 5 9 3
    4 4 7 5
    7 6 6 8
    3 4 2 5

```

We shall apply the upper bounds first. For each entry in Q we require the minimum ($\downarrow$) of the entry and the relevant value in $UPPER$. When dealing with Q_{ij} , the relevant value in $UPPER$ is the i th component, $UPPER_i$. In other words, $RES1_{ij} = Q_{ij} \downarrow UPPER_i$. As in the previous example we start with an outer product:

```

      PROD+Q . . \ UPPER

```

In index notation, this has the formula $PROD_{ijk} = Q_{ij} \cup UPPER_k$. To produce the required diagonal slice, we need $PROD_{ijj}$, which is obtained by:

```

RES1←1 2 1QPROD
RES1
1 5 9 2
4 3 7 5
7 1 6 8
3 4 1 5

```

Here, all the values which exceeded the relevant upper bound values have been replaced. Now we need to do a similar exercise on the lower bounds. The maximum ($\cap$) between the existing value and the relevant lower bound is given by $RES2_{ij} = LOWER_i \cap RES1_{ij}$. Consider the outer product:

```
PROD2←LOWER ∘. [ RES1
```

This has the formula $PROD2_{ijk} = LOWER_i \cap RES1_{jk}$. So, in APL:

```

RES2←1 1 2QPROD2
RES2
3 5 9 3
4 4 7 5
7 6 6 8
3 4 2 5

```

In one line, therefore, the complete result may be obtained using:

```

1 1 2QLOWER ∘. [ 1 2 1QQ ∘. [ UPPER
3 5 9 3
4 4 7 5
7 6 6 8
3 4 2 5

```

Finally, if you really want to show off, you can tackle this problem in four dimensions. Note the form of our two subscript equations:

$$RES1_{ij} = Q_{ij} \cup UPPER_i$$

$$RES2_{ij} = LOWER_i \cap RES1_{ij}$$

These may be combined in one equation:

$$RES2_{ij} = LOWER_i \cap Q_{ij} \cup UPPER_i$$

This suggests that we should look at the four-dimensional matrix produced by the double outer product:

$$MAT4D \leftarrow LOWER \circ. [Q \circ.] UPPER$$

The "slice" required (can you slice a four dimensional object?) is indicated by the subscript in the right hand side of the equation:

```

      1 1 2 1 MAT4D
    3 5 9 3
    4 4 7 5
    7 6 6 8
    3 4 2 5

```

So an alternative one-line solution is:

$$1\ 1\ 2\ 1 \leftarrow LOWER \circ. [Q \circ.] UPPER$$

You will note that I have carefully refrained from displaying *MAT4D*. The standard advice to climbers is "Don't look down! - You know what's below you, you don't have to see it." The most frightening thing about n-dimensional matrices is their appearance. Let's leave that side of things to our APL interpreters - that's what they are there for.

THE RANDOM VECTOR

The Newsletter of the APL Statistics Library

Editor David Eastwood

April 1992

Contents

Editorial	David Eastwood	34
ASL Ordering and availability	David Eastwood	35
BATS	David Eastwood	36
Simulating Sampling Distributions	Dietrich Trenkler	37
Implementing Cardano's Rule	Dietrich Trenkler	43

Editorial

As a slight change from our normal articles restricted to considerations of the ASL software, this issue of Random Vector features two articles by Dietrich Trenkler who is involved in statistics education at the Economics department at the University of Osnabrück in Germany. He uses APL in a course which complements and gives a deeper treatment to material covered in a one year introductory course in Statistics. His article 'Simulating Sampling Distributions' hints at the lively nature of the course and students. Dietrich also holds regular courses in APL for students who will be using APL when working towards their diplomas.

As far as the ASL software is concerned, this issue of Vector should be accompanied by an order form for the first two volumes of ASL to be ready for distribution. We cover the presentation made by Alan Sykes at a recent BAA meeting regarding availability and distribution of ASL software. If you want to order ASL software, you can also contact Alan Sykes at the address below.

Finally, I guess it was inevitable that errors would creep into Random Vector and the last issue found us wrongly attributing the ownership of the BATS software. The error was compounded by the fact that BATS is actually correctly listed in Vector's product guide! By way of apology, I have included some brief details of the BATS software to put the record straight.

As usual, I give contact addresses for the ASL Management Committee:

Chairman: Jake Ansell, Business Studies, University of Edinburgh,
William Robertson Building, 50 George Square,
Edinburgh EH8 9JY
031-650 3806

Deputy Chairman: Alan Sykes, European Business Management School,
University of Wales, Swansea, Singleton Park,
Swansea, SA2 8PP
0792-295296

BAA Representative: David Eastwood, MicroAPL Ltd, South Bank Technopark,
90 London Road, London SE1 6LN
071-922 8866

BAA Representative: John Searle, 4 Hawks Mews, Greenwich,
London, SE10 8RA
081-858 6811 (home)

ASL Status Report British APL Association Meeting 14th February 1992

Alan Sykes gave a short update on the status of the ASL project at the BAA meeting on February 14th 1992. Alan brought some copies of the printed documentation for two volumes of software (Regression and Basic Graphics) and announced that order forms for this software would be distributed with the next issue of Vector. In the short term, Alan will be coordinating distribution from Swansea although ultimately the BAA Administration will take over this activity. Alan then covered the status of each of the first set of ASL volumes:

ASLGREG

The Regression and Generalised Linear Models volume is complete and ready for distribution. Versions are available for:

APL*PLUS
APL2/PC
APL.68000
Dyalog/PC

and Alan thanked all those who had helped in preparing the different versions of this volume. Alan maintained that it was a sheer coincidence that the first ASL volume ready for distribution was the one he had the greatest part in writing.

ASL Basic Graphics

The Basic Graphics volume, prepared by Tony O'Hagan, is available for Dyalog/PC. The version for APL*PLUS is nearing completion, but Alan Sykes asked the meeting (and also readers of Vector!) for assistance in preparing versions for APL2/PC and APL.68000.

ASLBSTAT

The third volume of the initial ASL set, Basic Statistics, is still not ready for distribution as the documentation is still being finalised.

BATS

In the last issue of Vector, the program BATS - Bayesian Analysis of Time Series - was wrongly attributed. BATS is produced by and copyrighted to Andy Pole, Mike West and Jeff Harrison (see the entry in the Product Guide).

For those readers who are unfamiliar with BATS, I thought it might be useful to include a little detail on the software and the authors.

Andy Pole is visiting professor at, and Mike West is Director of, the Institute of Statistics and Decision Sciences at Duke University in North Carolina, USA; Jeff Harrison is professor of statistics at Warwick University.

BATS is an interactive system for modelling, analysing, and forecasting univariate time series using dynamic linear models. A wide range of dynamic models may be selected, including components for polynomial trend; seasonal patterns; exogenous variable regression; and variance laws. The analysis features include reference or user-defined initialisations, forecast monitoring, subjective intervention, step ahead forecasting, and smoothing. Non-stationarity, missing data, and outlying observations are routinely handled by the Bayesian paradigm. BATS is graphically oriented. Tabulated results may be written to text files.

BATS was originally developed in APL*PLUS/PC but has now been redeveloped in C (with the same 'look and feel') to improve performance. Both versions are available from the authors but support and future development will be restricted to the C versions.

Simulating Sampling Distributions

by Dietrich Trenkler, University of Osnabrück, Germany

This paper aims at giving some impressions of the power of an innocent looking *APL*-function called *STATISTICS* which has been written to help in clarifying difficult facts from the realm of theoretical Statistics. I will exemplify its usefulness in the context of two topics covered in most introductory text books such as [1].

Two Problems

Most students of the Economics department of our university hate Mathematics. So an instructor is well advised not to turn his back to write down a proof.¹ So why not take a simulation approach?

For instance, understanding the concepts of point estimation definitely is a goal but they are best understood by *seeing* them in action. To be specific we ask two questions:

- (i) How can we get an idea of the concept of (un)biasedness or efficiency of point-estimators?
- (ii) A well-known theorem of Mathematical Statistics states that $\bar{X} = \sum_{i=1}^n X_i$ and $S^2 = \frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2$ are independent if and only if the X_i are normally distributed, [1], p. 243. How can we get conceivable evidence of this fact without clinging to (incomplete) stringent proofs?

The Simulation Approach

All the students have to comprehend is how to generate random numbers from a specified distribution. I favor the table-look-up method for discrete distributions and the inversion method for continuous distributions, [2]. Both can be implemented in *APL* quite easily if

- there is a function *UNIFORM* to generate uniformly distributed random numbers from (0,1).
- there is a function to compute values $F(x) = P(X \leq x)$ of the distribution function of a discrete distribution.
- there is a function to compute $F^{-1}(u)$ where $F(x) = P(X \leq x)$ is the distribution function of a continuous distribution.

¹For me the word *proof* is tantamount to *turmoil*.

For instance, let *DISTRBINOM* compute $F(x)$ for a binomial distribution $B(n, p)$. Then

```
50 TABLELOOKUP X, [1.1] 20 0.3 DISTRBINOM X-0, 20
```

will generate 50 random numbers from a $B(20, 0.3)$ -distribution. Here is a listing of *TABLELOOKUP*.

```
VR←N TABLELOOKUP XP
[1] R←(ΦXP[;1])[+f(ΦXP[;2])•.≥UNIFORM N]
V
```

On the other hand, let *IEXP* compute $F^{-1}(u)$ for an exponential distribution $\mathcal{E}(\lambda)$:

```
VR←L IEXP U;X;P
[1] R←-(÷L)×#1-U
V
```

Then

```
2 IEXP UNIFORM 100
```

generates 100 random numbers from an $\mathcal{E}(2)$ -distribution. *UNIFORM* looks as follows:

```
VR←AB UNIFORM N
[1] A(0=□NC 'AB')/'AB-0 1'
[2] R←AB[1]+(-/ΦAB)×(?Np~2+R)÷~1+R÷2÷31
V
```

These are the main ingredients to realize the simulation approach for one or several statistics. The argument of the monadic function *STATISTICS* consists of essentially three parts:

- The first part describes how to generate samples of size n .
- The second part determines the number M of samples and the sizes n thereof.
- The rest describes the way(s) how to compute k statistic(s) from the samples.

```

VR←STATISTICS par;n;m;i;j;stat;sp;nv;distr;nnv
[1] par←box par
[2] distr←par[1;]
[3] nnv←pnv←par[2;]
[4] m←1↑pstat+2 0 ↓par
[5] R←0 0 pi+1
[6] M1:n←nv[i] 0 sp←distr 0 j+1
[7] M2:R←R,stat[j;] 0 (m≥j+1)/M2
[8] ←(nnv≥i+1)/M1
[9] R←((pnv),(pR)÷pnv)pR
▽

```

Its output is an (M,k) -matrix where row i is the result of the k statistics administered to sample i . *box* should be familiar to readers of *VECTOR*. It is modified version of *BOX* taken from [4] to allow for a left argument describing the selector.

```

VR←S box X;I;J;K;M;REP
[1] ←(2#NC 'S')/'S' ' ' ' '
[2] I←(X←1↑X)/ρX←X,S
[3] X[I]←' '
[4] J←1+I-0,-1↓I
[5] K←(M↑I/J)-J
[6] REP←(ρX)ρ1
[7] REP[I]←K
[8] R←((ρI),M)ρREP/X
▽

```

Conducting the Simulations

Let us return to the problems mentioned above.

(i) Assume that a population is distributed according to the density

$$f(x) = \begin{cases} \frac{\theta}{x^2}, & \theta \leq x, \\ 0 & \text{else,} \end{cases}$$

and let $X_1, \dots, X_n$ be a random sample from that distribution. It is not hard to see that

- the maximum-likelihood estimator of θ is

$$\hat{\theta}_1 := \min\{X_1, \dots, X_n\}.$$

- an estimator for θ based on the method of moments is

$$\hat{\theta}_2 = \frac{1}{\frac{2}{n} \sum_{i=1}^n \frac{1}{X_i}}.$$

This estimator is constructed from $E[1/X] = 1/(2\theta)$.

Now that there are two estimators for θ which is the one to be preferred? To get an insight into their distributional properties a simulation seems appropriate. For $\theta = 1$ note that $F^{-1}(u) = 1/(1-u)$. Accordingly

```
DIS←'÷1- UNIFORM n ÷'
SMPL←'(100p5),100p20 ÷'
STATS←'1+2×MEAN ÷sp ÷ [sp'
ST←STATISTICS DIS,SMPL,STATS
BOXANDWHISKER (100 2†ST),100 04ST
```

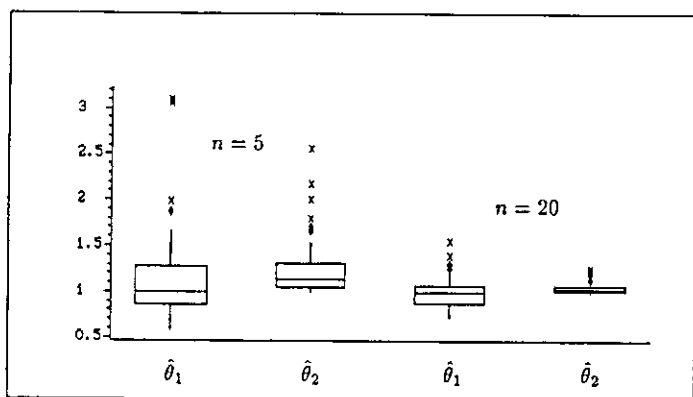


Figure 1: Comparing the performances of $\hat{\theta}_1$ and $\hat{\theta}_2$ for $n = 5$ and $n = 20$.

Figure 1 displays the result of the comparing box-and-whisker plots. Several aspects of point estimation can now be faced:

- $\hat{\theta}_2$ is unbiased while $\hat{\theta}_1$ is not. Of course, since $E[\hat{\theta}_1] = n/(n-1) > 1$ there is an obvious modification of $\hat{\theta}_1$ to $\hat{\theta}_3 = (n-1)\hat{\theta}_1/n$ to yield a second unbiased estimator.
- $\hat{\theta}_2$ reveals more variability than $\hat{\theta}_1$. This is measured from the widths of the boxes. Alternatively one could display box-and-whisker plots of $|1 - \hat{\theta}_j|$ or $(1 - \hat{\theta}_j)^2$ by

```
BOXANDWHISKER |1-(100 21ST),100 04ST
BOXANDWHISKER (1-(100 21ST),100 04ST)*2
```

to introduce the concepts of mean absolute error and mean square error.

- The performances of both estimators become better by increasing the sample sizes from $n = 5$ to $n = 20$.

(ii) Simulating to convey what this theorem says is now straightforward:

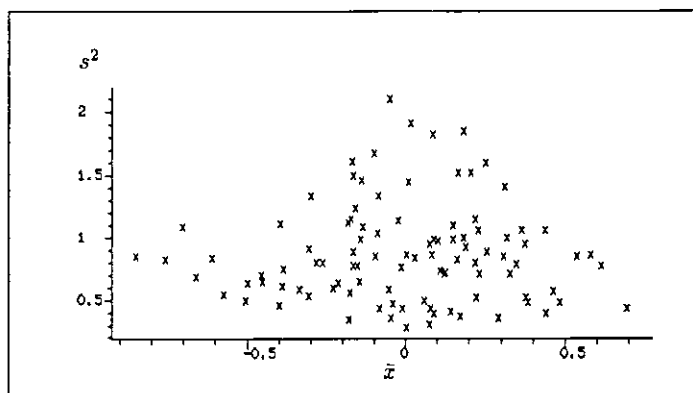


Figure 2: Scatter plot of s^2 versus $\bar{x}$ based on 100 samples of size $n = 10$ from an $\mathcal{N}(0,1)$ -distribution. Is there any visible relationship between s^2 and $\bar{x}$?

```
ST ←STATISTICS 'BOXMUL n ◇ 100p10 ◇ MEAN sp ◇ S2 sp'
```

BOXMUL uses the well-known algorithm to generate independent random numbers from a standard normal distribution, [2], p. 235. Figure 2 displays a scatter plot of $ST[;2]$ versus $ST[;1]$. What happens if we assume that the population is exponentially distributed with $\lambda = 1$?

```
ST ←STATISTICS '1 IEXP n ◇ 100p10 ◇ MEAN sp ◇ S2 sp'
```

Figure 3 displays a scatter plot of $ST[;2]$ versus $ST[;1]$. This experiment has also been described in [3].

Concluding Remarks

The versatility of this approach is amazing. Now it takes only a few key-strokes to grasp the meaning of the Central Limit Theorem. Even the ideas of jackknifing or the bootstrap can be learned from simulation experiments. Generalizing *STATISTICS* to cover simulations in the context of regression

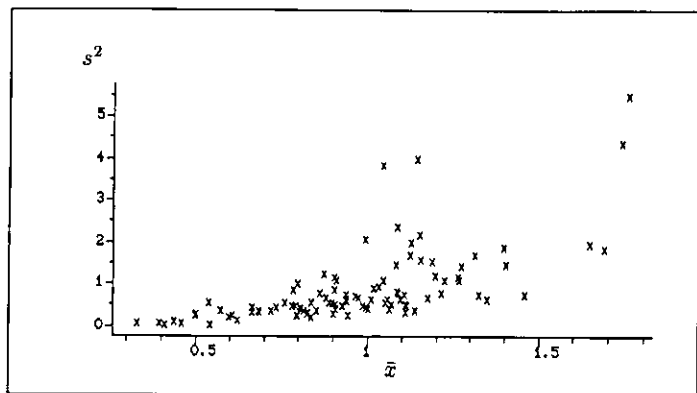


Figure 3: Scatter plot of s^2 versus $\bar{x}$ based on 100 samples of size $n = 10$ from an exponential distribution with $\lambda = 1$. Is assuming independence appropriate?

models should be obvious. With this program questions concerning the robustness or resistance of ordinary least squares estimation can be tackled.

Once again, it is the ease of answering *What if*-questions that glorifies enlistment in the *APL*-navy.

References

- [1] Mood, A.M., Graybill, F. A, Boes, D.C. (1974): *Introduction to the Theory of Statistics*, 3rd edition, McGraw-Hill, New York.
- [2] Morgan, B.J.T. (1984): *Elements of Simulation*, Chapman and Hall, London and New York.
- [3] Naeve, P., Trenkler, D., Wolf, P. (1991): *Making the Teaching of Statistics Roar*, Computational Statistics Quarterly 4, 325-353.
- [4] Sykes, A. (1990): *Doing Useful Things with Strings*, VECTOR 6(4), 24-29.

Implementing Cardano's Rule with an Application to Approximating the Inverse of a Standard Normal Distribution

by Dietrich Trenkler, University of Osnabrück, Germany

A cubic equation with derived forms

Let us be given the equation

$$Ax^3 + Bx^2 + Cx + D = 0, \quad A \neq 0.$$

Dividing by A leads to its *normal form*

$$x^3 + ax^2 + bx + c = 0, \quad ,$$

where

$$a = B/A, \quad b = C/A, \quad c = D/A.$$

Substituting $x = y - \frac{a}{3}$ finally renders the *reduced form*

$$y^3 + py + q = 0$$

where

$$p = b - \frac{a^2}{3}$$

and

$$q = c + \frac{2a^3 - 9ab}{27}.$$

Cardano's rule

Define the *discriminant* by

$$\Delta = \left(\frac{q}{2}\right)^2 + \left(\frac{p}{3}\right)^3.$$

Further let

$$u = \sqrt[3]{-\frac{q}{2} + \sqrt{\Delta}}$$

and

$$v = \sqrt[3]{-\frac{q}{2} - \sqrt{\Delta}}.$$

Depending on the sign of Δ Cardano's rule distinguishes three cases to solve the reduced form, [1].

- $\Delta < 0$ (*casus irreducibilis*): The three real solutions are

$$y_1 = 2\sqrt{\frac{|p|}{3}} \cos \frac{\phi}{3}$$

$$y_2 = -2\sqrt{\frac{|p|}{3}} \cos \left(\frac{\phi - \pi}{3} \right)$$

$$y_3 = -2\sqrt{\frac{|p|}{3}} \cos \left(\frac{\phi + \pi}{3} \right)$$

where

$$\phi = \arccos \left(\frac{-\frac{q}{2}}{\sqrt{\left(\frac{|p|}{3}\right)^3}} \right)$$

- $\Delta = 0$: There are three real solutions, two of them being identical. They are given by

$$y_1 = u + v$$

$$y_{2,3} = -\frac{u + v}{2}$$

- $\Delta > 0$: There is one real solution and two complex ones. They are given by

$$y_1 = u + v$$

$$y_2 = -\frac{u+v}{2} + \frac{\sqrt{3}}{2}(u-v)i$$

$$y_3 = -\frac{u+v}{2} - \frac{\sqrt{3}}{2}(u-v)i$$

The program *CARDANO* can be used to get the solutions of any cubic equation.

Approximating $\Phi^{-1}(p)$

Let Φ be the distribution function of a standard normal distribution, i.e.

$$\Phi(z) = \int_{-\infty}^z \frac{1}{\sqrt{2\pi}} e^{-t^2/2} dt.$$

Recently, Derenzo [2] and Lin [3] considered approximations to $\Phi(z)$. Derenzo established the approximation

$$\Phi(z) \approx \begin{cases} \frac{1}{2} e^{-p_D(-z)} & , \quad z \leq 0 \\ 1 - \frac{1}{2} e^{-p_D(z)} & , \quad z > 0 \end{cases}$$

where

$$p_D(t) = \frac{83t^3 + 351t^2 + 562t}{165t + 703},$$

see also [4].

Lin's method amounts to

$$\Phi(z) \approx \begin{cases} \frac{1}{2} e^{-p_L(-z)} & , \quad z \leq 0 \\ 1 - \frac{1}{2} e^{-p_L(z)} & , \quad z > 0 \end{cases}$$

with

$$p_L(t) = 0.717t + 0.416t^2.$$

Its advantage is the fact that a simple formula is available to approximate z_p with $\Phi(z_p) = p$. It is given by

$$z_p \approx \frac{-0.717 + \sqrt{0.514089 + 1.644a(p)}}{0.832} = z_{pL}$$

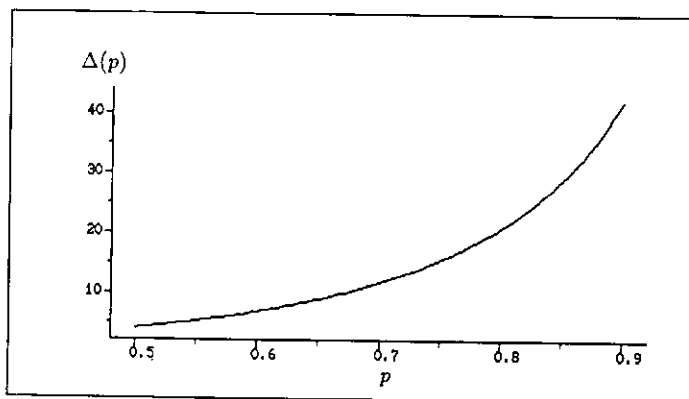
where $a(p) = -\ln(2(1-p))$.

Inverting Derenzo's formula is not so obvious, since for $p > 0.5$ we have to solve

$$83x^3 + 351x^2 + (562 - 165a(p))x - 703a(p) = 0.$$

Let $\Delta(p)$ be the discriminant of to this equation. To see if $\Delta(p) > 0$ we used the program *DERTEST*. A plot of $\Delta(p)$ versus p shows that $\Delta(p) > 0$ for all p . Hence there exists a unique approximation based on Derenzo's formula.

PLOT P VS DERTEST P+100 AEKL .5 .9



The program *IDERENZO* computes the approximation z_{pD} for a vector of values $p \in (0, 1)$.

Although inserting Derenzo's values in Cardano's rule leads to a very clumsy formula its implementation is straightforward and yields excellent results

as can be seen by comparing it to Lin's method. The following table corresponds to that in [3] although values p greater than .9995 have been taken into account. It reveals the fact that the absolute relative error rz of the new approximation is never greater than 0.34% for all values of p .

p	z_p	z_{pD}	z_{pL}	rz_D (%)	rz_L (%)
0.500000	0.000000	0.000000	0.000000	0.000000	0.000000
0.600000	0.253347	0.253261	0.269179	0.033907	6.249075
0.700000	0.524401	0.524515	0.542005	0.021740	3.357063
0.800000	0.841621	0.841877	0.854404	0.030447	1.518867
0.900000	1.281552	1.281774	1.285662	0.017374	0.320770
0.950000	1.644854	1.644962	1.643760	0.006618	0.066510
0.975000	1.959964	1.959978	1.956721	0.000701	0.165453
0.990000	2.326348	2.326287	2.323588	0.002614	0.118620
0.995000	2.575829	2.575744	2.575194	0.003311	0.024658
0.997500	2.807034	2.806943	2.809597	0.003233	0.091325
0.999500	3.290527	3.290463	3.303295	0.001931	0.388044
0.999750	3.480756	3.480712	3.498733	0.001275	0.516455
0.999900	3.719016	3.718998	3.744378	0.000496	0.681953
0.999950	3.890592	3.890592	3.921829	0.000010	0.802889
0.999975	4.055627	4.055642	4.092928	0.000374	0.919743
0.999995	4.417173	4.417211	4.469071	0.000858	1.174914

References

- [1] Bartsch, H.J. (1971) *Mathematische Formeln*, Buch- und Zeit-Verlags-gesellschaft mbH, Köln.
- [2] Derenzo, S.E. (1977) *Approximations for Hand Calculators Using Small Integer Coefficients*, *Mathematics of Computation* **31**, 214-225.
- [3] Lin, Jinn-Tyan (1989) *Approximating the Normal Probability and its Inverse for Use on a Pocket Calculator*, *Appl. Statist.* **38**, 69-70.
- [4] Velleman, P.F and Hoaglin, D.C. (1981) *Applications, Basics, and Computing of Exploratory Data Analysis*, Duxbury Press, Boston.

Appendix: The Programs

$\forall \Delta \leftarrow \text{DERTEST } p; AP; A; B; C; D; P; Q; a; b; c; \Delta$

- [1] $AP \leftarrow -2 \times 1 - p$
- [2] $A \leftarrow 83 \diamond B \leftarrow 351 \diamond C \leftarrow 562 - 165 \times AP \diamond D \leftarrow -703 \times AP$
- [3] $a \leftarrow B \div A \diamond b \leftarrow C \div A \diamond c \leftarrow D \div A$
- [4] $P \leftarrow b - a \times a \div 3 \diamond Q \leftarrow c + (\div 27) \times (2 \times a \times 3) - 9 \times a \times b$
- [5] $\Delta \leftarrow ((Q \div 2) \times 2) + (P \div 3) \times 3$

∇

$\forall R \leftarrow \text{IDERENZO } p; a; b; c; p; q; D; u; v; A$

- [1] $A \leftarrow -2 \times 1 - p$
- [2] $a \leftarrow 351 + 83 \diamond b \leftarrow (562 - 165 \times A) \div 83 \diamond c \leftarrow -703 \times A \div 83$
- [3] $p \leftarrow b - (a \times a) \div 3$
- [4] $q \leftarrow c + ((2 \times a \times 3) - 9 \times a \times b) \div 27$
- [5] $D \leftarrow (0.25 \times q \times q) + (p \div 3) \times 3$
- [6] $u \leftarrow (x \times u) \times (|u| \times (-0.5 \times q) + D \times 0.5) \div 3$
- [7] $v \leftarrow (x \times v) \times (|v| \times (0.5 \times q) + D \times 0.5) \div 3$
- [8] $R \leftarrow u + v - a \div 3$

∇

$\forall R \leftarrow \text{CARDANO } ABCD; a; b; c; p; q; D; u; v; f$

- [1] $a \leftarrow ABCD[2] \div ABCD[1]$
- [2] $b \leftarrow ABCD[3] \div ABCD[1]$
- [3] $c \leftarrow ABCD[4] \div ABCD[1]$
- [4] $p \leftarrow b - (a \times a) \div 3$
- [5] $q \leftarrow c + ((2 \times a \times 3) - 9 \times a \times b) \div 27$
- [6] $a \leftarrow -a \div 3$
- [7] $\rightarrow (-1 \ 0 \ 1 = x D + (0.25 \times q \times q) + (p \div 3) \times 3) / DSO, DEO, DGO$
- [8] $DSO: 'Three real solutions: '$
- [9] $R \leftarrow 2 \text{ of } (-2 \times (-0.5 \times q) \times ((p \div 3) \times 3) \times -0.5) \div 3$
- [10] $R \leftarrow a + (2 \times (|p \div 3| \times 0.5) \times R, -2 \text{ of } -1 \ 1 \times 0 \div 3$
- [11] $\rightarrow 0$
- [12] $DEO: 'Three real solutions (two identical): '$
- [13] $R \leftarrow a + (-u + u), R, R \leftarrow u \times (0.5 \times q) \div 3$
- [14] $\rightarrow 0$
- [15] $DGO: 'Two conjugate complex and one real solution: '$
- [16] $u \leftarrow (x \times u) \times (|u| \times (-0.5 \times q) + D \times 0.5) \div 3$
- [17] $v \leftarrow (x \times v) \times (|v| \times (0.5 \times q) + D \times 0.5) \div 3$
- [18] $R \leftarrow u + v + a$
- [19] $(\sqrt{a - 0.5 \times u + v}), ' + ', (\sqrt{(0.5 \times 3 \times 0.5) \times u - v}), ' i '$
- [20] $(\sqrt{a - 0.5 \times u + v}), ' - ', (\sqrt{(0.5 \times 3 \times 0.5) \times u - v}), ' i '$

∇

APL92: Provisional Programme

Invited Speakers

Igor Pospelov - *Russia on the Way from Plan to Market: The Experience of Modelling.* Professor Pospelov is a well-known Russian mathematician who specialises in economic modelling. He works in the laboratory for mathematical modelling of economic systems in the Computing Center of the Russian Academy of Sciences. Prof. Pospelov has over ten years' experience using APL. His presentation deals with the modelling of different transformations of the economy, along with a discussion and interpretation of the results.

Howard Peelle - *Education and the Future of APL.* Dr. Peelle is an accomplished APL teacher, author, and educator. As Associate Editor of *APL Quote Quad*, he continues to make a contribution to this area. His presentation will cover a number of questions of great concern to every APL enthusiast. Potentially, the educational system can provide a well-head for new members of the APL community. Dr. Peelle is working together with a team to prepare other activities on the theme of APL and Education for APL92.

Per Gjerlov - *The Song of the Whale.* Since APL90 (Per was Program Chair) our closing Plenary speaker has been travelling the Atlantic Ocean with an oceanographic team, recording and analysing audio data from groups of whales. Per will explain some of his work in detail, and show us some marvellous images (of whales and data!) to bring APL92 to a rousing close.

The final program will include additional invited speakers on APL and its development and future in Russia.

Tutorials

On the day before APL92 officially opens, a Tutorial day will be held in classrooms at the Education Center.

An introduction to APL - by Ray Polivka, noted author and teacher of APL, and winner of the ACM/SIGAPL Iverson award. This tutorial is oriented towards those with little or no exposure to APL.

An APL tutorial for Russian Children - is being organized in conjunction with the "APL in Education" theme.

Russian for APLers - Proposed tutorial, to run on Sunday and during APL92. Those attending this tutorial will gain some understanding of simple Russian sufficient to ask and understand simple questions, small phrases, and words useful for polite greetings and common dialogue.

Please note: Scheduling for this tutorial is based on the interest level expressed by those registering. A check-box on the registration form can be used to show your interest, and a tutorial session may then be organized.

Call for Posters

Poster sessions, and poster submissions, are traditional for conferences in many technical disciplines. Posters are relatively new to APL conferences, having been introduced first at APL89. The name "Poster" comes from the practice of submitting material in the form of a poster (or sign), which is displayed at the conference.

Posters are used to show novel ideas and reports of work in progress. A separate session will be devoted to poster presentation. Authors of posters can discuss the poster, and distribute related materials as conference participants circulate to discuss them.

If you would like to show a poster, please send a title and abstract by 1 June to the APL92 conference office. We can include it in the final program, and ensure that space will be available for your poster presentation.

Vendor Exhibition

An area of the Center has been set aside for use by vendors for showing products and services of interest to the APL community. We also encourage exhibitions from any organization needing exposure to well-placed computing professionals within and beyond the Commonwealth of Independent States.

Vendor forums are offered for those who wish to present commercial products in detail. Some forums will not conflict with the APL92 technical program, and provide attendees with a good chance to meet APL vendors. If you attend APL92, we expect that vendor forums will provide another interesting facet of the conference. If you are a vendor, please contact one of the APL92 offices, as some exhibit facilities and vendor forum times are still available.

Accepted Papers - APL92

Warren Julian, Australia - *Writing Macintosh Applications in APL*. Features of a student enrolment and record system for use on Macintosh computers are described, emphasising the development of the interfaces between the user and the APL file environment.

Ruby Lim, Australia - *Cannibals and Missionaries*. The old game of Cannibals and Missionaries is presented. The solution (sometimes surprisingly) is derived, and is implemented using APL68000.

Moishe Sniedovich and Suzanne Findlay, Australia - *Jogging With APL Along the Shortest Path*. The classical shortest path problem is examined and the modelling issues are illustrated by formulating APL code for a number of generalizations, including multi-objective problems.

Walter G. Spunde, Australia - *Challenging the Mathematics Curriculum*. Packages accompanying mathematical texts often try to make computing power serve as a simple non-disruptive adjunct to the current syllabus. The author asserts that instead it should serve to accomplish that which previously could not be done, rather than merely to expedite drudgery.

Andreas Geyer-Schultz and Thomas Kolarik, Austria - *Distributed Computing with APL*. APL is suggested as a prototyping and debugging tool for distributed applications in the growing UNIX market segment. Several programming constructs are implemented via shared variables in the distributed environment.

Jacob Milohner, Austria - *Porting APL Programs via Readable ASCII Transliteration*. A translation scheme for writing APL code to a plain text file is introduced, and a transfer system is developed which reads and writes objects to and from files.

Robert Bernecky and Gert Osterberg, Canada - *Compiler Tools in APL*. Compiler development tools in APL, including finite state machines and parsers, are shown to be of significant value in a number of common applications. Examples include a financial trading system and a UNIX tab expander.

Edward Y.H. Lin, Canada - *Multiple Choice Programming: An APL Approach*. This paper introduces a set of APL code for the optimisation of multiple choice programming using the special-ordered-set branch-and-bound procedure incorporating the partitioning strategy of the weighted mean method.

Norman Thomson, England - *APL2 Specification of Statistical Algorithms*. APL2 has striking affinities with the apparatus needed to specify statistical algorithms. Some rules and conventions for the basis of such a standard are proposed, and the appropriateness of APL2 is illustrated.

Gustav Toffet and Peter Harrison, Finland - *A Bezier Curve Builder*. A Bezier curve provides a cubic approximation to a curve between two points. Two additional points specify a tangent and a relative weight at each end point. The paper describes techniques used to smooth a broken line approximation. Applications (Font generation, Graphics) are covered and emphasised.

Jean-Jacques Girardot, France - *The CTalk Programming Language: A Strategic Evolution of APL*. CTalk, a new programming language quite different from APL, is presented. An attempt to combine the best features of APL and languages such as LISP and C, it looks "modern" according to the criteria of the mainstream of computer science.

Gerard A. Langlet, France - *Towards the Ultimate APL-TOE*. Can the laws of nature be reconstructed independently from research in theoretical physics (on elementary particles and matter in general)? Can the same process be applied to algorithms using APL?

W. Hahn, Germany - *Auxiliary Processor AP87*. An auxiliary processor, supporting extended precision (IEEE 754 standard) arithmetic for APL/PC is described. This tool provides a number of techniques for dealing with extended precision numbers in APL.

Cristoph Pospiech, Germany - *An APL2 Tool Box for Investigating Schwarz Methods*. An APL tool box is described which allows one to interactively investigate and compare algorithms for solving elliptic partial differential equations by Schwarz methods.

Tien Chi Chen, Hong Kong - *Globally Convergent Polynomial Iterative Zero Finding Using APL*. The paper describes a global iteration theory for polynomial zero-finding. A new formula, based on the target function and its first two derivatives, is demonstrated to have better convergence properties than two classical schemes.

Renato Capra, Italy - *3D Layout Manipulation Functions with a Glance to Optimization Problems*. APL syntax provides a very simple and expressive way to generate and manipulate 3D parametric objects. The paper describes a user-friendly interactive graphic system for 3D layout manipulation.

F.H.D. van Batenburg, Netherlands - *Sorting and Optimizing Star - A Case Study of Suffering and Surfacing*. After a brief description of an application in micro-biology, the paper reports on the migration of the system between workstation platforms.

Valery Kalinin and Boris A. Makeev, Russia - *APL as a Tool for Scientific Forecasting*. An expert system for forecasting development within fields of science is discussed. APL*PLUS/PC is chosen as the implementation language and the rationale for the choice is presented.

Alexander Skomorokov, Russia - *Do Russian Children Like APL?* Describes experiences teaching children about computer science using APL2. Both goals and methods are included, along with proposed extensions to use APL to teach many other subjects.

Alexey N. Skurihin, Russia - *Neural Networks in J*. The attempt to implement Artificial Neural Systems is described. The language J is used to facilitate a straightforward implementation.

Manuel Altonseca, Spain - *Object-Oriented Graphics in APL2*. The paper describes a general and extensive graphic system in two or three dimensions, built in APL2 by means of the object-oriented programming paradigm. The system fully demonstrates the appropriateness of the language to develop real-life applications.

Martin Gfeller, Switzerland - *Walks into the APL Design Space*. APL is evolving. The choices for future work constitutes a "design space" for future development. This paper assists the reader in exploring the APL design space on one's own. References to books and articles about problems and solutions are given.

J. Philip Benkard, USA - *Introduction to Nested Arrays and Operators*. Continuing exploration of the implications of nested data structures, comparing and contrasting the importance of the rank operator.

James Brown and Erik Kane, USA - *APL2 Implementation of Cross System Shared Variables*. The requirement for inter-machine communication is discussed. It is met with APL2's cross-system shared variables. In particular, the mapping of APL2's cooperative peer-to-peer communication model onto a client server data transport mechanism is described.

Evan Jennings, USA - *An Object-Oriented APL2 Expert*. An expert system which can answer questions posed in English about APL2 is described. The programs are written in APL2 using object-oriented programming techniques. The knowledge is hierarchical and frame based.

Curtis A. Jones, USA - *Left to Its Own Devices, APL Plots on the Bus*. A PC with APL can be used as a device on the IEEE-488 General Purpose Interface Bus. An APL workspace which emulates an x-y plotter illustrates some uses.

Peter A.W. Lewis, USA - *A Full-Screen, Scrollable, APL2 Spreadsheet Input/Output Editor*. A spreadsheet editor based on IBM's APL2 language for 386 machines is used for entering, analysing, editing, and outputting data. APL functions can be applied to blocks of data, drawing on the full computational and notational power of APL.

John Mizel, USA - *Using SCCS to Manage APL Development Projects*. A Change Control System for managing large-scale APL projects is described. Certain APL-specific problems are discussed.

Richard Neldinger, USA - *Differential Equations Are Recurrence Relationships in APL*. A differential equation can be seen as a recurrence relation, which can be used to produce the Taylor coefficients. The paper describes an addition to an automatic differentiation workspace, which produces a useful tool from this idea.

Steve Promisel, USA - *A Multi-National Sales Planning System in APL*. An APL-based logistics planning system is described, with emphasis on the discussion of requirements arising from the need for the system to operate across international borders. Topics include managing multiple currencies, languages, product orientation, and time-phased planning.

Alvin J. Surkan, USA - *WSDM: Weighted Sparse Distributed Memory Prototype Expressed in APL*. A functional style application of APL notation succinctly describes the architecture and principles of operation of one kind of Connection Machine. APL programs can provide precise descriptions of operational machines. Two adaptive variants of the sparse distributed memory machine are modelled in APL and presented in the paper.

Robert G. Willhoft, USA - *Matrix Operations Over Integral Domains Using Nested APL*. The paper deals with the use of APL to solve matrix theory problems with elements that use members of an integral domain.

Robert G. Willhoft, USA - *Petri Net Evaluation Using APL2*. A Petri net is a graphical and modelling tool useful in the analysis of a variety of problems; the paper introduces a method of representing Petri nets in APL2.

Scott Williams, USA - *EZHONE: A Perfect Application for APL2 Name Association*. A widespread application on a modified VSPC has been migrated to APL2. Migration chains are described, with emphasis on how the name association facility can replace disparate older facilities.

Workshops and Panel Sessions

A series of workshops is planned for APL92. These events will be at minimum two hours' duration, but more normally a complete half day is planned. Attendance at the sessions will be limited. Please indicate your interests on the

registration form. We will endeavour to honour workshop preferences if you register. The sessions described below are in an advanced stage of planning, but circumstances beyond our control may prevent one or more from taking place. If these sessions are critical to your attendance plans, please contact the APL92 office before making your travel plans. We will try to confirm the exact status of workshops for you. Indeed it is likely that additional workshops will be set up between now and July. We welcome your enquiries. You are invited to contribute a short (one page or less) written statement for distribution to other participants at workshop sessions you have checked on the registration form. Please send any materials you wish to be distributed to: Dick Bowman, 2 Dean Gardens, London E17 3QP, England, by 1 June, 1992. The preferred format is as an ASCII file on an MS-DOS compatible diskette.

Applied Statistics Library Workshop: to be presented by Alan Sykes. The Applied Statistics Library is a project which has been underway for several years, building a library of statistical software written in APL and intended to allow the professional statistician to bypass the limitations of commercial packages. The workshop will explore ASL, and its use in a number of case studies.

Using J: presenter to be announced. J represents a further development of some notational aspects of APL (although everybody has their own personal view on this). The purpose of the workshop is to explore the implications of using J to tackle real-life problems, and to identify the benefits and difficulties that this new notation brings.

APL and Education: presenter Howard Peelle. The dual concepts of introducing APL in education and educating people about APL have been constant themes of APL conferences for many years. This workshop brings together many people who have been active in these areas, and aims to take a proactive stance in defining a future and effective strategy for APL.

Modern User Interfaces: presenter(s) John Scholes/Richard Nabavi. There have been many advances in user interfaces in recent years; APL has arguably either ignored these advances or been left behind by them. What are the considerations behind applying a modern user interface to applications? What tools does APL offer? How do we build modern user interfaces into modern APL applications?

Out of the tool box: conducted by Robert G. Brown, with others. APL software developers require more tools than are provided with many interpreters. This workshop will survey some of the commonly needed tools, attempt to define a required set of tools for all APL users, and techniques that can be adapted to their use. Software related to this workshop will be available via the Software Exchange.

The Future of APL: moderator Lynne Shaw. This plenary panel discussion brings together a number of well-known and experienced APL personalities from several countries. This panel will confront issues relevant to the future use of APL, and possible roles for APL in the software development cycle. Current obstacles (and possible solutions) to further development of APL will be addressed.

If APL is so great, why isn't everybody using it? Conducted by Cory Skutt. The history of computing is rich in languages that have been bypassed. About 10 years ago, LISP was about to become one such language, until the LISP community took some steps to avert the demise of LISP. There are lessons to be learned from this experience, and they can be used by the APL community to help secure a future for APL in an increasingly competitive computing environment. Cory will also be a participant in *The Future of APL*, but will develop his ideas in more depth for potential APL activists.

Registration

You are encouraged to complete and return the registration form with your payments as soon as possible. The deadline for early registration is 8 May. A participant will be considered registered only when all fees are paid. Payments including accommodation must be made in advance to APL92, by credit card, international money order in US dollars or by cheque on a United States bank in US dollars. The Registration Fee covers:

For Participants (members and non-members): attendance at all sessions, the conference proceedings, coffee breaks and refreshments; lunches Monday, 6 July, through Friday, 10 July, and all social events (with dinners) Sunday, 5 July, through Thursday, 9 July.

For Students: attendance at all sessions, the conference proceedings, coffee break refreshments; five lunches Monday to Friday, and all evening social events except the closing banquet on Thursday (additional tickets for the closing banquet are available for US \$35.00).

For Accompanying Adults: sightseeing package and all evening social events (including dinners). The sightseeing package consists of four sightseeing tours selected from those offered Sunday through Friday. Each tour includes lunch in a city restaurant.

For Accompanying Children: all the features for accompanying adults except the Exhibitors' Reception on Monday and the Closing Banquet on Thursday. (Additional tickets for closing banquet are available for US \$35.00. Child care will be available during these two events.)

A participant who includes an ACM/SIGAPL membership form and dues payment with the APL92 registration form is entitled to register for the conference at the discounted member rate.

Please return the APL92 registration form as soon as possible. An official invitation from the Russian Academy of Sciences will be extended to all registrants. This invitation, necessary to obtain a Russian (business) visa, will be sent when the registration form is received. Residents of Finland should send the registration forms to Erkki Juvonen.

Fees:

	Before 8 May	After 8 May
ACM/SIGAPL Member	\$400	\$500
FinnAPL Member	\$400	\$500
Non-Member	\$475	\$575
Accompanying Adult	\$275	\$275
Accompanying Child	\$120	\$120
ACM/SIGAPL Student	\$120	\$120
Single Day	\$125	\$125

Payment

You may pay your registration and other fees in US dollars by cheque drawn on a US bank payable to APL92, by money order or by credit card (MasterCard, Visa, or American Express). We are unable to accept cheques drawn on non-US banks. They will be returned and the participant will not be considered registered until correct payment is made. If your national currency is not convertible, please contact a Conference Office to discuss payment alternatives.

APL92 Conference Office ...

Before 1 July:

POB 43 SF-00331 Helsinki Finland E-mail: APL92@sms.ccas.msk.su
Fax: +358-0-459 4442 Phone: +358-4-943 4497

After 1 July:

APL92 Conference Aerodromnaya, 4 POB 728 197348 St.Petersburg Russian Federation E-mail: APL92@iha.spb.su
Fax: +7-(812)-394 5005

Registration/Questions:

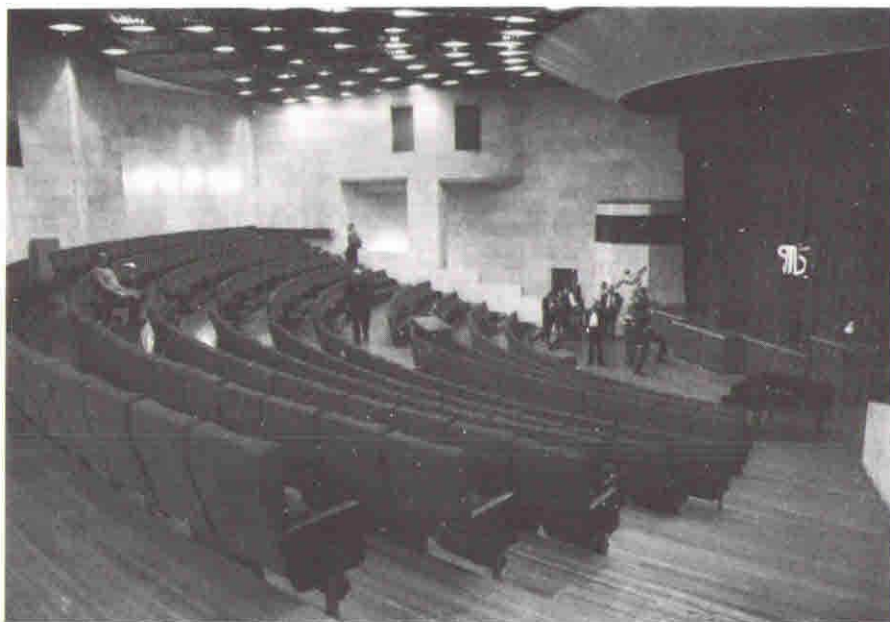
Lynne C. Shaw, 417 Riverside Drive, Apt. 9B New York, NY 10025, USA

E-mail: Lynne_Shaw@mts.cc.wayne.edu

Fax: (212) 662-0913

Phone: (212) 662-2406

Cancellation. All requests for cancellations and refunds must be in writing. There will be a \$50 administrative charge deducted for cancellations received prior to 10 June 1992. After 10 June the administrative charge will be 10% of registration and housing fees. Cancellations for post-conference tours and refunds for those charges will be handled by the tour agent. Participant substitutions can be made up to 1 June without charge; after 1 June, there will be a \$50 administrative charge.



A view of the hall for the plenary sessions.

Report on St Petersburg Visit

by Anthony Camacho

I attended the meeting of the APL 92 program committee in St Petersburg from 9 to 12 January 1992 as a representative of the British APL Association. I had to be bullied to go as at that time of year the hours of daylight are few and the temperature is low; I don't like the dark and cold. The conference itself will be in July when you can read your watch by natural light at midnight and the weather is warm.

First Impressions

Nikolai Puntikov met me and drove me to the centre in his car. The airport is the far side of St Petersburg from the centre. Even though it was dark it was plain that St Petersburg is a very beautiful city, with hundreds of eighteenth century buildings and intersected by about seventy canals and rivers; there are about three hundred bridges.

The Leningrad Education Centre

The Director told us they are in process of reorganising; for 20 years they have worked in industrial training. They don't intend to change direction, but to undertake much more international work, especially in management and marketing. They already have many links with foreign partners and hold many events in conjunction with institutes and companies abroad (there was a flag in his office left by a group from Dallas). There are lots of problems to solve if they are to keep going in this unstable time. They have to earn money independently; there has been no state subsidy for the last two years.

At present the centre is still a State Institute called the Leningrad Education Centre but they may change the name to reflect both St Petersburg and the fact that there is no longer a USSR for them to be a State Institute of. Perhaps it will become an independent international management school; they are seeking partnership with a European Bank. Obviously they are jumping at the chance to cater for an international conference at least partly paid for in hard currency.

The Director expects that by the summer a satellite telephone link to the world network will be working (a firm from Denver is currently installing it). The Centre contains its own medical facilities and staff. The building is 7 minutes from the nearest metro and 15 minutes by car from the centre of St Petersburg.

The Leningrad Education Centre is three buildings on six floors. It contains:

Rooms for about 300 people,

A restaurant and bar,

One large hall which will seat over 300 in raked seats and which has room for even more seats if needed.

Two smaller halls with TV projectors which will each seat 60 or more, also with raked seating.

One specially large room which will seat 100 or more

About fifteen rooms which could seat 20 to 30 each

A room of 60 square metres which could be the software exchange

A library and the usual associated facilities.

Each room used for presentations will have two OHPs, one with an LCD screen for projecting a computer screen for the audience to read.

The Accommodation for Delegates

I had a single room with bed, desk, wardrobe, table, desk lamp, silent fridge (absorption) and a television aerial (some rooms had sets - I guess the one from mine was being repaired). There was a private bathroom with shower, WC, washbasin, heated towel rail and a bowl with a washboard for washing clothes. These are the very same rooms that will be used for the conference. The accommodation was much better than at Stanford or Manchester.



A view of my room

Food for the Conference

The food for the conference will be much better than the news would lead one to believe. The real problem is the volatility of the currency. Farms are having to trade by barter to ensure that they don't lose too much when the value of the rouble changes rapidly so it is hard to buy for roubles — i.e. you queue. You can buy anything with foreign (hard) currency. Since the conference expects to entertain many visitors from abroad the organisers have signed contracts for the food with a Russo-Finnish company to be paid for in Finnmarks. This is a contract which is very attractive to the supplying company and provides them with some stability in the Russian world of unstable prices. This company can thus buy any food they want in Russia and anything they can't get there, they can import from Finland. It is about four hours by road from Helsinki.

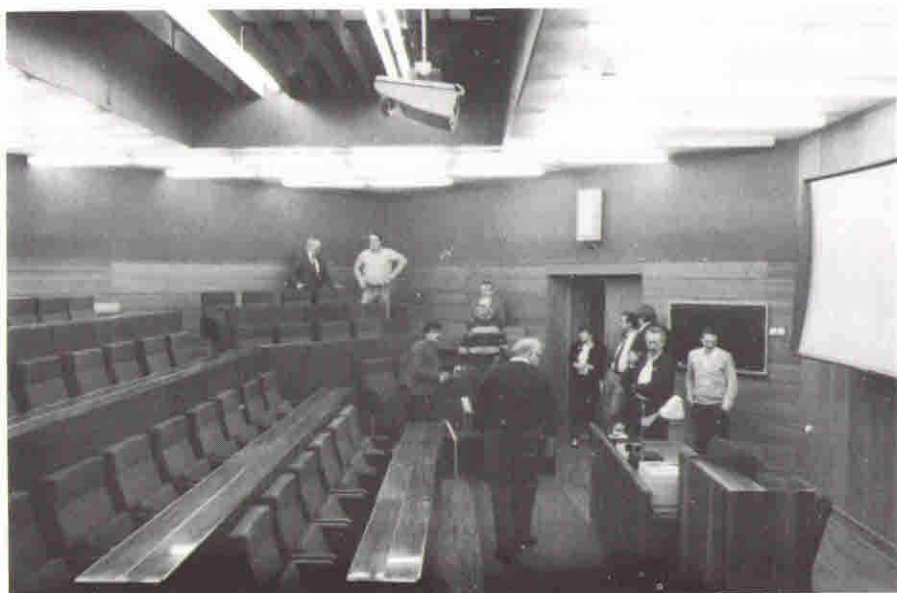


The room for the Software Exchange

As Tourists, Delegates may be Rich

While I was there the shortage of hard currency was so severe that five U S dollars would buy food for a family of four for a week. This makes the price of souvenirs — indeed more or less everything — very cheap indeed for visitors from any country which uses hard currency. There were touts who were offering

bottles of Russian Champagne (very good it is) to tourists on buses at \$2 a bottle. It seemed cheap but not when one found out what it could be bought for elsewhere. One can hope that the currency problems are reduced by the time the conference happens; if they aren't life as a tourist will be comparatively luxurious and incredibly cheap.



A view of the 60-seat lecture theatre

Conclusion

APL 92 will be well fed, comfortably accommodated and in a beautiful city. Visitors from abroad are likely to find that the exchange rate is still very favourable. My impression is that the Russian organisers are making a great effort to ensure that the occasion will be a memorable one. I haven't seen the program yet, but of course I have a good idea of some of the papers. Sylvia and I are going. I hope we see you there.

APL Product Guide

Compiled by Gill Smith

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We do depend on the alacrity of suppliers to keep us informed about their products so that we can update the Guide for each issue of VECTOR. Any suppliers who are not included in the Guide should contact me to get their free entry - see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage.

The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages. Where no UK distributor has yet been appointed, the vendor should indicate whether this is imminent or whether approaches for representation by existing companies are welcomed.

For convenience to readers, the product list has been divided into the following groups:

- Complete APL Systems (Hardware & Software)
- APL Timesharing Services
- APL Interpreters
- APL Visual Display Units
- APL character set printers
- APL-based packages
- APL Consultancy
- APL Training Courses
- Other services
- Vendor addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

Note: 'poa' indicates 'price on application'.

All contributions and updates to the APL Product Guide should be sent to Gill Smith, at the address on the inside back cover.

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace Ltd	AWL486	4,450	486 based 25MHz PC, 140MB Disk, 4MB RAM, VGA Colour. (inc. 1 year on site maint.)
	AWL 386	3,095+	386 based 25 & 33MHz PC, 140MB Disk, 4MB RAM, VGA Colour. (inc. 1 year on site maint.)
APL People	IBM PCs & compatibles	poa	includes PC, mono/colour monitor, APL interpreter, operating system software, plus optional printers, graphics boards, additional memory etc.
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,656	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
	IBM RS/6000 MD540	122,842	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
MicroAPL	IBM RS/6000	12,000+	POWER range of RISC systems running AIX. Dumb terminal or graphical interface.
	Aurora	20,000+	Multi-user APL computer using 68020 CPU. Std. configuration 2Mb RAM, 16 RS232 ports, 68 Mb hard disc, 720K diskette

APL TIMESHARING SERVICES

COMPANY	PRODUCT	PRICES(£)	DETAILS
Reuters Ltd	SHARP APL	poa	International Network application systems and public databases.
Uniware	APL*PLUS	call	STSC's mainframe service

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace Ltd	DYALOG APL DOS 386		Dyadics PC 386 APL interpreter
APL Software	APL*Plus/PC Release 9	535	STSC's APL for IBM PC, PC/AT and PS2. Upgrades from earlier Releases also available.
	Run-time	poa	Closed version of APL*Plus/PC which prevents user exposure to APL.

Cooking/Drury	APL*Plus II	1600	Incorporates mainframe features & performance in a version of APL for the PC
	Run-time	poa	
	Dyalog APL	1000-10,000	2nd generation APL for Unix systems
	APL2/PC	376	IBM's APL 2 for the PC.
	APL*PLUS PC Rel 10	410	STSC's full featured APL for IBM's and compatibles - Version 10 includes the Quad-NA facility to interface to non-APL software, support for Microsoft Windows and mouse devices. The User-command processor has been built in to the interpreter.
			Upgrades to version 10 are available from Version 9 and earlier releases.
	APL*PLUS PC Run-Time	175 for 5	Closed version of the interpreter for developers, prevents user exposure to APL.
	APL*PLUS PC Developer System	950	Gives rights to distribute an unlimited number of copies of Run-Time application.
	APL*PLUS II System	1200	High powered APL interpreter for the 80386 chip. Price includes one year's maintenance and free upgrades - volume discounts VERSION 4.0 NOW AVAILABLE - THE ONLY APL THAT LETS YOU BUILD WINDOWS 7 DOS APPLICATIONS FROM ONE PACKAGE.
	APL*PLUS II Developer System	3200	Unlimited distribution of APL*PLUS II Run-Time applications!
Dyadic	APL*PLUS II for UNIX	poa	STSC's 2nd generation APL for all major Sparc and Risc Unix workstations.
	APL*PLUS VMS	poa	2nd generation APL for DEC VAX computers running under VMS
	APL*PLUS Mainframe	poa	Enhances VS APL with many high performance, high productivity features. For VM/CMS and MVS/TSO offers simple upgrade from VS APL.
	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and dSM Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle Interface available for IBM, Sun and Xenix versions.
I-APL Ltd	I-APL/PC or RML Nimbus	4.50	ISO conforming interpreter. Supplied only with manual. (see 'Other Products' for accompanying books)
	I-APL/BBC	4.50	As above
	I-APL/Archimedes	4.50	As above
	ISI Products		I-APL is the UK agent for ALL ISI products, including: Sharp APL + enhancements to the ISI-APL standard Versions of J (and the associated texts) for PC, Mac, Arc, Atari! Please call for price list and order form.
IBM (APL Products)	APL Version 2 Release 1	poa	Full APL2 System for IBM 370 and 390. Product No. 5688-228. See your IBM Branch Office.
	APL2 Application Env't Ver2 Rel1	poa	Run-time Environment for APL2 Packages (IBM 370 and 390). Product No. 5688-229. See your IBM Branch Office
	APL2 for the RISC System/6000	poa	Product No. 5765-012. See your IBM Branch Office.
	APL2 for the IBM PC (US version)	\$495	Product No. 5799-PGG. Part No. 6242936. PRPQ No. RJ6411 (in HONE, RJ0411). (from IBM Direct, order by Part Number; from your IBM Branch Office, order by PRPQ Number)
	APL2 for IBM PC (European vers)	poa	Product No. 5604-260. Part No. 38F1753
	TryAPL2	free	Free APL2 for educational/demonstration use. Write to APL Products, specifying diskette size desired.

IBM UK	IBM PC APL2	348	APL2 for the IBM PC. Program 5799-PGG/1. PRPQ number RJ-0411. From all IBM dealers, including MicroAPL.
MicroAPL	APL68000 Level I	2000	First generation APL with numerous enhancements. Multi-user version (Unix, Mirage, MCS).
	APL68000 Level II	2500	Second generation APL. Nested arrays, user defined operators, selective specification etc. Multi-user version (Unix, Mirage, MCS).
	APL68000/X	1500-6000	Second-generation APL. Nested arrays, user defined operators, selective specification, etc. Multi-user AIX version with full OSF/Motif support.
	APL68000 Level I Mac, ST, Amiga	87	First generation APL. Single user, full windowing interface, software floating point support.
	Mac, Amiga	260	First generation APL. Single user, full windowing interface, hardware floating point.
	APL68000 Level II ST	170	Second generation APL. Full windowing interface, software floating point support.
	Amiga	260	Second generation APL. Full windowing interface. Hardware and software floating point support.
	Mac	520	Second generation APL. Full windowing interface. Hardware and software floating point support.
	APL*PLUS Rel 10	450	
Reuters Ltd	APL*PLUS II V 4.0	1395	
	SHARP APL	poa	for IBM mainframes
Unlware	APL*PLUS/PC	495	STSC's full feature APL for IBM PC/XT/AT, Compaq, Olivetti.
	Run-Time	call	Closed version of APL*PLUS/PC which prevents user exposure to APL.
	APL&PLUS/UNIX	call	STSC's full feature APL for UNIX based computers
	APL*PLUS II	call	STSC's full feature APL for 386 machines.

APL VISUAL DISPLAY UNITS

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL People	IBM & compatibles	poa	IBM and 'budget' APL VDUs - monochrome/colour/graphics.
Dyalac	IBM 3151	599	Monochrome APL/ASCII vdu with APL keyboard. Supports downloaded Dyalog APL font.
	IBM 6154	1,226	Colour APL/ASCII vdu with APL keyboard. Supports downloaded Dyalog APL font.
General Software	Mellordata	poa	
Shandell	HDS3200/10 APL	965	15" screen, 8 page memory, windows. 60/132 columns, full overstrike. Multi-host multi-session support. ANSI X3.64, DEC VT100, VT220, Tektronix 4010/4014 1024 x 390 resolution.
	HDS3200/25 APL	1065	As above plus switchable 25 or 50 line screen. 75 Hz refresh. Resolution 1024 x 780 in graphics mode.
	HDS3200/35 APL	1265	As HDS3200/25 plus local pan & zoom.
	HDS3200/5C APL	1395	14" colour monitor. 75 Hz refresh, 8 or 16 colours from palette of 256. Display memory 96 lines of 80 or 132 columns. APL processing & keyboard with full overstrike. Windows, multi-host, multi-session support. ANSI X3.64, DEC VT220, VT100, VT52 emulation.

APL PRINTERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL People	Epson series	200	Inexpensive dot-matrix and NLQ printers
	Queen-data & Qume etc	500	Daisy-wheel printers
Dyadic	Various	poa	Range of APL printers available.

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace Ltd	Syndicate Manager	poa	Lloyd's managing agent's syndicate / company accounting system. Stamp & Personal accounts (inc. Run offs)
APL-385	APL-385	50(PC),125(mf)	Including ...
	FSM-385		Screen development
	DRAW-385		Screen design
	DB-385		Relational W.S.
	GEN-385		Miscellaneous Utilities
APL Software Ltd (mainframe)	RDS	poa	Relation Data Base System
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions, includes a Forms Processor.
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	RDS	990	Relational Database System
Cocking/Drury (for VSAPL)	E'MENTS & SHAREFILE	poa	Component files, quad- functions & nested arrays for VSAPL under VM/CMS & MVS/TSO
	COMPILER	poa	The First APL compiler!
	FILEPRINT	poa	Print APL component files
	FILECONVERT	poa	Converts non-APL files to APL
	FILEMANAGER	poa	Extends APL primitives to database management
	TOOLS + UTILITIES	poa	APL Software development tools
	DATAPORT	poa	Information Centre spreadsheet incorporating data exchange between APL, FOCUS, IFPS, SAS, APL/DI, ADRSII, Lotus123, Visicalc, Multipian & DIF
(for APL2)	SHAREFILE/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage
	FMT	poa	Full featured FMT for APL2
	WSDOC	poa	Workspace documentation utilities
	FILEMANAGER	poa	Extends APL primitives to database management
(for PC's)	APL*PLUS PC Tools	275	Utilities including: RAM disk, full screen data entry, menu input, report generation, exception handling and games.
	IRMA Module	90	327x IRMA support.
	FIN & STAT. LIBRARY	250	Financial & Statistical routines
	SPREADSHEET MGR	150	APL-based spreadsheet for APL*PLUS/PC. Cell arithmetic; transfers to ASCII & Lotus
H.M.W.	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping

	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
HRH Systems	APL Utilities	poa	PC Utilities Including: APLMAC (windows); Unlock (unlocks functions in .AWS); DTEX (text and spreadsheet imp/exp). Mostly available in English or French.
Impetus Ltd	Impetus	poa	Corporate Modelling and Reporting System.
INFOSTROY	Russificator	poa	Drivers and documentation for use with APL*PLUS/PC system and other STSC software with Cyrillic alphabet (PC).
Interprocess	IEDIT	1900-3200	Full screen APL2 editor with immediate APL execution, and a full-screen debugger
(mainframe)	AFM	8200-9800	High performance component and keyed file system (VS APL and APL2)
	Format	1650	A QuadFMT data formatter for VS APL and APL2
	FSM124	1650	AP124 programming for APL applications without GDDM (APL2)
	PowerCode	1300	External functions for APL2
	CALL/AP	3000	for calling non-APL programs (VS APL and APL2)
	UCF	1800	Inter-user data transfer for VM users via IUVC
(PC)	AFM	115	Single user component and keyed files for APL2/PC
Mercia	STATGRAPHICS 4	684	Integrated statistics/graphics system for the PC. Now with macros. Bulk and educational discounts available.
	Upgrade 3 to 4	215	
	Upgrade pre 3 to 4	375	
	MICROSPAN	250	Comprehensive APL tutor
	LOGOL	poa	Logistics management system for PC and 386. Sales Forecasting, Inventory Control, Master Scheduling, Distribution Requirements, Planning etc
	TWIGS		A modular library of tools to teach and explore state-of-the-art materials management concepts.
	T	299	Time series forecasting
	W	99	Warehouse replenishment
	I	199	Inventory Management
	G	99	Grouping requirements into EOQ's
	S	99	Scheduling production/purchasing
		599	All 5 modules above
		4999	All 5 modules site licence
MicroAPL	MicroTASK	250	Product development aids
	MicroFILE	250	File utilities and database
	MicroPLOT	250	Graphics for HP plotters etc
	MicroLINK	250	General device communications
	MicroFORM	250	Full screen forms design
	MicroSPAN	250	Comprehensive APL tutor
	MicroPLOT/PC	250	For APL*PLUS/PC product
	MicroSPAN/PC	250	APL self instruction for APL*PLUS/PC
	STATGRAPHICS Rel 5	590	
Reuters Ltd	GLOBAL LIMITS	poa	Exposure management for banks
	IPSA/CONNECT	poa	Mainframe to micro link
	MAILBOX	poa	Electronic Mail
	MAILBOX/PC V.2	75	Full screen front end to IPSA mailbox

	upgrade to V.2	15	
	NEWSFLASH	poa	Real time message exchange
	VIEWPOINT	poa	4GL - Info centre product
Unware (mainframe)	STSC's ENHANCEMENTS	poa	Quad-functions & nested arrays for IBM VSAPL under VM/CMS and MVS/TSO
	STSC's SHAREFILE	poa	component files for IBM VSAPL under VM/CMS and MVS/TSO aimed for IBM APL2
	PROGRAMMER TOOLS & UTILITIES	POA	
	FILEPRINT	poa	
	FILESORT	poa	
	FILECONVERT	poa	
	FILEMANAGER(EMMA)	poa	STSC's database package
	EXECUCALC	poa	Mainframe spreadsheet compatible with VISICALC and part of LOTUS 1-2-3 under VSAPL(VM or TSO)
(microcomputer)	STATGRAPHICS	poa	Statistics and graphics for PCs
	STATGRAPHICS UNISTAT	poa	An add-on module to STATGRAPHICS: Data analysis software.
	APL*PLUS TOOLS		
	-VOL1	poa	Incl. 327 /E IRMA support, RAM disk, full screen data entry, menu input, report generation, games
	-VOL2	poa	Incl. File documentor, screen editor, exception handling.
	SPREADSHEET MNGR	poa	APL spreadsheet with built-in ASCII, LOTUS and SYMPHONY Interfaces.
	APL*PLUS/PC FIN & STAT.LIBRARY	poa	Collection of financial and statistical utilities.
	POCKET APL	poa	Smaller version of APL*PLUS/PC.
	UNIASM	poa	Collection of assembler routines for APL*PLUS/PC users.
	UNITAB	poa	APL*PLUS/PC spreadsheet-like data entry and validation system.
	The APL DEBUGGER	poa	First released APL*PLUS/PC debugger.
	APL2C	poa	Interface between APL*PLUS/PC and DATALIGHT C language
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.

APL CONSULTANCY

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace Ltd	Consultancy	poa	PC Based APL system design, programming and implementation.
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation,, of APL products in all APL environments
APL People	Consultancy	poa	Consultants available at all levels, with experience in: VS APL, APL*PLUS, APL2, Sharp APL, Dyalog APL, APL6800, C/Unix, TSO/MVS, VM/CMS, graphics, Operational Research etc. Expertise in APL system design, project management, prototyping, financial applications, decision support systems, MIS, links to non-APL systems, documentation, etc.

Buckland Management Systems	Consultancy	poa	Business and Technical systems in commerce and industry - designing, programming and implementing applications.
Camacho	Consultancy	poa	Specialising in programming & manual writing.
Chapman	Consultancy	150-300	24-hour programmer: APL, C, assembler, graphics; PC, mini, mainframe, network.
Cocking/Drury	Consultancy	175-275	Junior consultant
		275-350	Consultant
		300-450	Senior consultant
		400-600	Principal Consultant
		450-750	Managing consultant
Peter Cyriax	Consultancy	100-150	Junior Consultant
		120-200	Consultant
		160-300	Senior Consultant
Delphi	Consultancy	poa	APL system development on mainframes and micros.
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
E & S	Consultancy	poa	System prototyping: all types of information system, engineering software, graphics and decision support systems APL*PLUS/PC, APL2, Dyalog APL
General Software	Consultancy	from 120	
Michael Hughes	Consultancy	poa	Consultant with 10+ years experience with various APL interpreters and C.
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Ian A. Clark	Consultancy	poa	Computer-based Information Systems Implementation where acceptance is critical. APL on PC and Macintosh. Human Factors of HCI; novice ease of use; online assistance; training courses; distance-learning materials.
INFOSTROY	Consultancy	poa	Localization of APL software for the Soviet Union software market
Intelligent Programs Ltd	Consultancy	175-350	Systems development, enhancements, support.
	Documentation	150-250	Preparation of new manuals, rewriting of existing materials.
	Training	150-250	Training for APL experts through to non-technical system users.
Mercia	Consultancy	poa	APL*PLUS & VSAPL consultancy.
MicroAPL	Consultancy	poa	Technical & applications consultancy.
Ellis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
M.T.I.C.	Consultancy	240-500	Business analysis and APL consultancy
Parallax Systems Inc	Consultancy	\$750	Introductory APL, APL for End-user & Advanced Topics in APL
QB On-Line	Consultancy	250	Specialising in Banking, Financial & Planning Systems.
Reuters Ltd	Consultancy	poa	Consultancy & support service world-wide.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Rex Swain	Consultancy	poa	Independent consultant, 15 years experience. Custom software development & training, PC and/or mainframe.
Wickliffe Computer Ltd	Consultancy	poa	System design, consultancy, programming and documentation. Especially project management and decision support systems

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES (£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL People	Employment Agency	poa	Permanent employees placed at all levels. Contractors supplied for short/long-term contracts, supervised or unsupervised. Executive Search service available.
HMW	Employment	poa	Contractors and permanent employees placed.
I-APL Ltd	An APL Tutorial	2.50	45pp by Alvord & Thomson
	An Encyclopaedia of APL (2d Ed)	5.50	228pp by Heizer
	APL In Social Studies	2.50	36pp by Traberman
	I-APL Instruction Manual (2d Ed)	2.50	55pp by Camacho & Ziemann
	APL Programs for the Mathematics Classroom (Springer-Verlag)	14.50	185pp by Thomson
** Please add one pound packing charge per order. **			
IBM (APL Products)	APL2 Keycaps	poa	Keycaps for the PC and PS/2 (USA and UK standard). Product Number SX80-0270.
	APL2 Keyboard Stickers	poa	Product Number SC33-0604.
Reuters Ltd	Productivity Tools	poa	Utilities for systems, operations, administration & analysts; auxiliary processors, comms software, international network.
	Databases	poa	Financial, aviation, energy and socioeconomic.
ISI	Tangible Math	\$19	An APL Approach to Math - Shareware, includes base Sharp APL
	J	\$24	Dictionary APL simplified and enhanced - Shareware (Mac,PC)
	SharpAPL/PC	\$74	Registered Shareware and Reference Manual
	ISI APL	\$99	Improved APL/PC - enhancements, performance, large workspaces
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.

OVERSEAS ASSOCIATIONS (Sample entries)

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.	Visa/Mcd
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$15	N/N
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)	
Dutch APL U.G.	Holland	-	Mini-congress, APL ShareWare Initiative		
APL Club Austria	Austria	-	Quarterly Meetings	200AS per person, 1000AS per company	N/N
Sydney APLUG	Sydney, Australia	Epsilon	Monthly Meetings		

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
Active Workspace Ltd	Ross D Ranson	Moulsham Mill Centre, Parkway, Chelmsford, Essex, CM2 7PX. Tel: (0245) 252414 ext.240
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ LEXMOND, Netherlands. Tel: 31.3474.2337, fax: 31.3474.2342
APL 385	Adrian Smith	Brook House, Gilling East, York. Tel: 04393-385
APLBUG	Jorge Mezei	117 East Creek Dr., Menlo Park, CA 94025, USA
APL Club Austria	Erich Gail	IBM Österreich, Obere Donaustrasse 95, A-1020 Wien, Austria
APL People	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
APL Software	David Ails Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
BACUS	Joseph de Kerf	Rooienberg 72, B-2570 Duffel, Belgium.
Buckland Management Systems	John Buckland	Westwood, 19 Grange Road, Camberley, Surrey, GU15 2DH Tel: 0276 684327
Anthony Camacho		2 Blenheim Road, St. Albans, Herts AL1 4NR. Tel: St. Albans (0727) 860130
Paul Chapman		18, Trevelyan Road, London, SW17 9LN Tel: 081-767 4254
Cocking & Drury Ltd.	Romilly Cocking	180 Tottenham Court Road, LONDON, W1P 9LE Tel: 071-436 9481 Fax: 071-436 0524
Datatrade Ltd.	Tony Checksfeld	38 Billing Road, Northampton, NN1 5DQ. Tel: 0604-22289
Delphi Consultation	David Crossley	Church Green House, Stanford-in-the-Vale, Oxon SN7 8LQ. Tel: 0367-710384
Dutch APL U.G.	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein. Tel: 03474-2337
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL. Tel: 0256 811125 Fax: 0256 811130
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS. Tel: 0689-24741
General Software Ltd	M.E. Martin	22 Russell Road, Northholt, Middx, UB5 4QS. Tel: 081-884-9537
H.M.W. Trading Systems	Stan Wilkinson	Hamilton House, 1 Temple Avenue, Victoria Embankment, LONDON EC4Y 0HA Tel: 071-353 4212 Telex: 926604 HAMHSEK Fax: 071-353 3325
HRH Systems	Dick Holt	Box 4496, Silver Spring, Maryland 20904, USA.
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics., LE16 8QL Tel: 0536 770998
Ian A. Clark		9 Hill End, Frosterley, Bp. Auckland, Co. Durham DL13 2SX Tel: 038852-7190
I-APL Ltd	Anthony Camacho	2 Blenheim Road, St. Albans, AL1 4NR. Phone 0727-860130 for queries, order forms, bulk orders
	J C Business Services	56 The Crescent, Milton, Weston-super-Mare, Avon, BS22 8DU
IBM (APL Products)	Nancy Wheeler	APL Products (M46V/D12), IBM Santa Teresa, PO Box 49023, 555 Bailey Avenue, San Jose CA 95161-9023, USA. Tel: 1-408-463-4031. Fax: 1-408-463-3114.
IBM UK Ltd	Nat. Enq. Centre	414 Chiswick High Rd, London W4 5TF Tel: 081-747 0747
Impetus Ltd	Cedric Huddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD Tel: 0732-885126
INFOSTROY	Alexei Miroshnikov	3 S. Tulenin Lane, Leningrad 191186 USSR. Tel: 812-238-6392 Fax: 812-319-9709
Interprocess Systems	Stella Chamberlain	9040 Roswell Road, Suite 690, Atlanta, Georgia 30350-1131, USA Tel: (404) 992-8400
Intelligent Programs Ltd	Mike Bucknall	Unit 7, Hermitage Court, 6-10 Sampson Street, London E1 9NA Tel: 071-481-4813

ISI	Eric Iverson Orders	33 Major Street, Toronto, Ontario, Canada M5S 2K9 Tel:(416) 925-6096 3512 Cameron Mills Road, Alexandria, Virginia, USA 22305-1103 Tel:(703) 548-1799
Mercia Software Ltd.	Gareth Brentnall	Aston Science Park, Love Lane, Birmingham B7 4BJ. Tel: 021-359 5096
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN Tel: 071-922 8866
Ellis Morgan		Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants. Tel: 0730-263843
M.T.I.C.	Ray Cannon	21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS Tel: 0252 874697
Peter Cyriax Systems	Peter Cyriax	12 Gloucester Place, London W1H 3AW Tel: 071-935 2983
Parallax Systems Inc.	Kevin Weaver	Avery Road, Box 319, Garrison, NY 10524, U.S.A. Tel: 914-424-4265
QB On-Line Systems	Phillip Bulmer	5 Surrey House, Portsmouth Rd., Camberley, Surrey, GU15 1LB. Tel: 0276-20789
Renaissance Data Systems	Ed Shaw	P.O. Box 20023, Park West Finance Station, New York, NY 10025-1510, U.S.A.. Tel: (212)864-3078
Reuters Ltd	Paul Jackson	7th Floor B Block, Coventry Point, Market Way, Coventry CV1 1EA. Tel: 0203-256562.
The Rochester Group	Robert Pullman	50 S.Union St., Rochester NY 14607, U.S.A. Tel:716-454-4360 or 716-454-4641
Shandell Systems Ltd.	Maurice Shanahan	Chiltern House, High Street, Chalfont St. Giles, Bucks., HP8 4QH. Tel: 02407-2027. Fax:02407-3118
Sugar Mill Software Corp.	Lawrence H. Nitz	1180 Kika Place, Kailua, Hawaii 96734 Tel: (808) 261-7536
Rex Swain		8 South Street, Washington, CT 06793, U.S.A. Tel:203-868-0131
Sydney APLUG	Rob Hodgkinson	GPO Box 1425, Sydney, NSW 2001, Australia
Uniware	Eric Lescasse	15 Rue Erlanger, 75016 Paris, France. Tel:(1) 45-27-20-61. Fax:(1)45-27-20.61. Telex: 648348F UNIWARE
Wickliffe Computer Ltd	Nick Telfer	76 Victoria Rd., Whitehaven, Cumbria, CA28 6JD. Tel:0946-692568
Warwick Univ.	Prof.Jeff Harrison	Dept of Statistics, University of Warwick, Coventry, CV4 7AL Tel:0203-523369
Zark Incorporated	Gary A. Bergquist	53 Shenipsit St, Vernon CT 06066, USA.

IBM Systems Journal: APL 25th Anniversary Edition

reviewed by Adrian Smith

General Comments

This is clearly the most important collection of APL material for at least the past 10 years. It ranks alongside "A Source Book of APL" as an important milestone in the evolving history of the language, and I feel that anyone with a library of APL books should take steps to add this one to his or her collection.

Because of the importance of this work (and the relative difficulty and expense for non-IBM employees in getting hold of it), the British APL Association is hoping to agree a bulk shipment arrangement with the publisher, so that copies will be available to members at significantly less than the nominal \$20 subscription rate. Details will be advised to members as soon as possible.

The presentation is excellent, with good typesetting of both APL and J, and diagrams up to *Scientific American* standard. There are remarkably few typos (I have just failed to re-find the only one I noticed on first reading), and the balance of material is about right. I was particularly pleased to see that the text is not totally limited to APL2 (NARS, Sharp APL and J all get coverage), and that several of the APL2 papers refer to implementations on small machines. A brief precis of each paper follows.

The Preface

How gratifying to see an explicit reference to the British APL Association, and specifically the ASL project, in the introduction to Norman Thomson's paper. If this helps to get the ASL concept known outside the APL world, it can only do us good.

The IBM family of APL systems

by A.D. Falkoff

A nice easy read, with a lovely diagram illustrating the genealogy of IBM's APL interpreters. Plenty of surprises, and some fascinating new insights on the influence APL has had on the designers of small machines. *Recommended for all readers.*

APL2: Getting started

by J.A. Brown & H.P. Crowder

A gentle introduction to second-generation APL, with some nice examples from graph-plotting to the manipulation of sparse arrays. *Recommended for all except APL2 buffs!*

Extending the domain of APL

by M.T. Wheatley

This is more or less a formalized version of Mike's talk to the British APL Association at Canterbury in 1988. As I said in Vector 5.2 (Nov 88) "A lot of really good hard 'inside information' on the early development of APL, and the way it fitted into its environment. The rationale behind $\square NA$ and the shared variable processors." *Recommended for mainly $\square SVO$ and $\square NA$ freaks.*

Storage management in IBM APL systems

by R. Trimble

This is quite a tough technical paper, but the excellent diagrams help a lot! It could have a major influence on interpreter design in the approaching world of multi-gigabyte workspaces. *Recommended for interpreter writers, and those of the rest of us who like to know how these things really work.*

Putting a new face on APL2

by J.R. Jensen & K.A. Beaty

More or less a rerun of their APL91 paper, but again the high quality diagrams and colour images help to make the point. *Definitely a skim-read for most of us; recommended for X-windows hacks.*

The APL IL Interpreter Generator

by M. Alfonseca, D. Selby & R. Wilks

I first came across this one at Copenhagen (at a G.U.I.D.E. meeting) in 1979! (see 'Design Handbook' Chapter 13 for my thoughts on using APL to run your central heating system.) It is quite hard going, but worth the effort for anyone who wants to know where IBM APL2/PC and APL2/6000 are coming from. *Recommended for all techies and most of the rest of us.*

Parallel expression in the APL2 language

by R.G. Willhoft

If APL is to exploit the parallel architectures of the future, it must build on the high degree of parallelism already implicit in the language. This paper explores the boundaries of present APL2 implementations, and makes a number of important recommendations for future language design. I suspect it may generate quite a vigorous debate! *A skim-read for most of us ... recommended for language designers and anyone interested in the far future of APL.*

The foundations of suitability of APL2 for music

by Stanley Jordan & Erik S. Friis

This is a much easier read than the preceding papers! It shows how APL2 nested arrays can be used at a high level to model underlying musical structures (not just the actual notes). *Recommended (not just for musicians).*

Verification of the IBM RISC System/6000 by a dynamic biased pseudo-random test program generator

by A. Aharon et al

This paper provides the general reader with an excellent introduction to the basic methodology of automated system testing, and then shows how (and why) VS APL and APL2 were used to build the test suite for the RS/6000 architecture. I felt it was most valuable as a primer in automatic verification techniques, rather than for its APL content. *Recommended for anyone interested in systems methodologies.*

APL2 as a specification language for statistics

by N.D. Thomson

Norman briefly introduces the ASL project, and goes on to show how APL2 can be used to specify a collection of basic statistical algorithms. *Recommended.*

Advanced applications of APL: logic programming, neural networks, and hypertext

by M. Alfonseca

The title says it all! The sections on Neural Nets and a Hypertext on-line dictionary are probably those of most general interest. *Recommended for appropriate specialists.*

Language as an intellectual tool: from hieroglyphics to APL

by D.B. McIntyre

If you ever wanted to know how the ancient Egyptians managed to do division (with Hieroglyphs), you should read this paper! It is a fascinating ramble through the long and confused history of mathematical notation, and I would say it has something for everyone. The APL examples are shown in the classic ∇ form, in direct definition and also in J; I found it most refreshing that an IBM publication was able to include so much material from non-IBM versions of APL!

There are over two pages of closely-spaced references which should form a superb starting point for any reader who wants to take the subject further. *Highly recommended, but don't try to take it all in at one sitting.*

A personal view of APL

by K.E Iverson

This paper needs to be read in conjunction with several earlier works (e.g. "The Design of APL" and "The Evolution of APL") to which it refers. It is actually a thinly disguised polemic on where APL went wrong, and why J is better! It discusses in detail such issues as the banning of niladic functions from J, and the ruthless elimination of the $[I; J]$ notation on favour of an indexing function. APLers should be prepared to take several deep breaths, and count to 100 slowly. *Recommended for J freaks and APLers of an equable temperament.*

Summary

Whether or not you agree with everything in it, if you are serious about your APL you should find a way of obtaining this collection of important and influential papers.

Byte 'Classic Languages' Article on APL Byte Magazine, December 1991, pp141 - 146

reviewed by Adrian Smith

Whenever I see 'APL Feature' billed on the cover of a computing magazine, I get that unpleasant sinking feeling! I was half-expecting Byte to run an article on APL in its 'Classic Languages' series, and I knew that Byte was one of the few computing journals to take APL seriously, so I opened my December 1991 issue at page 141 with a mixture of anticipation and dread.

Fortunately, I need not have worried; STSC had helped them over the obvious technical hurdle by providing a reasonable typeface, and Doris Appleby did a really good job of providing the average Byte reader with a broad-brush introduction to the notation.

Her article covers the early history of APL, and reviews the current commercial applications areas. It stresses the power of APL as a prototyping tool, and mentions its recreational value for "people who get a kick out of thinking". Modern implementations (e.g. APL68000, APL2/6000, APL*PLUS and J) get good coverage, as does the 'A' development at Morgan Stanley. I-APL gets an honourable mention, as does APL92 at St Petersburg (the international appeal of APL is noted).

The Byte "Action Summary" concludes ...

If you want to do complex mathematical operations in just a few keystrokes, the APL language is well suited to your needs, but it will require a modified keyboard. Nonetheless, APL is used by specialists throughout the world.

You can't say fairer than that ... well done Byte.

APL Goes Fuzzy

previewed by Oleg Pöhl

Just as the principles of statistical quality control were invented in the West (by Edward Deming and Dr Walter Shewhart in the 1940s) and exploited in the East, so might be the obscure ideas of Fuzzy Logic. The core concepts go back to Poland in the 1920s, with the work of Jan Lukasiewicz, and the theory was developed by Prof. Lotfi Zadeh of the University of California (See Robert Bittlestone's review in Vector [1]). He coined the term "Fuzzy Logic" in his paper in the academic journal "Information and Control" in 1965.

Since that time, fuzzy systems have occasionally turned up in real process control applications, for example running a cement kiln in Copenhagen in 1980, but limitations in available hardware have prevented mass acceptance of the technology. For example, IBM Research developed APL/360 (so called because it *reduced* the power of the System/360) in late 1976, but unfortunately it reduced it so much:

1.2 1.9 3.03 3.98 5 5.92 7.12 8.0002 9.21 9.976 11 12.07

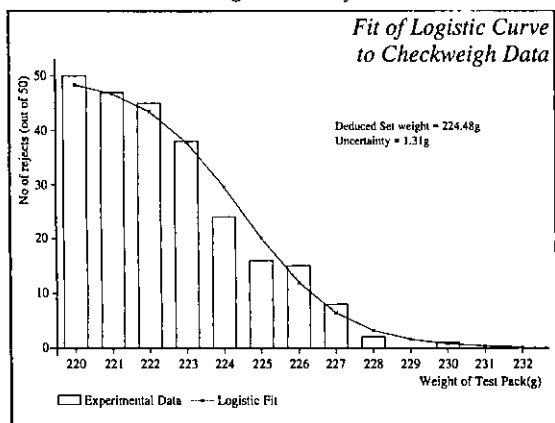
... that an expression like the above took over 3½ hours to execute on a 370/148. Even IBM realised that this was over-doing it, and APL/360 never made it to the marketplace.

Understanding Fuzzy Systems

The best way to approach the ideas of fuzzy logic is to find a really fuzzy system, and watch it working for a while. A typical example was explored in Vector by Smith [2], where APL was used to derive the uncertainty function for an automatic checkweigher. This device is supposed to accept all boxes over a certain weight, and reject all boxes that fall below. A perfect device could be drawn as a step-function, with the probability of acceptance jumping from 0% to 100% as the set weight was passed.

Of course in the real world the machine is subject to random influences (vibrations, small variations in the spacing and orientation of the packs etc.) which 'blur out' this sharp edge. You can demonstrate this effect by making up a set of test packs at known weights which straddle the set point, passing them

repeatedly over the checkweigher in a random sequence, and recording the pass/fail rate at each weight. What you find looks a bit like this:



The fitted curve was generated using the Logistic function:

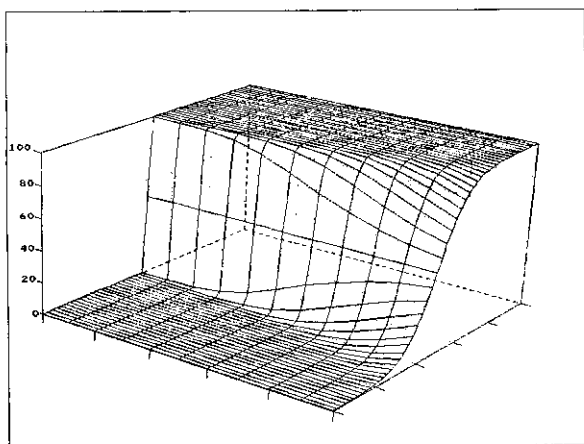
PACKS+219+113 A Set of test packs at 1gm intervals

UNC+1.31 A Uncertainty parameter

SET+224.48 A Best fit to 50% rejection rate

$$FITREJ+100 \times \div 1 + * (PACKWT-SET) \div UNC$$

If you extend the plot to 3 dimensions, the importance of the uncertainty parameter becomes clear:



A Natural Extension to APL

If we discount those first-generation APLs which implemented fuzz accidentally, we find that some kind of nested array structure is essential for an efficient representation of fuzzy data. For some time, IBM Japan have been working on a series of extensions to APL2/PC which exploit the power of the Togai InfraLogic Fuzzy expansion card. Instead of generating simple vectors, functions such as `ρ` produce nested objects, where each element is a 'fuzzed' set of numbers, obeying the logistic probabilities:

```

      ρFUZZ+5 0.5      a each point generates 5 numbers, with U=0.5
      NOS+15
      ρNOS
5      a Shape is not fuzzy!
      ρ"NOS
5 5 5 5 5
      2>NOS
1.91 1.96 2 2.04 2.09

```

Things get interesting when you start doing arithmetic with these fuzzy objects ... the system responds by adding levels of nesting, and effectively producing the 'outer product' of the fuzziness:

```

      XX+NOS*2      a note that '2' is also fuzzy
      ρXX
5
      ρ"XX
5 5 5 5 5
      ρ2>XX
5 5 5 5 5

```

... to stop things getting totally out of hand, the implementors have added a second system variable `ρFUZZDEPTH` which results in the aggregation of the blurring when the arrays get to this depth. By default, `ρFUZZDEPTH+5` which is more than adequate for the majority of applications. The APL/360 example at the top of this preview effectively had `ρFUZZDEPTH+0`, but used a different (randomised) strategy for generating the uncertainty.

Applications

Already, fuzzy technology is on sale in the UK in the shape of Panasonic's new NV-S1 camcorder; this effectively eliminates 'jitter' by storing images and using earlier images to smooth incoming data.

In the APL world, there are obvious uses in the areas of Sales Forecasting, Dealing-room systems, and so on, in fact anywhere where the system must take decisions based on uncertain information.

In a typical forecasting system, there will be an inherent uncertainty associated with each product group (fleet cars are a more reliable market than luxury limousines), and also an extra uncertainty which increases with time. It is also possible that some markets might have more volatile sales patterns than others. If you add to this the effects of currency fluctuations, you soon realise that $\square FUZZDEPTH+5$ is likely to be about right, and that it is essential that the system handles these rapidly ramifying structures for you. In fact, the implementors are experimenting with a further extension to $\square FUZZ$ to allow a 2-column matrix to represent increasing uncertainty with time:

```

      □FUZZ+(4+112), [1.5] 0.5*(112)*.5
      NOS←15
      ρNOS
5
      ρ" NOS           a Still 5 elements
5 6 7 8 9           a but more detail underneath

```

Note that many functions and system variables (including of course $\square FUZZ$ itself) implicitly localise $\square FUZZLEVEL+0!$

Availability

Fuzzy APL will be available exclusively from APL-38(4.5 5 5.5) from 1st April (1994.6 1995 1995.4) (UK and Rockall only)

For more details, contact:

Hugh Phelfrict (no relation),
 Brook House, Gilling East (or maybe West), York
 Tel: 04393-38(4.9 5 5.1)

References

- [1] *Fuzzy Sets and Decision Analysis* by Zimmermann, Zadeh & Gaines, reviewed by Robert Bittlestone. Vector Vol.1 No.4 page 51
- [2] *Using the Logistic Distribution to Model Fuzzy Decisions*; Adrian Smith, Vector Vol.2 No.2 page 109.

J

This section of VECTOR continues the theme of J, with articles following on from the special feature in Vector 8.3

Correspondence ...		
Typesetting J: some Comments	R.H Oates	82
Comments on J Questionnaire	Roger Hui and Ken Iverson	
Comments on Papers in Vector 8.3	Ken Iverson	
Iverson's <i>Arithmetic</i> reviewed	K.W. Smillie	87
Verb Tables	Roger K.W. Hui	91
Using J with External Data	Donald McIntyre	97

In future issues of Vector, we will welcome submitted articles about J, and will give them the same consideration as submitted material in traditional APL.

CORRESPONDENCE

Typesetting J: some Comments

From: R.H. Oates

February 26, 1992

I saw your request for comments on J typefaces in the January Vector. The quote character on page 72 is too light. I was reading Vector on the bus and thought both `f f` expressions had open quotes until I looked over at the APL side. And the bulk of `$` makes that character too dark. It almost looks bold. How is your `@`? You want to establish an even blackness throughout the font, and to do that you have to compensate for the relative bulk of the characters. You can't just make them all two pixels wide, or whatever. It's not easy.

I find Programming in J and Tangible Math easier to read than the Dictionary and Arithmetic because they use bold characters for J (at least in the 1991 editions) consistently throughout. If you go with bold, then the problem becomes generating a bold Dot and Quote, say, that will be blacker than the normal At and Dollar Sign. It gets harder. My own fonts, which I got for APL2, are not very good, even within a font and particularly in the contrast between bold and normal fonts.

The J dictionary does not use bold consistently, just at the heading of each entry, and that is a mistake. Ken is following English dictionary format too closely. When people complain about the dictionary what they usually mention is its terseness. I like the terseness and think the typography is the real problem.

There is more white space (to accommodate figures) within the entries of the J dictionary than in an English dictionary. The contrast between bold and normal in the J dictionary is less than an English dictionary. Entries in the J dictionary divide into sub-entries in small caps, while entries in an English dictionary are homogeneous. Entries in the J dictionary begin either with a single J word or with a phrase like `m : n`. J words average 1.65 characters in length and phrase words average 1.1 characters in length, while English words average maybe 5 or 6 characters.

All of these things conspire to keep the J dictionary page from breaking down into distinct entries like a page in an English dictionary does. The J page is just one big blob. It would be much improved if a blank line were left in front of each entry and if the bold typefont were used consistently throughout for J words.

When I get a new J dictionary the first thing I do is rule it off in black ink. I know that sounds stupid, but I do and it makes a big difference.

Typography is an art, of course, not a science, a profession, actually. But I wish you well and look forward to experimenting with your font.

R.H. Oates,
333 E. 30 St. A18E,
NY, NY 10016, USA.

Typesetter's note: your comments are much appreciated, and I hope I have fixed the obvious infelicities in the design. See Roger Hui's article (page 91) for the revised '\$@%& in action! At the moment, this is implemented as a PostScript type-3 font, using stroked characters (except for simple fills like dot). I am in the process of converting my APL2741 face to an Adobe type-1, which demands filled characters throughout. Fortunately, the font currently exists as numbers in an APL workspace, so much of the geometry is automatic, but I am not clever enough to convert complex shapes like α totally mechanically!

The obvious advantage of a type-1 is that all Windows software will display it (at any sensible size) and will also render it on paper on most reasonable printers (like HP Deskjets) very well. All you need to do is buy Adobe Type Manager, or wait for Windows 3.1 and TrueType. I will try to have both fonts available by APL92.

Comments on J Questionnaire

From: Roger Hui and Ken Iverson

4 February 1992

We were pleased by the interesting articles appearing in the J Section of the January VECTOR, and by the report on the questionnaire. We offer the following comments:

1. "The Atari user did not get an editor at all. Characters can be typed on the command line, and there is a destructive backspace, but nothing else." Terminal input/output are implemented in such a way as to avoid interference with underlying facilities. A line editor would have been available if the system had one; that line editing is not more "user friendly" on the Atari is a deficiency of the underlying system.
2. "... the Suite variable an awkward method of program control". Since the effect of `$.=. label` is equivalent to *branch to label*, it is hard to see what the difficulty is. Perhaps readers are looking for something like *if then else*, *do while*, or *case* statements, matters that are better handled by gerunds combined

with agenda (@.) and power(^:) as illustrated in part by the following examples from *An Introduction to J*:

```
+:'-: '*: '%: @. (4&|@<.)"0 1. 10
0 0.5 4 1.73205 8 2.5 36 2.64575 16 4.5
1: '(J*%:@<:) @. * "0 1. 6
1 1 2 6 24 120
```

3. "... or insist on tacit definition and operators for everything.". Whereas we encourage tacit programming, we do not agree that proscription (even of obscenities) belongs in language design. In particular, beginners in tacit programming find it useful to make an explicit definition in the form `m : d`, and then enter `m : 11` and `d : 11` to see the equivalent tacit forms.
4. "... would love graphics support.". The current availability of the documented J Source (in C) will make it possible for others to port J to further systems, and to provide links to graphics and other resources appropriate to the particular system.
5. "... if the 640K barrier was broken" This barrier was imposed by the limitations, both technical and legal, of the C compilers previously available. J for the 386 is now compiled using WATCOM C386, and the barrier no longer exists.
6. Although McIntyre's use of names such as `lesser_of` is valid, we would advise instead the use of names such as `lesserOf` and `LesserOf`. The reason is that a name with an underscore will be treated as a *locative*; for example, `abc_def` refers to the name `def` in the *locale* (or *file*) `abc`. Locatives are referred to and assigned in the normal manner.

Comments on Power Reduction and Stirling Numbers

From: Ken Iverson

January 12, 1992

I would like to comment on two papers in the October issue of Vector.

I believe that the point of the limits observed by Parkhouse might be clarified by the observation that if repeated application of a function `g` converges to a limit `y`, then `y` is the solution of the equation `y = g y`. In J, experimentation with this can be expressed using the *power conjunction* (dyadic operator) `^:`, and the corresponding *adverb* (monadic operator) `lim=.` `^: _` for the limiting case of infinite power.

Thus:

```

cos=. 2&o.
i=. 0 1 2 3 4 5
cos^: i 1
1 0.540302 0.857553 0.65429 0.79348 0.701369
lim=. ^:_
ly=. cos lim 1
0.739085
y=cos y
1
g=. 1.3&^
g^:i 1
1 1.3 1.40646 1.44629 1.46149 1.46733
g lim 1
1.47099
g lim 1 0 _1 0 j1
1.47099 1.47099 1.47099 1.47099

```

In his *Numerical Methods for Scientists and Engineers* (McGraw-Hill, 1962), R. W. Hamming introduces Stirling numbers in order to relate the coefficients c of a polynomial (expressed in J as $+/c*x \wedge i.\#c$) and the coefficients cf of a factorial polynomial.

If k is zero, the product over x plus k times $iota\ n$ is equivalent to x to the power n ; if k is negative *one*, it is Hamming's factorial function. Moreover, if c is the vector of coefficients of an ordinary polynomial and cf the coefficients of an equivalent factorial polynomial, then cf is given by the matrix product of $S2$ and c , where $S2$ is the matrix of Stirling numbers of the second kind. In J, the *fit* conjunction $!$. produces a "variant" function; in particular, the function $^!._1$ is Hamming's factorial.

Although Hamming says "There is no simple formula for the Stirling numbers ...", the relations outlined above lead to an expression of the form b times the inverse of a , where a is a Vandermonde matrix, and b is the corresponding "Vandermonde matrix" for the factorial polynomial. In J, the matter may be expressed as follows:

```

vm=. ^!._0/ i.@#
fvm=. ^!._1/ i.@#
x=. 2 3 4 [ c=. 1 5 10 10 5 1

x vm c
1 2 4 8 16 32
1 3 9 27 81 243
1 4 16 64 256 1024

(x vm c) +/ . * c
243 1024 3125
(x+1)^5
243 1024 3125

```

```

i=. i. 6
a=. i vm i
b=. i fvm i

a
1 0 0 0 0 0
1 1 1 1 1 1
1 2 4 8 16 32
1 3 9 27 81 243
1 4 16 64 256 1024
1 5 25 125 625 3125

b
1 0 0 0 0 0
1 1 0 0 0 0
1 2 2 0 0 0
1 3 6 6 0 0
1 4 12 24 24 0
1 5 20 60 120 120

S1=. b %. a
clean=. ".@(0&" :)
clean S1
1 0 0 0 0 0
0 1 _1 2 _6 24
0 0 1 _3 11 _50
0 0 0 1 _6 35
0 0 0 0 1 _10
0 0 0 0 0 1

S2=. %. S1
cf=. S2 +/ . * c
cf
1 31 90 65 15 1
(x fvm cf) +/ . * cf
243 1024 3125

```

The matrices $S1$ and $S2$ are Stirling numbers of the first and second kinds. Since Abramowitz and Stegun (and consequently Appleton) do not include the leading row and column, Appleton's result is given by $\} \}$. " $S1$ ". This formulation does not provide transformation of the constant term which, of course, remains the same in the two types of polynomial. Finally, functions to generate the Stirling numbers (as function of the order) are defined by:

```
ST2=. %.@ST1=. %./@(0 1&|:@(^!._1 0/~@i.))
```

Roger Hui & Ken Iverson
Apt 405
70 Erskine Avenue
Toronto, Ontario
Canada M4P 1Y2

Ed: The J is version 4.1x2, dated 11 January 1992.

Review: Iverson, Kenneth E., *Arithmetic*.

Iverson Software Inc., 33 Major Street, Toronto M5S 2K9, Canada. 118 pp., \$40.00 + 7% GST. UK Agent: I-APL Ltd (see Product Guide)

reviewed by K.W. Smillie

In 1889 Giuseppe Peano published his *Principles of Arithmetic* in which he attempted to put arithmetic on a rigorous axiomatic foundation. It is a small work of some thirty-six pages, the first twelve of which are devoted to defining the notation used and presenting an algebra of propositions. It is only then that Peano begins to construct the number system from the three primitive concepts of "zero", "non-negative whole numbers" and "is the successor of" which satisfy the five postulates which follow. He then proceeds to lay the foundations for arithmetic as a sequence of theorems expressed and proven in symbolic form with scarcely an explanatory note in natural language. He used this symbolism in his lectures at the Military Academy in Turin. When his students rebelled and could not be satisfied by all being given passing grades, Peano resigned and returned to the University of Turin where he remained a professor until his death in 1932.

Kenneth Iverson, possibly mindful of Peano's experiences, introduces the positive integers in the opening sentence of *Arithmetic*:

The list 1 2 3 4 5 6 7 8 9 10 11 12 shows the first dozen counting numbers, and any reader of this book could extend the list to tedious lengths.

He then asks the following three questions: "Do counting numbers continue forever?"; "Are there other numbers that precede the first counting number?"; and "Are there other numbers between the counting numbers or elsewhere?" In the next eighteen pages the reader is introduced, as an answer to these questions, to the successor and predecessor of a counting number, zero and negative numbers, addition and subtraction, relations, function tables, multiplication as repeated addition, and raising to an integral power as repeated multiplication.

The notation used throughout is, of course, J, and by the end of the first chapter the reader becomes familiar with the terminology and syntax of the language and a selection of the primitives. Unlike Peano, however, Iverson introduces concepts and notation only as needed. In Chapter 1 only 18 verbs, many of which are familiar from conventional notation, are used, together with the adverbs *insert* and *table* represented by /, and the conjunctions *atop* @ and *power* ^:. The

introduction of defined verbs, called *proverbs*, is simplified by tacit definition with no explicit mention of their arguments. The chapter ends, as do all chapters except Chapter 5, with a selection of exercises to be done both with pencil and paper and with a computer.

In Chapter 2 the concept of a *bond* is introduced which allows verbs to be ambivalent with the valency being determined from the context. Moreover, the bonding conjunction *with*, represented by $\&$, allows nouns to be bonded on either the left or right to verbs to produce monadic verbs or *monads*, as, for example, $\text{sq} = . \ \wedge \& 2$ to give squares of the argument, and $\text{p}2 = . \ 2\& \wedge$ which gives powers of 2 of the argument. The concept of bonding permits in the discussion which follows the definition of left and right distribution and left and right identities. All of this is accomplished with the introduction of only two new nouns ($_$ and $__$) for the identity elements for minimum and maximum, the two verbs *take* { . and *drop* } ., and the adverb *commute* ~.

Chapter 3 presents the adverbs *prefix* \ , *suffix* \. and *oblique* / ., and the verbs *from* { , *grade* and *sort* / : , *residue* | , *box* < and *open* > , and *word formation* :: which are required in later chapters. Chapter 4 uses the *base* and *antibase* verbs, #. and #:, in a treatment of integer representation and an analysis of integer multiplication. It is here that the important concept of a *train* of isolated verbs is introduced, and diagrams for monadic and dyadic forks are given although the term "fork" is not used. We might note that the use of a fork and the bonding of a noun to a verb permits the definition of a second-degree polynomial with roots 2 and 4 to be given in Chapter 8 as $\text{PARABOLA} = . \ -\& 2 \ * \ - \ \& 4.$

Chapter 5, "Proofs", is concerned with the nature of mathematical proofs, and the use of J in investigating informal proofs. The author has been influenced by the work of the mathematician Imre Lakatos whom he refers to in the Preface, and gives some of Lakatos' discussion from *Proofs and Refutations* of the Euler-Descartes formula relating the number of vertices, edges and faces of a polyhedron.

Persons unfamiliar with Lakatos' work, and this reviewer was one, could read the article "Lakatos and the philosophy of dubitability" in Davis and Hersch's *The Mathematical Experience*. There is much of interest in this absorbing book, and readers of this review might consider the relevance to APL and J of the following two sentences which occur in the short article "Symbols" in the same book:

"Yet it seems at times that symbols return more than was put into them, that they are wiser than their creators. There are felicitous or powerful mathematical symbols that seem to have a kind of hermetic power, that carry within themselves seeds of innovation or creative development."

Each of the last four chapters gives a development of a particular topic: propositional calculus and boolean algebra in Chapter 6, permutations and combinations in Chapter 7, classifications and sets in Chapter 8, and polynomials in Chapter 9. Additional material from J is introduced as needed.

In Chapter 6, to give just one instance, the justification of the verbs *or* +. and *and* *. is followed by the introduction of the adverb *boolean* b. which may be used to select any one of the sixteen boolean dyads, so that, for example, *and* = . 1 b. defines *and* and *and* / ~ 0 1 is the corresponding truth table.

Chapter 9 concludes with a discussion of a number of aspects of J not included previously, including the explicit definition of verbs, the definition of abverbs and conjunctions, and gerunds.

The theme of this monograph is the treatment of selected topics in arithmetic using J as both a notation and a programming language. Iverson states this position very clearly in Section R of Chapter 1, the first two paragraphs of which are as follows:

Notation, the term normally used to refer to the mode of expression in math, is defined [in the *American Heritage Dictionary* which this reviewer suspects is one of Iverson's favourite references] as "A system of figures or symbols used in specialized fields ...". An executable notation such as that used here is normally called a *programming language*; we will use the terms *notation* and *language* interchangeably.

Programming languages are commonly taught in specific courses, prerequisite to courses in topics that employ them. In mathematics, on the contrary, notation is not taught as such, but is introduced in passing as required by the subject. The same approach is adopted in this text.

Iverson succeeds in his aim of subordinating the notation to the subject matter so that the reader learns the language at the same time as gaining fresh insights into topics many of which are familiar from primary school. Comparison with *Programming in J*, which is a valuable reference for those using J, shows the advantages of introducing a programming language in a specific context. Furthermore, comparison with Iverson's Turing Award Lecture, "Notation as a tool of thought", which covers many of the same topics as the present monograph, shows how the language has evolved in the last ten years.

Arithmetic will be of interest to any person who wishes to learn about J, to teachers who are interested in trying a different approach to introducing some basic mathematics and computing in their classes, and to those already using J who wish to polish their programming and writing styles. This reviewer found

the monograph a most suggestive work in the meaning of the first definition of "suggestive" given in the *AHD*: "Tending to suggest thoughts or ideas". *Arithmetic* is an important and stimulating work, and deserves the serious attention of the APL community.

Keith Smillie
Department of Computing Science
University of Alberta
Edmonton, Alberta T6G 2H1
Canada

References

- [1] Davis, Philip J. and Hersch, Reuben, 1981. *The Mathematical Experience*. Birkhäuser, Boston.
- [2] Iverson, K. E., 1980. "Notation as a tool of thought". Communications of the Association for Computing Machinery, vol. 23, no. 8, pp. 444 - 465.
- [3] Iverson, K. E., 1991. *Programming in J*. Iverson Software Inc., Toronto.
- [4] Kennedy, Hubert C., 1973. *Selected Works of Giuseppe Peano*. University of Toronto Press, Toronto.
- [5] Kline, Morris, 1972. *Mathematical Thought from Ancient to Modern Times*. Oxford University Press, New York.

Verb Tables

by Roger K.W. Hui

The verb tables produced by *f/* in J (see Reference [1]; *f/* is *o.f* in APL) are generalizations of the familiar addition and multiplication tables of arithmetic. The utility verbs *over* and *by* (defined and analyzed in the Appendix) are used to annotate verb tables. For example:

a by b over (a=.2 3 5)+/b=.1.5

	0	1	2	3	4
2	2	3	4	5	6
3	3	4	5	6	7
5	5	6	7	8	9

Verb tables facilitate systematic experimentation, and can reveal new patterns and new meanings even for familiar and seemingly quotidian verbs.

Cross (~) is an adverb such that *f~y* is *y f y* and *x f y* is *y f x*, and is useful in generating verb tables of a list against itself. Thus:

y by y over y */ y=.2--1.5

	_2	_1	0	1	2
_2	4	2	0	_2	_4
_1	2	1	0	_1	_2
0	0	0	0	0	0
1	_2	_1	0	1	2
2	_4	_2	0	2	4

y by y over %/~ y

	_2	_1	0	1	2
_2	1	2	__	_2	_1
_1	0.5	1	__	_1	_0.5
0	0	0	0	0	0
1	_0.5	_1	__	1	0.5
2	_1	_2	__	2	1

2 1 0 </2 1 3 0 1 2
 0 0 1 0 0 0
 1 0 1 0 0 1
 1 1 1 0 1 1

*/~0 1 +./~0 1
 0 0 0 1
 0 1 1 1

Note the pattern of signs in both the *times* (*) and *divide* (%) tables, the result of 0%0, and the infinite possibilities of dividing by 0 [3,4]. The signs of a real table *t* can be displayed using the expression ' +-' {~*t, using the *signum* (*) of *t* to select from (t) ' +-' . Tables of boolean verbs (*truth tables*) such as *less than* (<), *and* (*.), and *or* (+.) offer useful classifications.

Out of (1) produces binomial coefficients; !/ produces Pascal's triangle. 5&|@* is multiplication modulo 5:

y by y over !/~y=.4~-1.9

	_4	_3	_2	_1	0	1	2	3	4
_4	1	3	3	1	0	0	0	0	0
_3	0	1	2	1	0	0	0	0	0
_2	0	0	1	1	0	0	0	0	0
_1	0	0	0	1	0	0	0	0	0
0	1	1	1	1	1	1	1	1	1
1	4	3	2	1	0	1	2	3	4
2	10	6	3	1	0	0	1	3	6
3	20	10	4	1	0	0	0	1	4
4	35	15	5	1	0	0	0	0	1

y by y over 5&|@*/~y

	_4	_3	_2	_1	0	1	2	3	4
_4	1	2	3	4	0	1	2	3	4
_3	2	4	1	3	0	2	4	1	3
_2	3	1	4	2	0	3	1	4	2
_1	4	3	2	1	0	4	3	2	1
0	0	0	0	0	0	0	0	0	0
1	1	2	3	4	0	1	2	3	4
2	2	4	1	3	0	2	4	1	3
3	3	1	4	2	0	3	1	4	2
4	4	3	2	1	0	4	3	2	1

a =. -: 4~-1.9

b =. -: 3~-1.7

a by b over a ^/ b

	_1.5	_1	_0.5	0	0.5	1	1.5
_2	0j0.353553	_0.5	0j_0.707107	1	0j1.41421	_2	0j_2.82843
_1.5	0j0.544331	_0.666667	0j_0.816497	1	0j1.22474	_1.5	0j_1.83712
_1	0j1	_1	0j_1	1	0j1	_1	0j_1
_0.5	0j2.82843	_2	0j_1.41421	1	0j0.707107	_0.5	0j_0.353553
0			_1	1	0	0	0
0.5	2.82843	2	1.41421	1	0.707107	0.5	0.353553
1	1	1	1	1	1	1	1
1.5	0.544331	0.666667	0.816497	1	1.22474	1.5	1.83712
2	0.353553	0.5	0.707107	1	1.41421	2	2.82843

The *power* (^) table contains many subtle patterns. For example, examine the column headed by 0, and the rows headed by _1 and 0. The reader may wish to attempt the same table on other systems; the results may prove interesting.

The verb j. produces a complex number from its real and imaginary parts; thus 3 j. 4 is 3j4. The expression (1.y) <.@j.~/ y=.5~-1.5 produces a table of the verb <.@j.~/ on the *reverse* (1.) of y against y itself; i.e., *floor* (<.) on the lower left corner of the first quadrant of the complex plane. The pattern is made more evident by mapping the result to single glyphs, using the symbols s=. ' @.+&- \$*=%'.

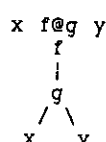
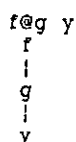
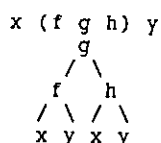
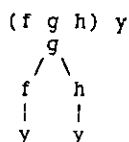
(|.y) by y over (|.y) <.@j.-/ y =. 5%~1.5

	0	0.2	0.4	0.6	0.8
0.8	0	0j1	0j1	0j1	1
0.6	0	0	0j1	1	1
0.4	0	0	0	1	1
0.2	0	0	0	0	1
0	0	0	0	0	0

```
s =. ' @.+&- $*=%'
map =. {&s @ (~. @, 1. ] )
map (|.y) <.@j.-/ y=.10%~1.20
```

```
@@@@@@@@.....++++++&
@@@@@@@@.....++++++&&
@@@@@@@@.....++++&&&
@@.....++&&&&
.....&&&&
.....&&&&
.....&&&
.....&&
.....&
.....
-      $$.
--     $$$$.....**
---    $$$$$$.....**
----   $$$$$$$$.....**
----- $$$$$$$$$$.....**
----- $$$$$$$$$$.....**
----- $$$$$$$$$$.....**
----- $$$$$$$$$$.....**
----- $$$$$$$$$$.....**
----- $$$$$$$$$$.....**
```

Thus, the floor of the complex plane is tiled by "sloping bricks" — unit-area rectangles inclined at minus 45°, whose long sides are the diagonal of the unit square [2]. *map* is a composition of two verbs, {&s (*from* bonded with *s*) atop the fork ~. @, 1.]; it works by computing the indices of the floor values in the nubbed ravel (~. @,) of such values, which are then used to select from *s*. *Fork* and *atop* are defined by the following diagrams:



Appendix: over and by

```
a =. _0.5 0 0.5
b =. _1 _0.5 0 0.5
```

```
a^/b
_2 0j_1.41421 1 0j0.707107
_2      1.41421 1 0.707107
```

```
] x =. b, a^/b
_1      _0.5 0      0.5
_2 0j_1.41421 1 0j0.707107
_2      1.41421 1 0.707107
$ x
4 4
```

```
{. y
_1      _0.5 0      0.5
```

```
({.y) ; }.y
```

_1	_0.5 0	0.5	_2 0j_1.41421 1 0j0.707107
			_2 1.41421 1 0.707107

```
colhead =. {. ; }.
colhead y
```

_1	_0.5 0	0.5	_2 0j_1.41421 1 0j0.707107
			_2 1.41421 1 0.707107

```
colhead ": b , a^/b
```

_1	_0.5 0	0.5	_2 0j_1.41421 1 0j0.707107
			_2 1.41421 1 0.707107

```
rowhead =. ' ' &; @ ,.
colhead =. {. ; }.
over    =. colhead @ ": @ ,
by      =. rowhead@[ ,. ]
```

```
] y =. ": x
_1      _0.5 0      0.5
_2 0j_1.41421 1 0j0.707107
_2      1.41421 1 0.707107
$ y
4 26
}. y
_2 0j_1.41421 1 0j0.707107
_2      1.41421 1 0.707107
```

```
over =. colhead @ ": @ ,
b over a^/b
```

_1	_0.5	0	0.5	_2	0j_1.41421	1	0j0.707107
						1	0
				2	1.41421	1	0.707107

```
2 1 $ b over a^/b
```

_1	_0.5	0	0.5
_2	0j_1.41421	1	0j0.707107
		1	0
2	1.41421	1	0.707107

```
..a          $ ..a
_0.5          3 1
0
0.5
```

```
' ' ; ..a
```

_0.5
0
0.5

```
rowhead =. ' ' & @ ,
rowhead a
```

_0.5
0
0.5

```
2 1 $ rowhead a
```

_0.5
0
0.5

```
(rowhead a) .. c=.b over a^/b
```

	_1	_0.5	0	0.5
_0.5	_2	0j_1.41421	1	0j0.707107
0			1	0
0.5	2	1.41421	1	0.707107

```
1 2 3 .. 9 10 11
```

```
1 9
2 10
3 11
```

```
$ 1 2 3 .. 9 10 11
3 2
```

```
(rowhead a[c] .. a]c
```

	_1	_0.5	0	0.5
_0.5	_2	0j_1.41421	1	0j0.707107
0			1	0
0.5	2	1.41421	1	0.707107

```
1 2['abc'
```

```
1 2
```

```
1 2] 'abc'
```

```
abc
```

by =. rowhead@[,.]
a by c

	_1	_0.5	0	0.5
_0.5	_2	0j_1.41421	1	0j0.707107
0			1	0
0.5	2	1.41421	1	0.707107

a by b over a^/b

	_1	_0.5	0	0.5
_0.5	_2	0j_1.41421	1	0j0.707107
0			1	0
0.5	2	1.41421	1	0.707107

References

[1] Iverson, K.E., *ISI Dictionary of J*, Version 4, 1991 12.

[2] McDonnell, E.E., *Complex Floor*, APL73 Conference Proceedings.

[3] McDonnell, E.E., *Zero Divided by Zero*, APL76 Conference Proceedings.

[4] McDonnell, E.E., and Shallit, J.O., *Extending APL to Infinity*, APL 80, North-Holland Publishing Company, 1980 6.

Using J with External Data: Two Examples

by Donald B. McIntyre

Introduction:

J is a powerful dialect of APL created by Kenneth Iverson and Roger Hui [1]. At the APL91 Conference at Stanford, California, I gave an account of my experience with the language [2]. Some of the difficulties I met when first trying to use J were due to features that have been modified or dropped from the language; for example, the manner in which one function inherited the rank of another. Although there was ample theoretical justification for the original way rank was inherited (and I gave an example of this in [2]), my experience helped to persuade the designers to change the rules. Because J is developing fast, it is essential to specify the version used. The files *Status.Doc* and *Xenos.Doc*, provided with the system, document changes from earlier versions and should be examined closely. The examples given here were executed with Version 4.1x2

The special issue of the *IBM Systems Journal*, commemorating 25 years of implemented APL, contains several examples of the same function, or *verb*, written in conventional APL, Direct Definition, and J [3]. These may be helpful to those already familiar with APL. In an earlier paper in *Vector* I have provided an introduction to J with examples from elementary arithmetic [4]. The standard references written by Iverson are, of course, essential reading [5-9]. References to some of the more technical papers will be found in reference [4].

Dictionaries like *The Oxford English Dictionary* and Lewis and Short's *Latin Dictionary* not only define words but include phrases illustrating their use by standard authors. It would be impractical to attempt to learn a language from such sources alone. It is necessary to read literature written in the language at an appropriate level, and preferably dealing with a topic already understood by the reader. This paper is intended as a further contribution to the literature for beginning users of J.

I always find that, like my students, I benefit most from examples in a context to which I can relate. In the context of two practical examples, I give here examples of features of J, including hooks and forks, rank, cut, raze, ravel items, format, do (execute), copy (compress), box, open, under (dual), amend, and the reading and writing of ASCII files. I hope that others will find these helpful as they learn J.

In every definition only *necessary* parentheses are used. It is helpful to experiment by starting with fully parenthesised expressions and gradually

reducing the number of parentheses included. The successive definitions should be checked to see whether they give the same result as before, and when they do not they should be displayed in boxed and tree form to see how they have been interpreted by the J system [4].

The First Problem Stated:

A friend, who was starting to investigate J, had monthly total rainfall measurements for a period of eight years. He had used a word processor to put the values into an ASCII file and he wondered how he could use J to compute various statistics. Missing observations were indicated by a letter. This is how typical data appear in a file named `data.in`:

3.02	0.88	6.69	1.78	3.48	3.37	3.81	1.57
4.33	1.59	2.72	1.74	0.86	0.65	X	0.87
0.22	2.77	2.66	2.08	1.65	2.48	X	3.41
1.74	5.33	2.44	1.89	2.91	1.93	0.48	3.72
0.25	0.09	2.33	1.67	1.40	1.54	2.37	4.18
2.28	2.23	0.77	4.01	1.48	2.88	1.47	0.13
0.94	X	2.72	5.11	X	2.46	1.54	0.56
2.03	0.99	3.27	2.35	3.29	3.55	0.98	0.78
1.26	2.07	4.02	1.09	1.84	0.37	2.30	1.32
3.82	2.06	2.33	7.00	3.10	0.66	4.28	2.14
3.08	1.54	0.88	2.60	4.20	2.52	3.82	1.22
2.28	4.09	1.19	0.76	2.39	4.94	2.23	3.06

Input from ASCII to J:

J has been criticised for having too few system functions, but this is a misunderstanding. APL's system functions are not really part of the language; in J they are implemented with the "foreign" conjunction (`!:`). As the name suggests, conjunctions are dyadic; they *join together* two arguments producing verbs (functions) or adverbs (monadic operators). Thus `1!:1` is *file read*, and the argument of the resulting verb is the boxed specification of the file. Similarly `1!:2` is *file write*, a dyadic verb, whose left argument is the character string to be written, and right argument the boxed file specification. It is convenient to name these as follows:

```
read=. 1!:1 @ <      NB. x=. read 'input.fil'
write=. [ 1!:2 <@]   NB. 'string' write 'output.fil'
```

In my experience it is rather easy, even for a beginner, to think in terms of a fork, and the verb *write* is defined here as a fork. When I began to work with J I thought that both forks and hooks were abstruse concepts unlikely to be useful

to someone at my level. Very quickly, however, I found forks everywhere. For a long time I thought that hooks were much rarer. The reason is that every hook can be written as a fork, and it easier for the novice to use the fork form; as indeed I have done here.

When a fork consists of the composition of a dyadic verb, a monadic verb, and either a *left* (t) or a *right* (J) verb, then we can use the hook form. In the dyadic case (as in the verb *write*):

x (g h) y is the same as x g (h y)

which implies that *g* is dyadic and *h* is monadic. Thus:

write=. 11:2 < NB. Write to a file. Hook

I shall draw attention to similar examples as they occur.

Execution of the following sentence causes the data to be read as a character string named *y*.

y=. read 'data.in'

y is the name of a character string containing line-feed, new-line, and end-of-file codes which are as follows (where *a.* is the Atomic Vector, or list of the 256 ASCII codes, and { is the verb *from* used by J instead of *a*[13] for indexing):

n1=. 13 { a. [lf=. 10 { a. [eof=. 26 { a.

Replacement of the Character X to Characters _1:

The indexes of the character *X* in the string are given by:

Ji=. u # i. # u=. y = 'X'
130 202 450 474

Note that where # is preceded by 1. it is the monadic *tally*; where it is preceded by *u* it is the dyadic *copy* (equivalent to *compress* or *replicate* in older APL).

Because rainfall is never negative, we can replace the letter *X* with the pair of characters *_1* as a step in converting the character string into numbers amenable to arithmetic. The indexes of all positions to be changed are found by using the *table* adverb (/) to produce the outer product.

,1 +/0 1
130 131 202 203 450 451 474 475

The information to be entered at these positions is:

```
(+:#1)$'_1'
_1_1_1_1
```

Substitution is then made by *amend*. For further examples see [2, p.270], but note that *amend* was changed in Version 4 [10].

```
z=. ((+:#1)$'_1') (,1 +/0 1))y
```

Defining verbs to accomplish this, we can proceed as follows:

```
h=. 'X'&=@] # i.@#      NB. Fork
h y
130 202 450 474

g=. .@(+/&0 1)@h
g y
130 131 202 203 450 451 474 475

f=. $&'_1' @ (+:##@h)
f y
_1_1_1_1
```

Then:

```
z=. (f y) (g y)) y
```

The explicit form is straightforward:

```
amend=. '(f y.) (g y.)) y.' : ''
z=: amend y
1
```

It was simple to amend a matrix with earlier versions of J using desk calculator mode [2, p.270]. The syntax was `x i} m`, where *i* gave boxed indexes. But a definition in tacit form cannot be written easily. Four hours after discussing this case with Ken Iverson and Roger Hui, they telephoned back to say that they had extended J (Version 4) to permit definition of the following fork:

```
amend=. f g@]] ]
z=: amend y
1
```

This is dramatic testimony to the superior design of the language, the flexibility of the implementation, and the response and skill of both designer and implementer!

Conversion from Character String to Numeric Table:

z is a string of numbers, represented in character form and containing printer-control characters. Reshape it into a 12 by 8 table after converting from character representation to numeric, and after printer-control characters have been removed:

```
d=. 12 8 $ ". z #- -. z e. lf,nl,eof
```

We can, however, make this table without explicitly specifying its shape. Take a simple example to illustrate the procedure:

```
s=. '1 2 3',lf,'4 5 6',lf,'7 8 9',lf
cut=. <;_2
```

The conjunction *cut* will box items delimited by the last item (2) — in this case *lf* — and exclude (*_2*) the delimiters from the intervals:

```
cut s
```

1 2 3	4 5 6	7 8 9
-------	-------	-------

Raze the result:

```
; cut s
1 2 34 5 67 8 9
```

Ravel items:

```
,. cut s
```

1 2 3
4 5 6
7 8 9

Open:

```
> cut s
1 2 3
4 5 6
7 8 9
```

Do, or execute (".) under (&.) open (>); i.e., open each box, convert to numeric, and then close each box again:

```
$ ".&.> cut s
3
```

Open the result of this operation and show that we have produced the required numeric table:

```
execute=. > @ (".&.> @ <;._2)
1 + execute s
2 3 4
5 6 7
8 9 10
```

To apply *execute* to our data, begin by eliminating the new-line and end-of-file characters while leaving the line-feeds as delimiters:

```
d -: data=. execute z#~ -. z e. nl.eof
1
```

```
data
3.02 0.88 6.69 1.78 3.48 3.37 3.81 1.57
4.33 1.59 2.72 1.74 0.86 0.65 _1 0.87
0.22 2.77 2.66 2.08 1.65 2.48 _1 3.41
1.74 5.33 2.44 1.89 2.91 1.93 0.48 3.72
0.25 0.09 2.33 1.67 1.4 1.54 2.37 4.18
2.28 2.23 0.77 4.01 1.48 2.88 1.47 0.13
0.94 _1 2.72 5.11 _1 2.46 1.54 0.56
2.03 0.99 3.27 2.35 3.29 3.55 0.98 0.78
1.26 2.07 4.02 1.09 1.84 0.37 2.3 1.32
3.82 2.06 2.33 7 3.1 0.66 4.28 2.14
3.08 1.54 0.88 2.6 4.2 2.52 3.82 1.22
2.28 4.09 1.19 0.76 2.39 4.94 2.23 3.06
```

Replace Negative Values by Row Means:

Substituting the most reasonable values for missing data is a matter that requires knowledge of the subject matter. In this case the columns are years (8) and the rows, or items, are months (12). We can replace missing values by the means for the row (month) over all years for which we have data.

We need the indexes of missing values (represented by _1). First write a directly executable expression and then define a verb to give the same result:

```
(,u)#i.#,u=. 0>data
14 22 49 52
h=. 0&>@,
g=. (] # i.@#)@h NB. A fork that can be written as a hook
```

Because *g* consists of a dyadic verb (*#*), a monadic verb (*i.*), and the verb *right* (*]*), it can be written as a hook.

```
g=. (# i.@#)@ h      NB. Hook
```

Fix *g* as *ir* (Indexes in Ravel) so that the names *g* and *h* can be reused.

```
ir=. g f.
ir data      NB. Index in Ravel of matrix
14 22 49 52
```

The row indexes are:

```
<.8%~ir data
1 2 6 6
(<.@(%88)) ir data
1 2 6 6
```

The column indexes are:

```
8|ir data
6 6 1 4
```

To find the indexes we made explicit use of the number of columns (8), but this number is given by:

```
{:$ data
8
```

Define verbs *row* and *column* depending solely on the data:

```
row=. <. @ (ir % {:@$)      NB. Fork
row data
1 2 6 6

column=. {:@$ | ir          NB. Fork
column data
6 6 1 4
f=. row ,. column          NB. Fork
f data
1 6
2 6
6 1
6 4
```

Note that *row*, *column*, and *f* are forks. Now box the items (rows):

```

ix=. <"1 @ (row .. column)
ji=. ix data


|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 1 | 6 | 2 | 6 | 6 | 1 | 6 | 4 |
|---|---|---|---|---|---|---|---|


jz=. i{data
-1 -1 -1 -1
z-: ({~ ix) data      NB. Hook
1
z-: (ir data){data
1
z-: (ir ( ,) data     NB. Fork
1

```

Compute the means of the rows with negative values.

```

mean=. +/%#                      NB. Fork
jm=. mean"1 (row data){data
1.47 1.78375 1.41625 1.41625

```

Note that `(row data){ data` is a candidate for a monadic hook:

```

m-: mean"1 ({~ row) data
1

```

But these means were computed over the eight values in the rows, and these include the `_1`, which is only a flag indicating a missing observation. We need a special mean that will be based only on positive or zero values:

```

mean n=. 1 _9 2 3 _1 4 0 5
0.625
(>:80 # j) n      NB. Fork
1 2 3 4 0 5

```

Every hook can be written as a fork, and the fork is often easier to grasp than the hook that could replace it. Remember that in the hook `(g h)`, `h` is always monadic and `g` is dyadic [4]. Arrange the two verbs in the order, from left to right, *Dyadic*, *Monadic*. If the dyadic function is not commutative, it may be necessary, as in this case, to use the *cross* adverb `(~)`.

```

h=. >:80                      NB. Monadic
h n
1 0 1 1 0 1 1 1
g=. #~                          NB. Dyadic
n g (h n)
1 2 3 4 0 5
(g h) n                        NB. Hook
1 2 3 4 0 5
(#~ >:80) n                    NB. Hook
1 2 3 4 0 5

```

Mean over positive and zero values:

```
pzmean=. mean @ (#~ >:0)      NB. Hook
pzmean n
2.5

jm=. pzmean"1 ({~ row) data
1.82286 2.18143 2.22167 2.22167
```

Amend the data by inserting these values. Note that the indexes (1) are with respect to the ravel of the matrix (data).

```
jx=. m 1}data
3.02 0.88 6.69 1.78      3.48 3.37      3.81 1.57
4.33 1.59 2.72 1.74      0.86 0.65 1.82286 0.87
0.22 2.77 2.66 2.08      1.65 2.48 2.18143 3.41
1.74 5.33 2.44 1.89      2.91 1.93      0.48 3.72
0.25 0.09 2.33 1.67      1.4 1.54      2.37 4.18
2.28 2.23 0.77 4.01      1.48 2.88      1.47 0.13
0.94 2.22167 2.72 5.11 2.22167 2.46      1.54 0.56
2.03 0.99 3.27 2.35      3.29 3.55      0.98 0.78
1.26 2.07 4.02 1.09      1.84 0.37      2.3 1.32
3.82 2.06 2.33 7          3.1 0.66      4.28 2.14
3.08 1.54 0.88 2.6        4.2 2.52      3.82 1.22
2.28 4.09 1.19 0.76      2.39 4.94      2.23 3.06

amend=. {r@j}
x:-: m amend data
1
```

Catenate Month and Year Means:

The verb ymean is a hook that catenates yearly means to the foot of the columns:

```
ymean=. , "2 1 mean
ymean x
```

Catenate the means of rows (months), the means of columns (years), and the grand mean (at bottom right corner):

```
js=. 6.2":((,"2 1 mean) x), "1 0 (mean"1 x), mean ,x
3.02 0.88 6.69 1.78 3.48 3.37 3.81 1.57 3.07
4.33 1.59 2.72 1.74 0.86 0.65 1.82 0.87 1.82
0.22 2.77 2.66 2.08 1.65 2.48 2.18 3.41 2.18
1.74 5.33 2.44 1.89 2.91 1.93 0.48 3.72 2.55
0.25 0.09 2.33 1.67 1.40 1.54 2.37 4.18 1.73
2.28 2.23 0.77 4.01 1.48 2.88 1.47 0.13 1.91
0.94 2.22 2.72 5.11 2.22 2.46 1.54 0.56 2.22
2.03 0.99 3.27 2.35 3.29 3.55 0.98 0.78 2.15
1.26 2.07 4.02 1.09 1.84 0.37 2.30 1.32 1.78
3.82 2.06 2.33 7.00 3.10 0.66 4.28 2.14 3.17
3.08 1.54 0.88 2.60 4.20 2.52 3.82 1.22 2.48
2.28 4.09 1.19 0.76 2.39 4.94 2.23 3.06 2.62
2.10 2.16 2.67 2.67 2.40 2.28 2.27 1.91 2.31
```

This illustrates the verb *format* (";), the conjunction *rank* (") applied to *catenation* (,), and the *hook* ("2 1 mean)

We now have a character array. Catenate to it the new-line and line-feed codes, and *ravel* to produce an ASCII string that can be sent to DOS:

```
(.s,"1 nl,lf) write 'data.out'
```

The file `Data.out` is the required ASCII file.

Using *Box* (<) for further Formatting:

```
f=. <@(6.2&":)
g=. f&:(,:@ mean"1) ,: f@ mean @ ,      NB. Fork
h=. f , f@ mean                          NB. Fork
table =. h ,"0 1 g                      NB. Fork
```

```
]t=. table x
```

3.02	0.88	6.69	1.78	3.48	3.37	3.81	1.57	3.07
4.33	1.59	2.72	1.74	0.86	0.65	1.82	0.87	1.82
0.22	2.77	2.66	2.08	1.65	2.48	2.18	3.41	2.18
1.74	5.33	2.44	1.89	2.91	1.93	0.48	3.72	2.55
0.25	0.09	2.33	1.67	1.40	1.54	2.37	4.18	1.73
2.28	2.23	0.77	4.01	1.48	2.88	1.47	0.13	1.91
0.94	2.22	2.72	5.11	2.22	2.46	1.54	0.56	2.22
2.03	0.99	3.27	2.35	3.29	3.55	0.98	0.78	2.15
1.26	2.07	4.02	1.09	1.84	0.37	2.30	1.32	1.78
3.82	2.06	2.33	7.00	3.10	0.66	4.28	2.14	3.17
3.08	1.54	0.88	2.60	4.20	2.52	3.82	1.22	2.48
2.28	4.09	1.19	0.76	2.39	4.94	2.23	3.06	2.62
2.10	2.16	2.67	2.67	2.40	2.28	2.27	1.91	2.31

`t` is a table with 2 rows and 2 columns:

```
$t
2 2
```

The Second Problem Stated:

Early versions of J were supplied with tutorial files that could be displayed while in J. In Version 4 of the Dictionary the tutorials are provided in an appendix [5]. In the files of one preliminary version each new-line code stood alone instead of being followed by a line-feed. Consequently, when these files were read lines of text were superposed. If the last line was the longest in the file, then that line was the only one displayed.

To create such a file for yourself, first use your word processor to prepare a text file (let us call it *junk*) with half a dozen short lines *without* wordwrap; i.e. press the **Enter** key at the end of each line. Next use a hex-editor, such as provided by PCTOOLS or NORTON, to see the sequence 0D 0A marking new-line and line-feed commands. Now change all occurrences of 0A to 20 in order to replace line-feeds by blanks. The problem then is how to use J to reverse the process by *inserting* a line-feed after every occurrence of new-line. Try reading this file from J by

```
Jx=. read 'junk'
```

Observe that each line is overlaid on the previous lines. Count the number of new-lines and line-feeds, verifying that the line-feeds have been eliminated:

```
+/ x =/ 13 10 {a.
```

The verb *h* cuts the string using the new-line codes as terminating delimiters. We can compose it from 3 verbs as a fork; a monadic verb, a dyadic verb, and the verb *right* (J). Or we can compose it from 2 verbs as a hook.

```
h=. =&nl <;.2 J      NB. Fork
h=. <;.2~ =&nl      NB. Hook
```

As in all hooks, the right-hand verb is monadic and the left-hand verb is dyadic. A hook combines one noun with the result of a verb that has been applied to a noun, either the same noun as the first (in the monadic case) or a different one (in the dyadic case). Use a hook whenever this situation occurs.

The boolean string produced by *equals* (=) points to the positions of new-line codes. This is the left argument for *cut* (< ;. 2). The 2 indicates that the delimiters mark the *ends* of the intervals and are to be included in the result [5, p.10-11].

```
each=. &.>
g=. ,&lf each
edit=. ; @ (g @ h)
input=. edit @ read
```

The definition should be *fixed* so that *input* will not be changed if the names *g* and *h* are reused.

```
input=. input f.
```

The adverb *each* opens the boxes produced by *h*, applies the verb (which is here *catenate lf to the tail*), and then closes the boxes again. After reading the file, *input* edits it by putting in the required line-feed codes. The file can now be displayed without superposed lines:

```
]x=. input 'junk'
```

To create a new file with the corrections:

```
x 11:2 <'junk.out'
```

The character string *x* is written to the new file (which must be designated by a *boxed* string). This file can now be read correctly (remember that our *read* verb boxes the name before using it):

```
read 'junk.out'
```

Leave *J* and use your hex-editor to see that line-feeds (0A) have been inserted between the new-line (0D) and space (20) codes.

We have modified a single file, but I had several files to edit. Let *infiles* and *outfiles* be the lists of boxed names of files for input and output:

```
infiles=. 'tut10.in';'tut11.in';'tut12.in';'tut13.in'
outfiles=. 'tut10.out';'tut11.out';'tut12.out';'tut13.out'
```

With the power of *J* we can create all the new files in one step. Switching arguments to avoid parentheses, my first successful solution was this:

```
output=. >@[ 11:2 ] NB. Fork
outfiles output~"0 input each infiles
```

The logic is as follows. The file names are already boxed. Use *each* to apply *input* (edit after read) and give the boxed strings ready for the output files. Let the boxed names of the output files be the right argument, and let the left argument be the boxed strings that are to go into these files. Then the *rank* conjunction ("0) can make output handle each file in turn. The syntax would be like this:

For one file:

```
'string' 11:2 (boxed file name)
```

Note the asymmetry: the right argument must be boxed, but the left argument must *not* be boxed.

For several files one might try:

```
(boxed edited strings) 1!:2"0 (boxed output file names)
```

The rank conjunction takes 1 box on the left along with one box on the right. However after the box on the left has been taken it must be opened before output begins. If we open the left argument too soon then we defeat the purpose of the rank conjunction! Consequently *open* (>) must be part of output. The other part is, of course, 1!:2

A fork is then obvious: open the left argument and don't open the right argument of 1!:2

```
output=. >@[ 1!:2 ]          NB. Fork
```

Because this fork (like many others I have written!) consists of a monadic verb (*open*), a dyadic verb (1!:2), and the verb *right*, it can be recast as a hook. Because there are two distinct arguments, the hook is dyadic:

```
x (g h) y      is the same as      x g (h y)
```

h is the monadic function, therefore we must switch arguments; it is the boxed string that is to be opened — not the boxed file name. The right argument must be the result produced by *input each infiles* — which is boxed; the rank conjunction sees to it that only one box is taken at a time, and the phrase denoted (*h y*) opens that box. Along with the rank conjunction, this provides the right argument for (1!:2), and the left argument is a single boxed file-name.

In using the hook (instead of the fork) we have no choice about which argument is to be on the left and which on the right. Because this conflicts with the positions of the arguments required for (1!:2), we **must** cross them (~). The result is therefore:

```
(boxed output file names) (g~ h) (boxed edited strings)
```

That is, we need the following hook:

```
output=. 1!:2~ >          NB. Hook
outfiles output"0 input each infiles
```

After this line is executed, all files named in *outfiles* can be read correctly.

Acknowledgements:

Kenneth Iverson and Roger Hui (Iverson Software Inc.) created the system a small facet of which I have described here. They have given unfailing assistance, coaching me by long-distance telephone and patiently answering my more naive questions. In response to my reported experience, in a few cases they modified the language. They are, of course, in no way responsible for infelicities in my use and exposition of their language. I claim only that the expressions given here work in the current version of the system (J4.1x2, 13 Jan 1992).

By asking for a solution to the first problem, Dr. Richard Fulford directed me into the realm of external communication. Graham Woyka and Anthony Camacho were always encouraging when the going was rough.

Luachmhor, Church Road
Kinfauns, Perth PH2 7LD
Scotland - U.K.
Telephone: 0738-86-726

References

- [1] J is available from Iverson Software Inc., 33 Major Street, Toronto, Ontario, Canada M5S 2K9. Phone (416) 925-6096; Fax (416) 488-7559.
UK agent I-APL Ltd (see Product Guide)
- [2] Donald B. McIntyre, *Mastering J*, APL91 Conference Proceedings, Stanford, California, August 1991. APL Quote Quad Vol. 21 Number 4 (August 1991), p.264-273.
- [3] Donald B. McIntyre«MDNM», *Language as an Intellectual Tool: From hieroglyphics to APL*, IBM Systems Journal, Vol. 30, Number 4 (1991) p.554-581.
- [4] Donald B. McIntyre, *Hooks and Forks and the Teaching of Elementary Arithmetic*, Vector, Vol. 8 Number 3 (January 1992) 101-123.
- [5] Kenneth E. Iverson, *The ISI Dictionary of J*, Version 4 with Tutorials, Iverson Software Inc., Toronto (1991) 29pp.
- [6] Kenneth E. Iverson, *Programming in J*, Iverson Software Inc., Toronto (1991) 72pp.
- [7] Kenneth E. Iverson, *Tangible Math*, Iverson Software Inc., Toronto (1991) 33pp.
- [8] Kenneth E. Iverson, *Arithmetic*, Iverson Software Inc., Toronto (1991) 119pp.
- [9] Kenneth E. Iverson, *An Introduction to J*, Iverson Software, (1992) 46pp.
- [10] Donald B. McIntyre, *Using J's Boxed Arrays*, Vector, Vol. 9 (1992) In Press

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hackers Corner: Reshaping your Mouse	Adrian Smith	112
Graph-plotting in I-APL/Mac 1.2	Ian A. Clark	113
Writing Operators for Dyalog APL	Phil Last	119

Hackers' Corner: Reshaping your Mouse

by Adrian Smith

There are times when a simple reverse-video block is not the kind of mouse pointer you need. To save you the (not inconsiderable) effort of decoding the appropriate page of Ray Duncan's book, here is a simple extension to *MSΔINIT* which does the trick:

```

V CH MSΔINIT CU;RC

[1]  A Initialize mouse ... sets global Δmouse+1 if OK ... else 0
[2]  A CU is the cursor type ... 0=mouse cursor; 1=hardware cursor
[3]  A {CH} is the character and colour to be used for the mouse block.

[4]  RC+0 □INT 51 ◊ Δmouse+0*1↑RC ◊ +Δmouse+0
[5]  →CU↑HW ◊ →(□NC 'CH')↑0

[6]  A Use left arg as mouse character, possibly with colour ...

[7]  Δ(0*1+0pCH)/'CH'+-1+□AV:CH'

[8]  A Colour defaults to bright white ...

[9]  CH+2↑CH,15

[10] A CX is set to 'F000' Hex to retain background only;
[11] A DX is the encoded character and foreground ...

[12] RC+(10 0 ,~4096,CH[1]+256×CH[2])□INT 51 ◊ →0
[13] HW:RC+ 10 1 □INT 51
V

```

This defaults to the normal mouse block, unless you provide a left argument. This may specify the mouse character (as a character scalar or □AV index), and optionally a colour. APL*PLUS II users should note that the ~4096 is really Hex(F000), but that in APL*PLUS/PC you trip over the 2-byte integer line, and have to pretend that you mean Hex(-F + FFF).

```

'□' MSΔINIT 0  A Mouse pointer is a white Domino ...
4 14 MSΔINIT 0  A ... or a yellow diamond!
MSΔSHOW        A Let's see it!

```

That's it folks. Next issue, the .EPS format exposed, or how to get your fancy PostScript graphs into MS Word, Aldus PageMaker and so on. Watch this space.

Graph-plotting in I-APL/Mac 1.2

by Ian A. Clark

Introduction

A “toy” APL with a leisurely gait and a 32K workspace limit sounds like no big deal in today’s high-powered marketplace. So why ever did I undertake to port I-APL to my current favourite machine, the Apple Macintosh? I was attracted to I-APL because:

- It conforms to the ISO standard
- It has direct definition of functions (an ISO-conformant extension)
- It is portable across different vendors’ machines, the different ports being workspace compatible at the byte-level
- I could try out some ideas for mitigating the human-factors shortcomings of the conventional APL interface
- It is non-proprietary.

To me, the last point is the important one. What it means, if you are a teacher, a lecturer or a writer of textbooks, is that you can develop working code as part of your course material and be beholden to nobody for their sufferance to distribute it to your class, together with a free copy of the interpreter bundled in.

No other language of any significant capability allows you to do this, since the days when MS-BASIC was bundled-in with all industry-standard personal computers. If you use a spreadsheet (like a lot of the business studies fraternity these days) or a single-platform development system like Hypercard or Toolbook, no matter how splendid, you are nailing your colours to the vendor’s mast. Your *magnum opus* will sink or swim with the proprietary product and its platform.

In the educational world, it’s risky flirting with a high-performance system which locks you into a single supplier. I’ve known people who were donated fancy new systems as part of some generously funded initiative, but who found themselves marginalised as a result. How they wished they had stuck to some freely-available tool with more modest capabilities.

Needs

However there is one thing that even the most unassuming APL interpreter aimed at an educational audience needs to do — draw graphs. Pictorial representation of data is something outside the scope of the ISO standard. Anyway to do it well is something highly machine-dependent. It is one thing to labour away for an hour composing a vector of numeric data. A student expects to do that. It is quite another thing to have to sit and wait five minutes whilst a fairly simple supplied graph dribbles onto the screen. Consequently, if I-APL/Mac was to be a serious competitor to one of the many Macintosh spreadsheets (which now include Lotus 123) it had to have a fast built-in graph-drawing facility. Perhaps not as slick or as generalised as Excel or Lotus 123 — after all, I-APL/Mac is only freeware. But as fast.

So, having sung the praises of a truly portable, truly open language interpreter, with all the pressure this entails to keep within the confines of the lowest common denominator of personal computer capabilities, why am I going on about a highly machine-dependent graph-drawer for the specialist's machine *par excellence*, the Macintosh?

Why? Because of the interface. It's the best compromise I can achieve between human-factors and machine-factors, intended to be easy to understand, get hacking with and maintain the results of, and it makes the lightest demands on a baby APL. If you, the amateur and beginner, like it, then it won't be difficult to program for other platforms.

At the last ASL conference at Greg-y-nog, Peter Lewis made a plea for what I understood to be a graph-plotting system in which the data was entirely separated from the format, the said format being storable as an APL object. Whether that was what Peter said or not, it seemed eminently sensible. It supported the teacher's task of developing by trial-and-error a collection of formats, each of which could be applied to a given set of data, as well as the student's task of studying a collection of functions or time-series by plotting them in a single given format, or one of a restricted choice of formats. Properly done, it would allow formats to be easily combined, a kind of mathematical mix-'n'-match.

Hitherto (in I-APL/Mac version 1.1) I had followed the "lego set" approach: to offer a more-or-less general function to draw the title, another to put a box round it, yet more to draw the axes, put tick-marks against them, draw a shaded bar, and so on. Each of these services was offered as a cover-function which invoked the machine-code interface ($\square MC$) within itself, to draw inside the graphics viewport. The user of such a package is then expected to write a multi-line APL

function, just one line of which actually accepts the given data vector(s). In I-APL/Mac this approach is inevitably too slow.

The Design

Here then was the challenge: to design an interface to a presentation-graphics facility that is used like this:

```
MYFORMAT DISPLAY MYDATA
```

which could be re-packaged according to the task to be done as either:

```
PLOT MYDATA
```

(*MYFORMAT* being delivered inside a global variable called *CMDSTRING* accessed by *PLOT*) or:

```
PLOT MYDATA USING MYFORMAT
```

(*USING* simply bouncing back its left argument and putting its right argument into the said global variable *CMDSTRING*). It is also useful to have a function called *TEST*:

```
TEST MYFORMAT
```

which simply looks in the global variable *TESTDATA* for the contents of *MYDATA*.

Because of the emulated nature of I-APL, *DISPLAY* would need to pass its data directly to the graphics facility, with little or no intervening APL processing. In fact the *DISPLAY* function is nothing but a cover function for a *⌈MC* call which could just as well be invoked by:

```
39 ⌈MC 2 9p'MYFORMAT MYDATA
```

What form should *MYFORMAT* take? In a second-generation APL the treelike nature of the possible parameters to a graph suggests a nested array of strings and numbers. In the absence of nested arrays one is thrown back on large sparse tables: characters, or numbers. Should I abandon the idea of a single format variable and have one of each? I decided to cling to the single variable, which therefore had to be a character array. It is straightforward and economical to represent numerals within a character array (particularly as they get read by C and not APL), but cumbersome to represent characters within a numeric array.

It was important for *MYFORMAT* not to take up too much room in the workspace. It should be easy for beginners to compose and customise. That ruled out having a large unfriendly fixed-format table needing table-generating functions to maintain it. Its syntax should also be indefinitely extensible, allowing later versions of the graphics facility to offer new features whilst still honouring existing formats.

Eventually I settled on making *MYFORMAT* a single character-vector, a command string. However a 2-D character array would be accepted too, just as if it had been previously unravelled. Using the built-in full-screen editor, character arrays are easier to maintain than long character vectors anyway. The syntax of this command string is a sequence of sub-strings each terminated by a semicolon. Multiple blanks are treated as single blanks. Each sub-string, or "command", consists of a 2-character mnemonic opcode followed by the required arguments, if any. In fact, just the sort of syntax that `scanf()` is happy with. (Non-C programmers note: `scanf()` is a C standard library function which is a bit like the INPUT command in BASIC.)

The command string is interpreted from left to right. Some commands like 'WI,' (wipe the window clean), 'MT' (move-to the following coordinates) and 'TF' (change the text font) perform their action there and then, but those describing the graph to be plotted alter the default values of internal parameters, so that their order of specification does not matter. However a subsequent command with the same opcode will override the earlier setting. A blank or null command string is valid, yielding a sensible display using the built-in defaults.

This design yields a bonus usually associated with object-oriented programming: the ability to package useful formats, yet override unwanted features and supply extra parameters. Thus if *MYFORMAT* produces a line-graph, one could substitute *MYFORMAT*, *BARCHART* say, and get a bar chart, all other parameters remaining the same. Furthermore *MYFORMAT* could easily be written to interpret a trailing catenated string as a title. It would be used either alone or as follows:

```
PLOT SALES92 USING MYFORMAT, 'SALES RESULTS 1992'
```

There is a wide variety of ways of storing command strings. One benefit of direct-definition in I-APL is that it yields a handy way for students to alter the values of parameters, whether character or numeric, by defining them, not as global variables, but as functions. Typing the name of the parameter followed by a colon displays its current definition. Not only can the existing value be overtyped, but the student does not have to hunt for the back-arrow key.

Thus a simple command-string might be directly-defined by:

```
MYFORMAT: 'A4; TI; ~M; FN100;'
```

as well as by:

```
MYFORMAT+ 'A4; TI; ~M; FN100;'
```

MYFORMAT can also be stored as an array of commented commands, like an assembler program. You can either rely on the fact that everything between the last parameter required by the command and the semicolon is ignored, or you can use the explicit "comment" labels:

```
'*' '--' 'R'
```

In this way, the above *MYFORMAT* could be stored as:

```
A4;      --4-quadrant axes at default origin (centre of screen);
TI;      --Tick Marks on both X and Y axes;
~M;      --No Y markings required;
FN100;   --finishing X-axis value is 100;
```

or as:

```
A4      4-quadrant axes at default origin (centre of screen);
TI      Tick Marks on both X and Y axes;
~M      No Y markings required;
FN100   finishing X-axis value is 100;
```

The workspace *PLOTDATA* is part of the graphics add-on product. It contains a sample format variable, *TRIGFORMAT*, to plot general trigonometric functions. This contains the following commands:

```
TRIGFORMAT
WI      wipe the window clean to start afresh;
O4 150 110  4-quadrant, origin at h=150, v=110;
TI      do tick-marks;
+X4      plotting interval 4 points;
SX-100    1st point starts -100 points from origin;
ST-126    1st X-tick starts -126 points from origin;
SN-20     1st X-tick mark labelled -20;
FN20      last X-tick mark labelled 20;
N.1      --with 1 place of decimals (-2.0 to 2.0);
+N10     label values step by 10;
+T63     X-tick interval 63 points (approx 10|| 2);
+U100    Y-tick interval 100 points;
SU-100    1st Y-tick starts -100 points from origin;
SM-10     1st Y-tick mark labelled -10;
FM10     last Y-tick mark labelled 10;
M.1      --with 1 place of decimals;
+M10     Y-label values step by 10;
MT235 135  move to h=235, v=135;
DS[] radians;  --label the X-axis;
```

PLOTDATA also contains a function, *DEMO*, defined as follows:

```
DEMO: PLOT SINEDATA USING TRIGFORMAT
```

Executing *DEMO* gives the following result:

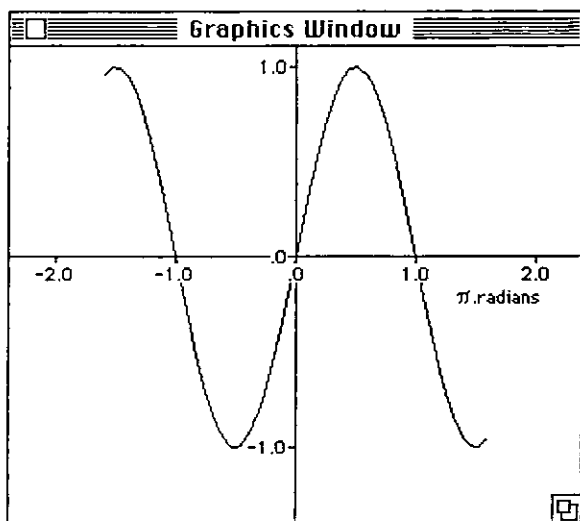


Figure 1: Result of running sample *DEMO* in workspace *PLOTDATA*

Conclusion

Well, there it is. Other vendors please copy — I'm offering it as a new standard. That means I need a name for it, I suppose. Sensible suggestions to VECTOR, please. Meanwhile, if you can lay your hands on a Macintosh, I can supply the latest version, 1.2, of I-APL/Mac (which is freeware) plus the graph-drawing kit described (which is not, but comes with a free educational site-licence). You can order all I-APL products from the official source, I-APL Ltd (address elsewhere in this issue). Specifically Macintosh products, however, you can order directly from me and pay by VISA or MasterCard. Send SAE for a price-list of inexpensive goodies, project kits and teaching resources to use with I-APL/Mac, including this graph-drawing package.

Ian A. Clark
IAC/Human Interfaces,
9 Hill End, Frosterley-in-Weardale,
Bishop Auckland,
Co. Durham DL13 2SX.

Writing Operators for Dyalog APL

by Phil Last

In this article I shall discuss operator syntax, introduce a new idiom, and describe some defined operators which use it.

In expressions and identities (distinguished by having ' $\leftrightarrow$ ' between two equivalent expressions) I shall try to use symbols consistently as:

(a) (b) (l) (r) are arrays.

(f) (g) (h) are functions.

(A) is an ambivalent object which might be an array or a function according to parameter substitution for operators.

(O) and (P) are operators.

A grasp of operator syntax and of the construction and derivation of derived functions is a prerequisite to the understanding of what follows.

I have used the old terminology of distinguishing OPERANDS from ARGUMENTS. In fact the word 'argument' is generic and can be used to refer to parameters to both functions and operators while the word 'operand' is (within APL) reserved for operator arguments. I shall use the words from now on as follows:

An ARGUMENT is exclusively an array parameter to a FUNCTION (primitive, defined or derived).

An OPERAND is a parameter (function or array) to an OPERATOR (primitive or defined).

Functions are ambivalent - left arguments may be elided. Operators are either monadic or dyadic - never ambivalent. Monadic Operators take their single operand on the LEFT. Operands are ambivalent in the sense that they may be either functions or arrays.

In the expression:

$a \ f \ O \ g \ b$

wherein (O) is a DYADIC operator

(f) and (g) are the operands to (O).

(a) and (b) are the arguments to the derived function (f O g).

$a \ f \ O \ g \ b \leftrightarrow a(f \ O \ g)b$ a for dyadic O

In the expression:

$$a \ f \ P \ g \ b$$

wherein (P) is a MONADIC operator

(f) is the operand to (P).

(a) is the left argument to the derived function ($f \ P$).

(b) is the right argument to function (g).

($g \ b$) is the right argument to the derived function ($f \ P$).

$$a \ f \ P \ g \ b \leftrightarrow a(f \ P)g \ b \quad \text{for monadic } P$$

In the expression:

$$R \leftarrow a \ f \ O \ g \ P \ h \ b$$

wherein (O) and (P) are DYADIC operators

(h) is the right operand to (P)

($f \ O \ g$) is the left operand to (P)

(f) (g) are the operands to (O)

(a) (b) are the arguments to function ($f \ O \ g \ P \ h$)

$$a \ f \ O \ g \ P \ h \ b \leftrightarrow a(f \ O \ g)P \ h \ b \quad \text{for dyadic } O \text{ and } P$$

It is this fact which gives rise to the oft mentioned misapprehension that in function expressions (containing operators) execution proceeds from left to right. In fact the left operand ($f \ O \ g$) is derived prior to the execution of (P) but is then only executed under the control of (P) if at all. In other words as always execution proceeds from right to left unless parentheses are used to alter that order. In the expression:

$$a(f \ O \ r)b$$

wherein (O) is a DYADIC operator

(f) is the left function operand to (O)

(r) is the right array operand to (O)

(a) and (b) are the arguments to derived function ($f \ O \ r$)

The parentheses are non-redundant.

The separation of array right operands from right arguments to the derived function can often be achieved without parentheses by use of a right identity:

- $\vdash \quad \mathbf{A}$ where implemented
- $+$ $\mathbf{A}$ where the domain is real
- $\cdot$ $\mathbf{A}$ where the argument is a vector etc.

$$a \ f \ 0 \ r \vdash b \leftrightarrow a(f \ 0 \ r)b \ \mathbf{A} \text{ for dyadic } 0$$

What follows makes extensive use of:

- $\vdash$ dex - the ambivalent right identity function

$$\begin{aligned} a \vdash b &\leftrightarrow b \\ \vdash b &\leftrightarrow b \\ f \vdash b &\leftrightarrow f \ b \end{aligned}$$

- $\circ$ the compose operator

$$\begin{aligned} a(f \circ g)b &\leftrightarrow a \ f \ g \ b \\ (f \circ g)b &\leftrightarrow f \ g \ b \\ (l \circ g)b &\leftrightarrow l \ g \ b \\ (f \circ r)b &\leftrightarrow b \ f \ r \end{aligned}$$

Either or both of these might be implemented using a different character on your system, or one or both might not have been implemented. They are both very simple to define ... combining dex and compose:

$$\begin{array}{ll} (\vdash \circ f)b \leftrightarrow f \ b & (i) \\ a(\vdash \circ f)b \leftrightarrow f \ b & (ii) \\ (\vdash \circ r)b \leftrightarrow r & (iii) \\ \vdash \circ (\vdash \circ f)b \leftrightarrow f \ b & (iv) \\ \vdash \circ (\vdash \circ r)b \leftrightarrow r & (v) \\ a\vdash \circ (\vdash \circ f)b \leftrightarrow f \ b & (vi) \\ a\vdash \circ (\vdash \circ r)b \leftrightarrow r & (vii) \end{array}$$

$(\vdash \circ f)$ and $(\vdash \circ (\vdash \circ f))$ are both ambivalent functions which ignore their left arguments and call function (f) MONADICALLY. $(\vdash \circ r)$ and $(\vdash \circ (\vdash \circ r))$ are respectively MONADIC and AMBIVALENT CONSTANT functions which return (r) whatever their argument(s).

An aside: by definition of reduction:

$$\begin{aligned} \supset f / \supset a &\leftrightarrow \supset f /, \supset a \leftrightarrow a & (viii) \\ \supset f / (\supset a), \supset b &\leftrightarrow a \ f \ b & (ix) \end{aligned}$$

We can now derive the idiom:

using (viii) and (ix)

$$\triangleright f / (0p \leq a), c b \leftrightarrow b \quad \text{A conditional execution} \quad (1)$$

$$\triangleright f / (1p \leq a), c b \leftrightarrow a f b \quad \text{A of dyadic function } f \quad (2)$$

substitute ambivalent function from (ii)

$$\triangleright \circ f / (0p \leq a), c b \leftrightarrow b \quad \text{A conditional execution} \quad (3)$$

$$\triangleright \circ f / (1p \leq a), c b \leftrightarrow f b \quad \text{A of monadic function } f \quad (4)$$

remove now redundant reference to (a)

$$\triangleright \circ f / (10), c b \leftrightarrow b \quad \text{A conditional execution} \quad (5)$$

$$\triangleright \circ f / (11), c b \leftrightarrow f b \quad \text{A of monadic function } f \quad (6)$$

substitute equivalent function from (vi)

$$\triangleright \circ (\circ f) / (10), c b \leftrightarrow b \quad \text{A conditional execution} \quad (7)$$

$$\triangleright \circ (\circ f) / (11), c b \leftrightarrow f b \quad \text{A of monadic function } f \quad (8)$$

substitute constant function from (vii)

$$\triangleright \circ (\circ r) / (10), c b \leftrightarrow b \quad \text{A condition selection} \quad (9)$$

$$\triangleright \circ (\circ r) / (11), c b \leftrightarrow r \quad \text{A of array } r \quad (10)$$

Most important is the similarity of 7, 8, 9 and 10 which differ only in a single logical value and the function/array ambivalence of f and r

These permit the coding of the conditional ambivalent expression:

$$\begin{aligned} \triangleright \circ (\circ A) / (1L), c b &\leftrightarrow b && \text{A when } L \text{ is untrue} && (11) \\ &\leftrightarrow A b && \text{A when } L \text{ is true and } A \text{ is a function} \\ &\leftrightarrow A && \text{A when } L \text{ is true and } A \text{ is an array} \end{aligned}$$

I believe this idiom to be new. It is very useful.

I have used dex and compose because compose is implemented on my system; an alternative pair will do the same job:

→ lev - the ambivalent left identity function

$$a \rightarrow b \leftrightarrow a$$

$$\rightarrow b \leftrightarrow b$$

$$f \rightarrow b \leftrightarrow f b$$

and lever - a defined operator

$$\begin{aligned}
 (f \text{ lever})b &\leftrightarrow f \ b && \text{cf. (iv) (v) (vi) and (vii)} \\
 (1 \text{ lever})b &\leftrightarrow 1 \\
 a(f \text{ lever})b &\leftrightarrow f \ b \\
 a(1 \text{ lever})b &\leftrightarrow 1 \\
 A \text{ lever} &\leftrightarrow \vdash \circ (\vdash \circ A) && (x)
 \end{aligned}$$

$(f \text{ lever})$ is an ambivalent function which calls the function operand (f) MONADICALLY. (1 lever) is an ambivalent constant function which always returns array (1)

Substituting (x) into (11) gives: (12)

$$\begin{aligned}
 \supset A \text{ lever} / (\vdash L), \subset b &\leftrightarrow b && \text{A when L is untrue} \\
 &\leftrightarrow A \ b && \text{A when L is true and A is a function} \\
 &\leftrightarrow A && \text{A when L is true and A is an array}
 \end{aligned}$$

If your system has $\circ$ (or equivalent) but lacks $\vdash$ and $\dashv$ then:

$$\supset \text{dex} \circ (\text{dex} \circ A) / (\vdash L), \subset b \quad \text{A from (11)}$$

... is the better alternative as dex does not have a great deal to do:

$$\begin{aligned}
 \forall \ R \vdash L \text{ dex } R \\
 \forall
 \end{aligned}$$

If your system has $\vdash$ and $\dashv$ but lacks compose then:

$$\supset A \text{ lever} / (\vdash L), \subset b \quad \text{A from (12)}$$

is preferable as the defined operator is unconditional:

$$\begin{aligned}
 \forall \ R \vdash A(F \text{ lever})B \\
 [1] \quad R \vdash F \dashv B \\
 \forall
 \end{aligned}$$

My first use of this idiom is in defined operator $IF_$:

$$\begin{aligned}
 \forall \ R \vdash (F \text{ IF_ } G)R \\
 [1] \quad \text{A IF } G \text{ or } G \ R \text{ then } R \vdash F \text{ or } F \ R \\
 [2] \quad \text{A } F: \text{ any array or monadic function} \\
 [3] \quad \text{A } G: \text{ any array or (conditional) monadic function} \\
 [4] \quad \text{A } R: \text{ any array in domains of } (F) \text{ and } (G) \text{ if functions} \\
 [5] \quad R \vdash \supset \vdash \circ (\vdash \circ F) / (\vdash 1 \in G \dashv R), \subset R \\
 \forall
 \end{aligned}$$

```

       $\nabla R \leftarrow (F \text{ IF\_ } G)R \dots$ 
[5] A alternative defn without primitive lev and dex:
[6]  $R \rightarrow \text{dex} \circ (\text{dex} \circ F) / (\text{lex}(\text{dex} \circ G)R), \text{c}R$ 
       $\nabla$ 

       $\nabla R \leftarrow (F \text{ IF\_ } G)R \dots$ 
[5] A alternative defn without primitive compose:
[6]  $R \rightarrow F \text{ lever} / (\text{lex}G \rightarrow R), \text{c}R$ 
       $\nabla$ 

      (-IF_ 1) 3 A operands: function and array
-3
      (-IF_ 0)3
      3
       $\div \text{IF\_} \times 3$  A operands: function and function
      0.3333333333
       $\div \text{IF\_} \times 0$ 
      0
      'WRONG' IF_(0 $\circ$  $\epsilon$ )1 2 3 A operands: array and function
      1 2 3
      'WRONG' IF_(0 $\circ$  $\epsilon$ )1 2 3 0
      WRONG

```

A simple switching of operands makes the following operator more convenient to use in certain syntactical circumstances.

```

       $\nabla R \leftarrow (F \text{ THEN\_ } G)R$ 
[1] A if F or F R THEN R+G or G R
[2] A F: any array or (conditional) monadic function
[3] A G: any array or monadic function
[4] A R: any array in domains of (F) and (G) if functions
[5]  $R \rightarrow \text{lex} \circ (\text{lex} \circ G) / (\text{lex}F \rightarrow R), \text{c}R$ 
       $\nabla$ 

      1 THEN_  $\phi$  14
      3 2 1 0
      0 THEN_  $\phi$  14
      0 1 2 3

```

When an alternative to the execution or non-execution of a function is required:

```

       $\nabla R \leftarrow L(F \text{ ELSE\_ } G)R$ 
[1] A if L then R+F or F R ELSE R+G or G R
[2] A L: any array (which may or may not contain a 1)
[3] A F: any array or monadic function
[4] A G: any array or monadic function
[5] A R: any array in domains of (F) and (G) if functions
[6]  $R \rightarrow \text{lex} \circ (\text{lex} \circ F) / (\text{lex}L), \text{lex} \circ (\text{lex} \circ G) / (\text{lex} \sim L), \text{c}R$ 
       $\nabla$ 

```

```

      1 (+\ ) ELSE_ (×/) 1 2 3 4
1 3 6 10
      0 (+\ ) ELSE_ (×/) 1 2 3 4
24

```

Here is a complete résumé of all four calling sequences for each of the three operators wherein:

cond fn0 and fn1 are monadic functions
 true ar0 and ar1 are arrays
 a condition is fulfilled if (1∈true) or (1∈cond R)

```

R+fn0 IF_ cond R ↔ IF cond R THEN R+fn0 R ELSE no-op
R+ar0 IF_ cond R ↔ IF cond R THEN R+ar0 ELSE no-op
R+(fn0 IF_ true)R ↔ IF true THEN R+fn0 R ELSE no-op
R+(ar0 IF_ true)R ↔ IF true THEN R+ar0 ELSE no-op

```

```

R+cond THEN_ fn1 R ↔ IF cond R THEN R+fn1 R ELSE no-op
R+(cond THEN_ ar1)R ↔ IF cond R THEN R+ar1 ELSE no-op
R+true THEN_ fn1 R ↔ IF true THEN R+fn1 R ELSE no-op
R+(true THEN_ ar1)R ↔ IF true THEN R+ar1 ELSE no-op

```

```

R+true fn0 ELSE_ fn1 R ↔ IF true THEN R+fn0 R ELSE R+fn1
RR+true(fn0 ELSE_ ar1)R ↔ IF true THEN R+fn0 R ELSE
R+ar1R+true(ar0 ELSE_ fn1)R ↔ IF true THEN R+ar0 ELSE R+fn1
RR+true(ar0 ELSE_ ar1)R ↔ IF true THEN R+ar0 ELSE R+ar1

```

The three operators are related in the following way:

$$f \text{ THEN_ } g \leftrightarrow g \text{ IF_ } f$$

$$\text{true } f \text{ ELSE_ } g \text{ } b \leftrightarrow (f \text{ IF_ true})(g \text{ IF_ } (\sim \text{true}))b$$

which identities can readily be observed in the code.

The idiom as stated above:

$$\triangleright \vdash \circ (\neg \circ A) / (1L), \circ b$$

can be extended in use if we remember that 1 has a larger domain than merely logical values.

$$\begin{aligned} \triangleright \vdash \circ F / (10), \circ b &\leftrightarrow b \\ \triangleright \vdash \circ F / (11), \circ b &\leftrightarrow F \text{ } b \\ \triangleright \vdash \circ F / (12), \circ b &\leftrightarrow F \text{ } F \text{ } b \\ &\dots \end{aligned}$$

Hence:

```

    V R+(F POWER_ G)R
[1] A DO R←F R FOR N=1 TO G or G R
[2] A F: a monadic fn whose codomain is a subset of its domain
[3] A G: any single non-negative integer or function on R
[4] A returning any single non-negative integer.
[5] A R: any array in domain of F (and of G if a function)
[6] R←→(F/(1G-R),cR
    V

```

Here are two more:

```

    V R+(F WHILE_ G)R
[1] A DO R←F R WHILE G R
[2] A F: any monadic function
[3] A G: any monadic (conditional) function
[4] A R: any array in domains of F and G
[5] R←→(F WHILE_ G)•F/(11G R),cR
    V

```

This is a vivid illustration that we are dealing with conditional execution without $\pm$ or $\rightarrow$. Whoever saw a recursive function without either?

```

    V R←L(F WHILEC_ G)R
[1] A DO R←F R WHILE G R FOR N=1 TO L
[2] A F: any monadic function
[3] A G: any monadic (conditional) function
[4] A R: any array in domains of F and G
[5] A L: non-negative integer defining maximum iterations
[6] R←→(F IF_ G)/(1L),cR
    V

(2•x) WHILE_ SINGLE 1
16
(*2) WHILE_ SINGLE 3
81
(2•x) WHILE_ SINGLE 10
10
(*2) WHILE_ SINGLE 1
INTERRUPT
WHILE_[2] R←→(F WHILE_ G)•F/(11G R),cR
    ^
    R
1
p□LC
1071
A infinitely recursive!
→
100 (*2) WHILEC_ SINGLE 1
1
(1.1•x) WHILE_ SINGLE 1
10. 83470594

```

```

    V R←SINGLE B
[1] R←B<10
    V

```

```

1.1*10.8347059425
25
(1.1*x) WHILEC_ SINGLE 1
10.83470594
24 (1.1*x) WHILEC_ SINGLE 1
9.849732676

```

Both of these operators have the domain of both operands as strictly FUNCTION. As indicated in the examples (*WHILEC_*) has the problems:

- (i) possible infinite recursion. While (*WHILEC_*) overcomes this problem by setting a limit on the number of iterations it introduces two others the first of which is also indicated in the examples:
- (ii) The condition might remain true throughout the permitted iterations although it would have become untrue some time later.
- (iii) The condition is tested for the full number of permitted iterations although it might have become untrue at or near the first iteration.

Problems (i) and (ii) are in the nature of the semantics of DO WHILE and therefore require no further consideration. There does not seem to be any way to overcome problem (iii) without undue additional processing overhead.

- (Unless of course, you code a loop! I hope (but I'm not too hopeful) that I am not going to overly great lengths to avoid looping. I'm afraid that I do have an aversion to '+', indeed, at least as much as to 'x'. Mainly I would like to see interpreter writers giving priority to parts of the language which facilitate the avoidance of these. I have been known (even if only to myself) to allow a less efficient construct when it avoids a loop, skip or execute.)

While we are at it here are the DO UNTIL counterparts of the above:

```

V R←(F UNTIL_ G)R;L
[1] A DO R←F R UNTIL G R
[2] A F: any monadic function
[3] A G: any monadic (conditional) function
[4] A R: any array in domains of F and G
[5] R←→=(F UNTIL_ G)/(1~G L),cL←F R
V
V R←L(F UNTILC_ G)R
[1] A DO R←F R UNTIL G R FOR N=1 TO L
[2] A F: any monadic function
[3] A G: any monadic (conditional) function
[4] A R: any array in domains of F and G
[5] A L: non-negative integer defining maximum iterations
[6] R←→=(F IF_(1~G))/(1L-1),cF R
V

```

As expected the test is done AFTER the operation as opposed to before as in the DO WHILE operators. All problems and advantages are exactly as above.

And a special case of UNTIL:

```

      V R←(F LIMIT_)R;L
[1]  A DO R←F R UNTIL R=F R
[2]  A F: any monadic function
[3]  A R: any array in domain of F
[4]  R←→(F LIMIT_)/(L≠R), <L←F R
      V

```

The function is applied. If its result is different from its argument then apply it again. Infinite recursion is yet again a possibility. I do have a (LIMITC_) alternative but it has to do some nasty tricks to avoid a loop and I'm not showing it to anyone.

This is getting a bit complicated now. The next operator actually runs into two lines of code:

```

      V R←L(F SEL_ G)R;I;J;K
[1]  A R←L M N... (F SEL_ G SEL_ H... )R
[2]  A SELECT
[3]  A IF L THEN IF FN F THEN R←F R ELSE R←F
[4]  A IF M THEN IF FN G THEN R←G R ELSE R←G
[5]  A IF N THEN IF FN H THEN R←H R ELSE R←H
[6]  A ...
[7]  A END
[8]  A L: logical vector
[9]  A F: any array or monadic fn or fn derived by SEL_
[10] A G: any array or monadic function
[11] A R: any array in domain of F or G if selected
[12] J←(V/J)>0(K+10J), I←1 0=J<L=1
[13] R←→(←F)/I, F/(J0<1+L), ←(←G)/K, <R V

```

The following description applies to a derived function which contains multiple calls to SEL_ as depicted in the comment.

LMN is a boolean vector equal in length to the number of operands in the SELECT statement (i.e. 1 more than the number of calls to the operator (SEL_)). It is treated as (<L M N ...) i.e. the first only if any (1) in the vector selects the operand to be used.

FGH the operands, are any arrays or monadic functions. Parentheses are only required around individual operands if they are derived within the statement and not the first, e.g.
 ((14)∈I) F00/ SEL_(F01\)SEL_ F02
 SEL_(F03") R

but are probably preferable in any case for clarity. The entire function expression excluding only the arguments must be parenthesised if either the first or last operand is an array.

R is any array in the domain of the function, if any, eventually selected to be run.

R becomes the result of (fn R) where fn is the selected function or it is the selected array or it is the unchanged argument R when no operand is selected.

```

      0 0 0 0 +/SEL_(+\)SEL_(*/)SEL_(*\) 1 2 3 4
1 2 3 4
      1 0 0 0 +/SEL_(+\)SEL_(*/)SEL_(*\) 1 2 3 4
10
      0 1 0 0 +/SEL_(+\)SEL_(*/)SEL_(*\) 1 2 3 4
1 3 6 10
      0 0 1 0 +/SEL_(+\)SEL_(*/)SEL_(*\) 1 2 3 4
24
      0 0 0 1 +/SEL_(+\)SEL_(*/)SEL_(*\) 1 2 3 4
1 2 6 24

```

The conditional execution of functions relies this time on identities (1) and (2) as well as (11).

$\triangleright f / (L \rho \leq a), \leq b$	$\leftrightarrow b$	<i>A when L is untrue</i>
	$\leftrightarrow a \text{ if } b$	<i>A when L is true</i>
$\triangleright \circ (\circ \circ A) / (\downarrow L), \leq b$	$\leftrightarrow b$	<i>A when L is untrue</i>
	$\leftrightarrow A \text{ } b$	<i>A when L is true and A is a function</i>
	$\leftrightarrow A$	<i>A when L is true and A is an array</i>

This is because we have also a conditional dyadic function call.

When the left argument is all zeros then none of the conditional expressions results in an operand being selected and the argument returned. When only the last item in the left argument is (1) the right operand is selected and its result returned. When other than the last item is (1) then one of two things occurs. If there are only two items then the left operand is selected and its result returned. If there are three or more items then the left operand is run dyadically with the minus one drop ($\bar{1}\downarrow$) of the left argument. This is because more than two items indicate that another call to (SEL_) is a component of the left derived function and a blind recursive call is made.

The next is a very similar operator which will run any number of its selected operands rather than only the first. Any selected function acts on the result of previously selected operands (or the right argument if it is the first selected)

while selected array operands replace previous results. In this case execution is forced to APPEAR to be left to right.

```

V R←L(F SEQ_ G)R;I;J;K
[1] A R←L M N... (F SEQ_ G SEQ_ H... )R
[2] A SEQUENTIAL
[3] A IF L THEN IF FN F THEN R←F R ELSE R←F
[4] A IF M THEN IF FN G THEN R←G R ELSE R←G
[5] A IF N THEN IF FN H THEN R←H R ELSE R←H
[6] A ...
[7] A END
[8] A L: logical vector
[9] A F: any array or monadic fn or fn derived by SEQ_
[10] A G: any array or monadic function
[11] A R: any array in domains of F and G if selected
[12]  $K \leftarrow 1 \div J + L = 1 \div J + (\sqrt{1 \div J}) \div \rho I + 1 \quad 0 \leq \backslash J$ 
[13]  $R \leftarrow (R \div (R \div G) / K, F / (J \div L), (R \div (R \div F) / I, R \div (R \div G) / K, C \div \phi R$ 
V
0 1 0 1 +SEQ_-SEQ_×SEQ_÷ 2 5
-0.5 -0.2

```

And last of this set:

```

V R←A(F PAR_ G)B;I;J;K;L;M
[1] A X Y Z... +I J K... (F PAR_ G PAR_ H... )X Y Z...
[2] A PARALLEL
[3] A IF I THEN IF FN F THEN X←F X ELSE X←F
[4] A IF J THEN IF FN G THEN X←G Y ELSE Y←G
[5] A IF K THEN IF FN H THEN X←H Z ELSE Z←H
[6] A ...
[7] A END
[8] A A: logical vector
[9] A F: any array or monadic fn or fn derived by PAR_
[10] A G: any array or monadic function
[11] A B: vector to items of which F and/or G may be applied
[12]  $K \leftarrow I + 1 \div (0 \div L), (\div L) \wedge M \quad J + 1 \quad 0 = 2 = \rho \div L \quad R \leftarrow (A = 1), [\square I O - 0.1] B$ 
[13]  $R \leftarrow (M / (R \div (R \div F) / I, C \div R), (J \div F / M + 1 \div L R), (R \div (R \div G) / K, C \div \phi R$ 
V
0 1 1 0 [PAR_[PAR_-PAR_÷ 1.1 2.2 3.3 4.4
1.1 3 -3.3 4.4

```

This one applies operands selected by its scalar conforming logical vector left argument to respective items in its conforming right argument. N.B. The term parallel is NOT, in this implementation, concerned with WHEN but WHERE to apply particular operands. In fact selected operands are applied temporally right to left but as they are applied to different items this is entirely irrelevant.

The array/function ambivalence permitted in operands and tolerated by idioms such as:

```
A ← b ↔ A      when (A) is an array
      ↔ A b      when (A) is a function
```

is a very powerful feature and allows us to write entirely ambivalent statements.

When left arguments to functions are elided, assigning the ambivalent right identity to the name by:

```
[2]  ±(0=□NC'A')/'A←±'
```

or

```
[2]  →1+(0≠□NC'A')ρ□LC ◊ A←±
```

or fixing an equivalent function by:

```
[2]  0 0ρ□FX 1 5ρ'R←A R'
```

instead of the conditional assignment of an array value permits us to continue with ambivalent statements throughout the function.

Calls to subfunctions with the elided left argument as their left argument are genuinely monadic (we don't have to guess what the SUBFN's defaults are),

```
[3]  R←A SUBFN 'this might well be a monadic call'
```

while default values can be assigned transparently when required.

```
[4]  PARM←A←default n A if an array, default otherwise
```

It seems probable that there are NO cases when this is NOT applicable.

```
~•(✓/)*~ ↔ ^/ n De Morgan
```

This surely is a case for an alternative form of function ambivalence wherein the left argument has an array value by parameter substitution or defaults to the ambivalent right identity, all done by the interpreter.

When I first wrote this that was as far as I was concerned an original thought. I have since read an article of Maurice Jordan (Vector 7.4) in which he suggested almost exactly the same thing.

This should banish at least the most common reason for conditional ± or →1+conditionρ□LC

The last operator I want to describe derives functions which are identical to that contentious and hypothetical animal, the function vector. Honest Guv! Come on, you must have known it was coming when you saw the previous one!

$$A \ B \ C \dots (F _ G _ H \dots) X \ Y \ Z \dots \leftrightarrow (A \ F \ X)(B \ G \ Y)(C \ H \ Z) \dots$$

$$(F _ G _ H \dots) X \ Y \ Z \dots \leftrightarrow (F \ X)(G \ Y)(H \ Z) \dots$$

$\nabla \ R \leftarrow (F _ G) B; L; N$

- [1] ∇ *Function vector operator*
- [2] ∇ *A: optional (scalar conformable with B)*
- [3] ∇ *F: function (possibly derived by this operator)*
- [4] ∇ *G: function to be applied to last item(s) of (A and) B*
- [5] ∇ *B: vector (may be a scalar if A is a vector)*
- [6] ∇ $\{ (0 = \square NC'A') / 'A \leftarrow \diamond F \leftarrow \diamond F \diamond G \leftarrow \diamond G' \}$
- [7] ∇ $L \ N+1 \ 0=2=p \Rightarrow R \leftarrow R \leftarrow (A \leftarrow B), [\square I 0-0.1] B$
- [8] ∇ $R \leftarrow (L/F / \Rightarrow "N \leftarrow R), (N / \Rightarrow F / \neg 1 \leftarrow "L \leftarrow R), G / \Rightarrow \diamond \phi " R$

∇

I have broken several of my own rules here so I shall list my excuses first:

The name I have given this operator is ' $_$ '

Single character names should be reserved for strictly local purposes but I have tended to view this operator since its creation as a semi-primitive, and its having an almost invisible name allows its derived functions more easily to be viewed as idioms.

I have used conditional execute (and with diamonds do you believe).

The functions derived by this operator have argument ambivalence. A conforming left argument is required for laminate on line [7], which subsequently becomes the left argument to functions run under reduction on line [8] whether the operands are dyadic or monadic. In the monadic case the functions must be run monadically under reduction. This requires:

$(\leftarrow \diamond F) /$ instead of $F /$ and $(\leftarrow \diamond G) /$ instead of $G /$

I could have put in a skip around one line with diamonds or three lines without. I could have put a skip at the start of a single line with diamonds. In the end I chose the expression you see as being the one of several more or less equivalent evils which looked nicest in view of the relative length of the other lines of code.

I have re-assigned parameter names within a defined operation (A , F and G).

Remarks above relate to (A). I suppose I could have assigned:

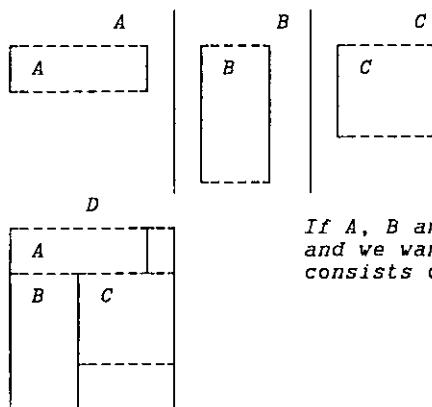
$$U \leftarrow F \diamond V \leftarrow G$$

prior to the conditional

$$U \leftarrow \circ U \diamond V \leftarrow \circ V$$

and changed line 3 to call U and V instead. Perversity won in this case (aesthetics played a hand).

And now an example:



If A, B and C are variable sized matrices and we want the overall size of D which consists of B next to C with A above:

$$\rho D \leftrightarrow ((1 \uparrow \rho A) + (1 \uparrow \rho B) \uparrow (1 \uparrow \rho C)), (\sim 1 \uparrow \rho A) \uparrow (\sim 1 \uparrow \rho B) + \sim 1 \uparrow \rho C$$

Look at the functions in the above:

$$+ \quad \uparrow \quad \uparrow \quad +$$

How about:

$$\rho D \leftrightarrow (\rho A) (+ \uparrow) (\rho B) (\uparrow +) \rho C$$

which is precisely what I coded several years ago when I required the answer and was not yet fully conversant with all the restrictions in APL and had never heard of function arrays, merely assuming that consistent and meaningful statements would give me the answer I required.

Well, now I can code:

$$\rho D \leftrightarrow (\rho A) +_ \uparrow (\rho B) \uparrow + \rho C$$

and it DOES give me the answer.

In fact a defined operator to do that task can be much simpler than that coded above as there are only two operands.

What we can do with this one is slightly more generalised.

```

      A dyadic vector/vector
      1 2 3 4 5 6 +_ - * _ ÷ _ [ _ ] 7 8 9 10 11 12
8 -6 27 0.4 11 6

```

```

      A dyadic scalar/vector
      1 +_ - * _ ÷ _ [ _ ] 2 3 4 5 6 7
3 -2 4 0.2 6 1

```

```

      A dyadic vector/scalar
      1 2 3 4 5 6 +_ - * _ ÷ _ [ _ ] 7
8 -5 21 0.5714285714 7 6

```

```

      A monadic vector
      +_ - * _ ÷ _ [ _ ] 1 2 3 4 5 6
1 -2 1 0.25 5 6

```

And it works uniformly with mixed, defined and derived functions and with nested arguments so long as at least one argument has length equal to the number of function operands and the other, for a dyadic call, is the same length or is a scalar.

The reason it works is as for operator (*SEL_*) described previously. The control on which function is run is by the length of the argument(s). When the length is 2 then the left operand must be the function to run on the first ($\Rightarrow$) item of the argument(s). If the length is greater than 2 then the left operand must be a function derived with a further call to '*_*' so its arguments are passed as the '*-1+*' of the arguments at this calling level and a blind recursive call is made. The right operand is always the function to be run on the last ($\Rightarrow \phi$) item(s).

That's it.

VECTOR

INDEX TO BACK NUMBERS

Vols 1 - 8 inclusive

Alphabetical by Author

APRIL 1992

Acknowledgements

Index to Vol.1 and Vol.2: David Preedy

Index to Vol.3 and Vol.4: Gavin Schwarzenbach

Index to Vol.5: Adrian Smith

Index to Vol.6 to Vol.8: Gill Smith

Copyright © : The British APL Association 1992

Author(s)	Title	Vol	No	Pg
Adams, John	Appropriate Use of APL in AI (1988)	5	3	56
Adams, Martyn	Why GKS is Unsuitable ...	2	3	83
Alfonseca, Manuel	APL/PC2 with Auxiliary Processors	3	2	112
Alis, David	Through IRMA to IMS	5	1	61
Ansell, Jake	APLIM - Generalized Linear Models	2	2	63
Ansell, Jake & Sykes, Alan	Beaton's Recipe for Least Squares	8	1	36
Ansell, Jake	ASL: The Basic Statistics Volume	8	1	45
Ansell, Jake	ASL: A Basic Statistics Volume Tutorial	8	2	37
Appleton, David	Some Questions of Programming Style for the APL Statistical Library	7	1	90
Appleton, David	Stirling Numbers - a Case Study	8	2	44
Appleton, David	Curve Fitting in APL	8	3	23
Barman, Jonathan	APL and Partitioned Data	1	1	129
Barman, Jonathan	The CIPS APL Toolkit (review)	6	1	51
Barman, Jonathan	Namespace Play	7	2	138
Barman, Jonathan	Review of APL*PLUS/PC Release 10	7	3	67
Barman, Jonathan	Notes on "Panel - Is J a dialect of APL?" at APL91	8	2	76
Barman, Jonathan	Notes on "Panel - APL as an Entrepreneurial Tool" at APL91	8	2	84
Barnetson, Paul	The Charlton Chase Package	4	1	70
Bassett, Mark	APL Trivia - Loopy, Quine & Turing	3	1	124
van Batenburg & Prins	QL/APL - a leap forward or back?	2	2	115
van Batenburg, F.H.D.	How Expensive is APL?	5	3	91
van Batenburg, et al	ASWI Programming Standards	8	2	111
Biddlecombe, Peter	Error Trapping Tutorial for IPSA APL	6	3	102
Bittlesone, Robert	FGL: Fifth Generation Language	1	1	89
Bittlesone, Robert	Why APL?	1	1	65
Bittlesone, Robert	XPL - an Expert Systems Framework	1	2	65
Bittlesone, Robert	Re-inventing Man	1	4	75
Booth, Desmond	Are 4GLs killing APL?	1	2	49
Bowman, Dick	APL and Graphics Standards	1	2	75
Bowman, Dick	The Outside World	3	4	56
Bowman, Dick	Coming out of the Closet	3	4	97
Bowman, Dick	APL Trivia - Wimbock APL	3	4	111
Bowman, Dick	Review of IBM's APL2 on the PC (16-bit)	6	2	45
Brand, Pauline	Error Trapping Tutorial in Dyalog APL	6	1	100
Branson, Peter	Workspace Management	4	2	99
Branson, Peter	Managing Multi-currency Accounting	6	3	73
Branson, Peter	Full-screen Methods with APL2	7	4	126
Brennan, Jerry M	Printing APL Matrices	6	1	64
Brewster, Christine	Large Commercial Database	2	3	61
Brewster, Christine	Whitbread: Large Scale Database	2	4	81
Camacho, Anthony	Public Domain Interpreter (Proposal)	3	2	127
Camacho, Anthony	A Demonstration of Direct Defn	4	3	95
Camacho, Anthony	I-APL: History & Achievements	4	3	89
Camacho, Anthony	The Ideal Screen Editor	7	4	78
Camacho, Anthony	Notes on "Panel - a 25-year Perspective" at APL91	8	2	61
Camacho, Anthony & Goodman	APL Printing on some Star Printers with I-APL	8	3	29
Camacho, Sylvia	Sharper Focus on APL	3	1	79
Camacho, Sylvia	The Road not Travelled	3	4	102
Camacho, Sylvia	Bell's Inequality	4	1	73
Cannon, Ray	Review of the STSC Compiler	7	3	64
Cannon, Ray	Mandelbrot Sets	7	3	123
Cannon, Ray	Mice do it on the Mat	7	3	134
Cannon, Ray	Mandelbrot Sets (2)	7	4	110
Cannon, Ray	Suggested Standards for a Component File System	8	2	105
Cannon, Ray	How STSC's quad-MF can help in Testing Workspaces	8	2	135
Carmichael, Michael	Moving Data from 1-2-3 to APL	1	4	130
Carmichael, Michael	Quotient Jottings	1	3	156
Chapman, Paul	Plea for Naming Conventions	3	1	115
Chapman, Paul	Diary of an Implementer	3	4	129
Chapman, Paul	I-APL: Under the Bonnet	4	3	48
Chapman, Paul	Cross Clocks in J	8	3	124

Author(s)	Title	Vol	No	Pg
Clark, Ian	Graph Plotting in I-APL/Mac 1.2	8	4	113
van Cleave, Phil	APL Lap-sized Computer	2	2	55
Cocking, Romilly	Setting up a Company I.C.	1	1	42
Cocking, Romilly	DIF-file interface	1	4	149
Cocking, Romilly	Developing Business Systems	5	1	66
Cocking, Romilly	Interfacing APL to C	5	1	49
Cocking, Romilly	The Benefits of Function-point Analysis	5	3	79
Cooper, Tony	Computing Environments for OR	3	1	62
Crossley, David	The Importance of being Nested	2	1	48
Crossley, David	Panel Design: an APL Programmers' Toolkit	8	3	74
Deimotte, Alain	Training the Mouse (in APL*PLUS/PC)	8	2	132
Doherty, David	SCREENIO: IBM full-screen manager	1	1	121
Donnelly, Peter	Duck a la Carte	1	4	65
Donnelly, Peter	CGI Graphics In Dyalog APL	4	1	59
Donnelly, Peter	Future Plans for Dyalog APL	5	4	78
Donnelly, Peter	Programming for Events In Dyalog	8	1	107
Douglas, J.B.	Polynomial Curve Fitting	3	4	117
Eastwood, David	OL/APL - popularising APL	1	4	81
Eastwood, David	GDDM / AP128 on APL68000	4	1	59
Eastwood, David	Attitudes to APL in Higher Ed'n	5	1	34
Eastwood, David	Error Trapping Tutorial for APL68000	8	1	96
Eastwood, David	Working with Windows	8	1	97
Eastwood, David	ASL Standards	8	3	46
Evero, Olie	NED: a Nesting Editor for APL*PLUS/PC	7	4	116
Falkoff, Adin	Comments on the APL2741 Typeface	6	4	116
Fieldsend, Graham	My first date with IRMA	2	1	60
Forfar, D.O.	APL for Financial Calculations	6	3	76
Franken, Ole	Mr Babbage's Secret	1	2	61
Gay, Allan	Dissembling	4	4	103
Gay, Allan	AP127: An Interface from APL to DB2	5	1	91
Gay, Allan	Dissembling	5	2	105
Gay, Allan	Writing Assembly Language Functions for quad-NA	7	4	100
Gray, Dick	APL for the Teacher	2	1	79
Gray, Dick	Do you Dig Keywords?	2	2	61
Gray, Dick	Comparison Tolerance	3	2	131
Harvey, Christopher	PATTIE: a Practical Expert System(1)	1	3	101
Harvey, Christopher	PATTIE: a Practical Expert System(2)	1	4	98
Hawkes, Alan	Complex numbers in APL	1	1	107
Hawkes, Alan	Numeric Integration & Probability	1	2	117
Hawkes, Alan	Statistical Computing with APL	1	2	113
Hayward, Iain	APL Printing from APL68000	6	1	127
Hayward, Iain	Review of APL68000 Level II	7	4	67
Hill, Richard	Desk Top Publishing on the Cheap	8	2	130
Hui, Roger & Iverson, Ken	J Questions Answered	8	3	94
Hui, Roger	Verb Tables	8	4	91
Hultin & Hagger	Power to APL	2	1	123
Hultin, Per	An APL Banking Application	3	1	65
Iverson, Ken	The Split in APL	3	2	47
Iverson, Ken	A Commentary on APL Development	5	1	78
Iverson, Ken	J: an Informal Introduction	7	1	67
Iverson, Ken	A Dictionary of J	7	2	99
Jensen, J.R & Beatty, K.A.	An Interface between APL2 and the X Window System	8	1	125
Jizba, Zdenek	Generic Local Objects	5	3	103
Jizba, Zdenek	Problems for APL Buffs (I)	6	3	109
Jizba, Zdenek	Problems for APL Buffs (II)	6	4	117
Jizba, Zdenek	Object Oriented Programming and APL	7	3	108
Jizba, Zdenek	Problems for APL Buffs (III)	7	3	140
Jizba, Zdenek	Science Education in California	8	2	22
Jizba, Zdenek	Introducing APL to Teachers	8	3	19
Jordan, Maurice	Supporting APL in a Large Organisation	5	1	68

Author(s)	Title	Vol	No	Pg
Jordan, Maurice	Function-point Analysis at British Airways	5	3	82
Jordan, Maurice	Thoughts on . . .	7	4	119
Jordan, Maurice	Differences in Second Generation APLs	8	3	52
Kelly & Thomson	Weighted Least Squares	4	3	115
Koppel & Kropp	APL2 or LISP	2	2	97
de Kerf, Joseph	Fast Fibbing	3	4	120
de Kerf, Joseph	Logic & the Algebra of Propositions	5	4	99
de Kerf, Joseph	Punctuation in APL	6	2	123
de Kerf, Joseph	A Note on Quad and Quote-quad	6	3	122
de Kerf, Joseph	A Survey of Quad-NC	7	1	127
de Kerf, Joseph	What's Wrong with Parentheses?	7	2	125
de Kerf, Joseph	A Note on the Match Function in APL	7	4	133
Langlet, Gerard	APL "RISC Programming Style"	6	2	23
Langlet, Gerard	The Steam Hammer and the Fly	7	4	138
Langlet, Gerard	Recreation with Transcendental Numbers	8	2	25
Last, Phil	Writing Operators for Dyalog APL	8	4	119
Lenihan, Mark	Migration to APL2	2	4	123
Livingstone, David	Development of APL through Standardization	3	2	115
Llewellyn-Jones, L.	APLp	1	2	109
Lochran, Brian	APL and Relational Databases	5	3	74
Lyus, Steve	Financial Planning at Imperial	2	4	93
Mayer & Sykes	Teaching Stats in Univ. Coll. Swansea	4	3	74
Mayer, Alan	Local Variables and the State Indicator	8	2	27
Mayer, Alan	How Much Water Under the Bridge?	8	4	25
Mayer, Alan	Pass Me Another Diagonal Slice, Please!	8	4	26
McCree, Christine	Membership Survey	4	1	79
McIntyre, Donald B.	Hooks and Forks and the Teaching of Elementary Arithmetic	8	3	101
McIntyre, Donald B.	Using J with External Data	8	4	97
McLean, Bill	Review of "APL for the Maths Classroom" by Thomson	6	2	20
MacLeod, George	GSS Graphics with APL*PLUS/PC	7	4	75
Miroshnikov, Alexei	Soviet APL: a Historical Outline	7	3	100
Moffat, David	One Small Step for APL	3	1	75
Moss, Jill	Membership Questionnaire	5	3	36
Nabavi, Richard	Networking APL micros	1	1	49
Nabavi, Richard	GKS - Opportunity for APL	1	2	78
Nabavi, Richard	APL and GKS	1	4	63
Nabavi, Richard	WIMPS and Bach on the Armiga	4	1	60
Nabavi, Richard	Interfacing to the Apple Macintosh	5	1	113
Nabavi, Richard	The Legacy of the Typewriter	8	2	99
O'Hagan, Tony	Symbolic Computation and Recursion	6	3	80
O'Hagan, Tony et al	The APL Statistics Library Project	7	1	80
O'Hagan, Tony	The Genesis of ASL	7	3	36
O'Hagan, Tony	WAGS: a Graphics Specification Language for ASL	7	3	41
O'Hagan, Tony	The Genesis of ASL (2)	7	4	36
Olavi, Gosta	Menu-oriented dialogue	1	3	83
Parkhouse, Graham	Future Directions of APL	3	3	97
Parkhouse, Graham	Origins	4	3	67
Parkhouse, Graham	APL Graphics from First Principles	7	4	83
Parkhouse, Graham	Power Reduction	8	2	96
Pearson, Duncan	Review of APL*PLUS II Release 3	8	1	73
Perkins, Fred	The Global Information Centre	1	1	45
Perkins, Fred	Information Centre as Strategy	1	4	90
Perry, Tim	IBM-based APL communications	1	1	62
Phillips, Harry	Assurance Quotation Services	5	3	88
Piper, David	GDDM and AP126	2	3	109
Piper, David	Using VSAPL under TSO	2	4	127
Piper, David	Quad-WIN in APL*PLUS/PC Vn.5	3	1	119
Piper, David	Using quad-FX with Aux Processors	3	2	123
Piper, David	Command-driven Interface for BDAM & QSAM	3	3	118
Piper, David	Using quad-NA for Data Translation	3	4	123

Author(s)	Title	Vol	No	Pg
Piper, David	Using quad-NA to Eliminate WS FULL	4	4	107
Piper, David	Parallel Computing and APL	5	4	113
Piper, David	Semantic Class and Arrays of Functions	6	1	118
Piper, David	A Design Framework for APL Systems	6	4	78
Piper, David	Packaged Workspaces and their Implications for Application Design	7	2	63
Ponomarev, Victor	Integrated System for Demographic Investigations	8	1	85
Preedy, David	Graphics for Decision-makers	1	2	83
Preedy, David	Recursion Revisited	2	2	47
Preedy, David	Graphics in the Boardroom	4	1	63
Prys-Williams, Allan	Random Contingency Tables	1	3	160
Prys-Williams, Allan	IBM's Logic Algorithms in Dyalog APL	4	1	104
Prys-Williams, Allan	The Limits of Forward Chaining	5	2	88
Prys-Williams, Allan	Using APL to Front-end Existing Software	5	3	57
Pullman, Bob	Generic Read & Replace in IPSA	1	3	128
Pullman, Bob	Quick & Dirty Apportionment	3	1	109
Pullman, Bob	The Programmer as Designer ...	3	4	100
Pullman, Bob	Text Inequalities	4	1	103
Pym, John	Interfacing Viewdata & APL	1	1	59
Rigg, Malcolm	Review of APL and J: Some Benchmarks	8	3	70
Robertson, Graeme	A Graphic Vision of APL	1	1	36
Robertson, Graeme	An APL Dialogue	1	3	135
Robertson, Graeme	APL Linguistics	2	2	118
Rushton, Patrick	Sales Forecasting System	2	3	60
Scholes, John	Operators & Nested Arrays	2	1	117
Scholes, John et al	Workshop on Defined Operators	6	4	64
Scholes, John	A New Development Environment in Dyalog APL	6	4	101
Searle, John	APL in the Soviet Union	7	2	81
Searle, John	Review of Dyalog APL/X	8	1	66
Selby, David	Future Plans: APL2 for the PC	5	4	82
Small, Nicholas	Review of IBM's APL2 on the PC (32-bit)	6	2	52
Small, Nicholas	Packaged Workspaces in APL2	7	2	50
Smillie, Keith	Book Review "Programming in J" by Iverson	8	3	67
Smillie, Keith	Book Review "Arithmetic" by Iverson	8	4	87
Smith, Adrian	"Matchmakers" Simulation	1	1	79
Smith, Adrian	Imperial Group Graphics	1	2	89
Smith, Adrian	Towards a Teachable Interface	1	3	91
Smith, Adrian	Getting the Best out of GDDM	1	4	65
Smith, Adrian	VSPC: for whom the bell tolls?	2	2	69
Smith, Adrian	Modelling Fuzzy Decisions	2	2	109
Smith, Adrian	Correcting the UK APL*PLUS/PC Kbd	3	1	102
Smith, Adrian	Windows and Pop-up Menus	4	2	62
Smith, Adrian	Poking the DOS Keyboard Buffer	4	2	92
Smith, Adrian	Custom Banners in APL*PLUS/PC	4	3	103
Smith, Adrian	Icon Design for EGA and VGA	5	1	85
Smith, Adrian	Vax-Oracle to APL*PLUS/PC	5	1	119
Smith, Adrian	Reading the TSO Session Screen	5	2	83
Smith, Adrian	DOS Environment Variables	5	4	95
Smith, Adrian	Typesetting APL	6	2	140
Smith, Adrian	Reviews of Myriade (APL*PLUS/PC) & AFM for APL2/PC	6	3	51
Smith, Adrian	A Vector Construction Kit	6	3	117
Smith, Adrian	Review of APL*PLUS/PC Release 9	6	4	54
Smith, Adrian	A Hypertext Interface to the Oracle	6	4	71
Smith, Adrian	The Falkoff Dual Keyboard	6	4	92
Smith, Adrian	Review of Dyalog APL for MS DOS	7	1	49
Smith, Adrian	Review of Powertools/CUA	7	1	54
Smith, Adrian	A Partial Implementation of PostScript in APL	7	1	96
Smith, Adrian	PostScript Graphics in APL	7	3	74
Smith, Adrian & Richard	Why I like MS Windows	8	1	78
Smith, Adrian	Proportional Tables in PostScript	8	1	92
Smith, Adrian	Fast Filling with $A_{ij} \times B$	8	2	125

Author(s)	Title	Vol	No	Pg
Smith, Adrian	Exploiting VGA Colours	8	3	132
Smith, Adrian	Arrays with Style	8	3	140
Smith, Paul	From Mainframes to Smaller Machines	4	3	57
Spunde, Walter G.	Function Sampling with I-APL	7	1	20
Spunde, Walter G.	Shape, Ravel and Roll	7	4	19
Sullivan, John	Fast Fibbing	3	1	113
Sullivan, John	Fastest File Input in TSO	4	1	99
Sullivan, John	VSAM Processing in APL	5	1	104
Sullivan, John	An Undocumented APL2 AP	5	3	112
Sullivan, John	Using the GEM file selector in APL 69000	6	3	113
Sykes & Ansell	When Domino is not Sufficient	1	4	155
Sykes, Alan	Fast Fibbing	3	4	121
Sykes, Alan	A Character Scatterplot in APL	5	3	116
Sykes, Alan	Numbers and Bases	6	1	31
Sykes, Alan	From Coin-tossing to the Weather	6	3	24
Sykes, Alan	Doing Useful Things with Strings	6	4	24
Sykes, Alan	More String Manipulations	7	1	27
Sykes, Alan	Searching for Strings in Strings	7	2	27
Sykes, Alan	An APL SCRIPT Function	7	3	24
Sykes, Alan	A to B or not A to B?	7	3	27
Sykes, Alan	Execution Time	7	4	25
Sykes, Alan	The Regression Shell	7	4	41
Sykes, Alan	Let's Integrate - Using APL	8	1	22
Sykes, Alan	Labelling	8	1	30
Sykes, Alan	APL and the Birthday Problem	8	4	20
Thomson, Norman	Guide to APL2 Nested Arrays	2	1	108
Thomson, Norman	Using Operators in APL2	2	3	118
Thomson, Norman	Go Pack your Knapsack	4	2	103
Thomson, Norman	Tutorial on Error Trapping in APL2/PC	6	4	105
Thomson, Norman	Comparison Tolerance Explained	7	2	135
Tickner & Oswald	Meta-APL: an APL Pre-processor	4	2	72
Timson, Emily	Review of ZARK: an APL Tutor	7	4	62
Toop, David	PC to Mainframe Communications	5	1	70
Trenkler, Dietrich	Densities well-suited for Statistics Problems	7	2	118
Trenkler, Dietrich	Simulating Sampling Distributions	8	4	37
Trenkler, Dietrich	Implementing Cardano's Rule	8	4	43
Waters, Keith	3D Facial Animation	4	1	60
Webster, Barrie	Finished Goods Inventory Control	4	2	70
Williams, Mike	Expcheck: Intelligent Diagnostics	4	2	83
Williams, Mike	Personal Construct Analysis in APL	6	1	73
Williams R.G. & Green R.A.	A note on Comparison Tolerance	6	4	123
Williamson & Wells	External Databases	2	3	79
Wilson, Anne	Tree-processing Algorithms	4	1	92
Wilson, Derek	Sweeten your Combinations (Comp)	3	4	114
Wilson, Derek	Sweet Combinations (Result)	4	3	109
Winfield, M.J.	Expert Systems	1	3	49
Wortham, P.	Interfacing to PC Assembler (I)	4	2	107
Wortham, P.	PC Assembler (II)	4	3	119
Wynn, Stephen	Experimental Design	8	1	88
Wynn, Stephen	A Look at Residues	8	2	137
Ziemann, David	International APL standard	1	1	143
Ziemann, David	ISO Standards update	1	2	125
Ziemann, David	APL88 Competition Review	3	2	96
Ziemann, David	APL88 Standards Report	3	2	110
Ziemann, David	APL Trivia - Funny Dates	3	3	115
Ziemann, David	I-APL Technical Specification	3	3	118
Ziemann, David	Efficiency in APL	4	4	111
Ziemann, David	Error Trapping Tutorial for APL*PLUS	6	1	91
Ziemann, David	A Direct Definition Handler	6	1	110
Zenth, Allan	A Case Story about Moving an APL Application	8	2	127

APL92 Registration Form

Please return to:

Lynne C. Shaw, APL92 Registration Chair
417 Riverside Drive, Apt. 9B, New York, NY 10025, USA
FAX: (212) 662-0913. Phone: (212) 662-2406, Lynne_Shaw@mts.cc.wayne.edu

All visitors to Russia require a visa!

Membership Information: Membership number: _____

☐ ACM/SIGAPL ☐ FinnAPL ☐ SovAPL

If other people are coming with you to APL92, please see the form labelled "Non-Registrant Visa Information". No Registration will be considered binding (and no official invitation issued) until payment in full has been received.

☐ Do not use my name for commercial mailings

Name - First: _____ Middle: _____ Last: _____

Company: _____

Address: _____

City: _____ State: _____ Postal Code: _____

Country: _____ Telephone: _____

E-mail: _____ Fax: _____

Note: If you don't indicate a fax number, the invitations will be sent to you by mail. This may take 2-4 weeks, and will delay your visa processing.

Workshop Selections:

☐ Russian for APLers (non-binding selection)

☐ Applied Statistics Library

☐ Using J

☐ Modern User Interfaces

☐ Out of the Tool Box

☐ If APL is so Great,

Why Isn't Everybody Using It?

☐ Reservation changes: By checking this box, I agree to have my reservation moved from the Center to the Hotel Petersburg, if the Center is filled when my reservation is received. I understand that APL92 will bill or charge me for the difference. If I do not remit the difference in room reservations, I understand that my room reservations cannot be guaranteed. We will notify you if this change is required.

☐ Room Sharing (Check only if you are willing to share):

☐ Male

☐ Female

☐ Smoking

☐ Non-Smoking

Information Required for Visa Processing:

Birth place: _____ Birth Date: _____
Citizenship: _____ Passport No: _____

Location (City, country) of the Russian Embassy or Consulate to which you will apply
for a visa (if there is more than one in your country): _____

If you don't know your travel information right now, please send it to the same
address you used for registration, or by electronic mail using the INTERNET
address before 1 July: ankonqsms.ccas.msk.

Non-Registrant Visa Information

Please fill out for each person coming with you to APL92. Please submit this
form when you register. If you are taking more than 3 people with you, make
copies of this form, or write the information in the same format on another sheet
of paper.

Family name: (Mr./Ms.) _____
First name: _____ Middle Name: _____
Place and date of birth: _____
Citizenship: _____ Passport No: _____
Date of Arrival: _____ Time: _____ Flight/Train No: _____
Date of Departure: _____ Time: _____ Flight/Train No: _____
City of Departure: _____

Family name: (Mr./Ms.) _____
First name: _____ Middle Name: _____
Place and date of birth: _____
Citizenship: _____ Passport No: _____
Date of Arrival: _____ Time: _____ Flight/Train No: _____
Date of Departure: _____ Time: _____ Flight/Train No: _____
City of Departure: _____

Family name: (Mr./Ms.) _____
First name: _____ Middle Name: _____
Place and date of birth: _____
Citizenship: _____ Passport No: _____
Date of Arrival: _____ Time: _____ Flight/Train No: _____
Date of Departure: _____ Time: _____ Flight/Train No: _____
City of Departure: _____

BAA: Membership Application Form

Membership of the British APL Association is open to anyone interested in APL. The membership year runs from 1st May to 30th April.

Name: _____
 Address Line 1: _____
 Address Line 2: _____
 Address Line 3: _____
 Post or zip code: _____
 Country: _____
 Telephone Number: _____

Membership category (please tick box): 91/92

UK private membership	£12	☐
Overseas private membership	£20	☐
Airmail supplement (not needed for Europe)	£8	☐
Corporate membership	£100	☐
Corporate membership overseas	£155	☐
Sustaining membership	£430	☐
Non-voting student membership (UK only)	£6	☐

I authorise you to debit my Visa/Mastercard account

Number: _____ Expiry date: ____|____

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

(please tick the required option above)

Signature: _____

The British APL Association will take the completion of this form as evidence that you do not object to your membership details being held on our computer. For details of the British APL Association's exemption from the Data Protection Act please see Vector, the Journal of the BAA, Vol.8 No.4

PAYMENT

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Access or Visa number. Send the completed form to:

British APL Association, c/o Rowena Small, 8 Cardigan Road, LONDON, E3 5HU

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1991/92 Committee

Chairman:	David Eastwood 071-922 8866	MicroAPL Ltd., South Bank Technopark, 90 London Road, LONDON SE1 6LN
Secretary:	Anthony Camacho 0727-860130	2 Blenheim Rd, ST ALBANS, Herts AL1 4NR.
Treasurer:	Nicholas Small 081-980 7870	8 Cardigan Road, LONDON E3 5HU
Journal Editor:	Jonathan Barman, 048839-575	Hill Top House, East Garston, NEWBURY, Berks RG16 7HD
Activities:	Dr Peter Branson 081-848 8989	Electronic Data Systems, Stockley Park, UXBRIDGE, Middx UB11 1BQ
Education:	Dr Alan Mayer 0792-295296	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Technical:	Peter Donnelly 0256-811125	Dyadic Systems Ltd., Riverside View, Basing Road, Old Basing, BASINGSTOKE, Hants RG24 0AL
Projects:	John Searle 081-858 6811	4 Hawks Mews, Greenwich, LONDON SE10 8RA
Publicity:	Misha Jovanovic 0753-853141	99 Oxford Road, WINDSOR, Berks SL4 5DX
Recruitment:	Jill Moss 0225-462602	APL People Ltd, The Old Malthouse, Clarence St., BATH, Avon BA1 5NS
Administration:	Rowena Small 081-980 7870	8 Cardigan Road, LONDON E3 5HU

Journal Working Group

Editor:	Jonathan Barman	048839-575
Secretary:	Anthony Camacho	0727-860130
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (04393-385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (04393-385)
Support Team:	John Searle (081-858 6811), Ray Cannon (0252-874697), Sylvia Camacho, Bridget Barman, Gill Smith	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for 'A Programming Language' - an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframe, mini and micro computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost:

APL People
The Old Malthouse
Clarence St. BATH BA1 5NS
Tel: 0225 462602
Fax: 0225 422652

Dyadic Systems Ltd
Riverside Way, Basing Road
Old Basing, BASINGSTOK
Hants. RG23 0AL
Tel: 0256 611125
Fax: 0256 611180

Cocking & Drury Ltd
189 Tottenham Court Rd
LONDON W1P 9UE
Tel: 071 495 9435

Henters Ltd
1-4 Singer St
LONDON EC2A 4BQ
Tel: 071 250 1122
Fax: 071 867 9722

Impetus Ltd
Rusper, Sandy Lane
Wyke, SEVENOAKS
Kent TN15 0PD
Tel: 0732 365126

Compass R&D Ltd
15 Frederick Sanger Rd
Surrey Research Park
CULDFORD, Surrey GU2 5YD
Tel: 0438 302249
Fax: 0438 302279

HMW Trading Systems Ltd
Hamilton House
1 Temple Avenue
LONDON EC4Y 0HA
Tel: 071 358 4212
Fax: 071 358 8825

MicroAPL Ltd
South Bank Technopark
90 London Road
LONDON SE1 0LN
Tel: 071 922 8866

Peter Gyron Systems
12 Gloucester Place
LONDON W1H 9AW
Tel: 071 935 2986

Dutch APL User Group
Postbus 884 3430B Nieuwegein
Netherlands
Tel: 0374 2887



The British Computer Society, 13 Mansfield Street, London W1M 0BD